DEVELOPMENT OF AN ENERGY EFFICIENT WIRELESS AIS TRACKER WITH ENERGY HARVESTING CAPABILITIES

Master Thesis

Author:

Md. Sarfaraj Sikder

Matriculation Nr.: 537607

Microelectronics and Microsystems

Supervisors:

Prof. Dr-Ing. Ulf Kulau

Prof. Dr-Ing. Görschwin Fey

External Supervisor:

Martin Leuschner

Institute of Smart Sensors Group

Am Schwarzenberg 3, 21073 Hamburg,
Germany

November 2025

Declaration

I hereby declare that I have written this thesis independently and that I have not made use of any aid or sources other than those acknowledge in this thesis. I further declare that neither this thesis nor any other similar work has been previously submitted to any examination board.

I also certify that all information sources and literature used are indicated in the text and a list of references is given in the bibliography.

In addition, I acknowledge that I have used Wood Mackenzie's licensed artificial intelligence tools to assist in language refinement and coding assistance in accordance with ethical research and academic integrity guidelines. All intellectual contributions and interpretations are my own.

Hamburg, November 2025

Md. Sarfaraj Sikder

Acknowledgement

I would like to express my heartfelt gratitude to my supervisors, Prof. Dr.-Ing. Ulf Kulau and Prof. Dr.-Ing. Görschwin Fey, for their guidance and encouragement throughout this thesis work.

I am also grateful to Hamburg University of Technology (TUHH), Vesseltracker, a Wood Mackenzie business and David Hewson for the opportunity to pursue this thesis externally with the company. I am also thankful to Martin Leuschner for actively supervising and guiding me during the process. Additionally, I am thankful to all my colleagues who have supported and helped me with ideas for the development of the prototype.

The support of my family, especially my wife, has been a constant source of strength and motivation. I am equally thankful to my friends for their encouragement and for always being there throughout this journey.

Abstract

Maritime vessel tracking is of crucial importance for surveillance safety, cargo management, and operational efficiency. Modern, live tracking of vessels involves integrating terrestrial, satellite and roaming AIS (Automatic Identification System) data to provide accurate vessel positions and cargo information. Terrestrial AIS trackers provide the highest quality of data; however, they are limited by geographical constraints, range of antenna, and lack of accessible hosts for power and internet connectivity in remote deployments. A major challenge for deploying remote terrestrial AIS trackers is maintaining a stable power supply for continuous operation.

This thesis presents a guideline for designing an energy-efficient, wireless terrestrial AIS monitoring device with energy harvesting options, and validates the approach through a functional but not yet feature-complete prototype AIS tracker. Developed for vesseltracker.com, a Wood Mackenzie company, the system addresses critical aspects of energy efficiency through hardware, software, communication, and system-level power optimization techniques, complemented by wireless energy harvesting which will allow continuous real-time transmission of data to AWS IoT Core cloud servers. The system architecture, hardware selection, software design, and performance analysis are provided to present a framework for secure and reliable deployment in energy constrained locations. This thesis contributes to expanding standalone terrestrial AIS coverage in remote areas, enhancing the reliability of vessel tracking data.

Die Verfolgung von Seeschiffen ist für die Überwachungssicherheit, das Frachtmanagement und die betriebliche Effizienz von entscheidender Bedeutung. Die moderne Live-Verfolgung von Schiffen umfasst die Integration von terrestrischen, satellitengestützten und Roaming-AIS-Daten (Automatic Identification System), um genaue Schiffspositionen und Frachtinformationen bereitzustellen. Terrestrische AIS-Tracker liefern Daten von höchster Qualität, sind jedoch durch geografische Einschränkungen, die Reichweite der Antennen und den Mangel an zugänglichen Hosts für Strom- und Internetverbindungen in abgelegenen Einsatzgebieten begrenzt. Eine große Herausforderung beim Einsatz von terrestrischen AIS-Trackern in abgelegenen Gebieten ist die Aufrechterhaltung einer stabilen Stromversorgung für den Dauerbetrieb.

Diese Arbeit präsentiert einen Leitfaden für den Entwurf eines energieeffizienten, drahtlosen terrestrischen AIS-Überwachungsgeräts mit Energy-Harvesting-Optionen und validiert den Ansatz anhand eines funktionsfähigen, aber noch nicht vollständig ausgestatteten Prototyps eines AIS-Trackers. Das für vesseltracker.com, ein Unternehmen von Wood Mackenzie, entwickelte System berücksichtigt kritische Aspekte der Energieeffizienz durch Hardware-, Software-, Kommunikations- und System-Level-Techniken zur Energieoptimierung, ergänzt durch drahtlose Energiegewinnung, die eine kontinuierliche Echtzeitübertragung von Daten an AWS IoT Core-Cloud-Server ermöglicht. Die Systemarchitektur, die Hardwareauswahl, das Software-Design und die Leistungsanalyse werden bereitgestellt, um einen Rahmen für den sicheren und zuverlässigen Einsatz an Orten mit begrenzten Energievorräten zu schaffen. Diese Arbeit trägt dazu bei, die eigenständige terrestrische AIS-Abdeckung in abgelegenen Gebieten zu erweitern und die Zuverlässigkeit der Schiffsverfolgungsdaten zu verbessern.

Table of Contents

Declaration 2

Acknowledgement 3

Abstract 4

List of Figures 8

List of Tables 10

List of Abbreviations 11

1 Introduction 14

2 Literature Review 15

2.1 Evolution of AIS 15

2.2 Terrestrial AIS tracker available in market 16

2.3 Energy Efficiency in IoT Devices 18

2.3.1 Hardware-Level Techniques 19

2.3.2 Software and system Level Techniques 22

2.3.3 Circuit and Architecture-Level Techniques 23

2.3.4 Communication-Level Techniques 25

2.3.5 Power Management and Energy Harvesting 26

3 RaspAIS Kit 32

3.1 System Architecture Overview 32

3.2 Hardware Details 33

3.2.1 Technical Specifications 34

3.3 Software Details 36

3.4 Power Analysis of RaspAIS Kit 37

4 Design Approach to EE AIS Tracker 39

4.1 System-Level AIS tracker Device Architecture 40

4.2 Current Trends and Key Players in the Low-Power Components Market 42

5 Prototype Implementation 43

5.1 System Architecture Design 43

5.2 Hardware Selection 44

5.3 Software Design 47

5.3.1 ThreadX RTOS Threads 48

5.3.2 Thread Stacks 54

5.4 Power Analysis 58

6 AWS IoT Core Integration 64

7 Energy Harvesting System 65

8 Performance Analysis and Results 68

9 Challenges and Solutions 72

10 Conclusion & Future Work *73*
11 Appendix *74*
11.1 Determining Uncertainty of USB Multimeter **74**
11.2 How to Operate the Prototype **74**
11.3 Picoscope Graphs **80**
12 Bibliography *83*

List of Figures

Figure 1 Working Principle of IoT Device [24] 18
Figure 2 Typical power profile of IoT applications [26]. 22
Figure 3 System Architecture Overview of RaspAIS (Illustrated by Author) 32
Figure 4 Connection Diagram and Antenna Erection of RaspAIS [38] 33
Figure 5 Block Diagram RaspAIS (Illustrated by Author) 34
Figure 6 Raspberry Pi 3B+ (Front Side) [62] 34
Figure 7 RaspAIS Receiver Board (Front Side) (Illustrated by Author) 35
Figure 8 RaspAIS Receiver Board (Back Side) (Illustrated by Author) 35
Figure 9 Soracom Onyx LTE USB Modem [69] 36
Figure 10 Measuring Current of Raspberry Pi 3B+ at Run Time (Illustrated by Author) ... 37
Figure 11 Measuring Current of RaspAIS Kit at Run Time with no Antenna (Illustrated by Author) 37
Figure 12 System-Level Architecture Block Diagram (Illustrated by Author) 41
Figure 13 Data Collection System Block Diagram of the Prototype (Illustrated by Author) 43
Figure 14 System-level Architecture of Implemented Prototype (Illustrated by Author) 43
Figure 15 dAISy Featherwing AIS receiver [79] 45
Figure 16 Renesas Synergy PK- S5D9 Kit [93] 46
Figure 17 Omni-directional Half Dipole SMA Antenna (Illustrated by Author) 47
Figure 18 Overview of Threads Used in Prototype (Illustrated by Author) 48
Figure 19 Higher Level Flow Diagram of AIS Thread (Illustrated by Author) 50
Figure 20 Higher Level Flow Diagram of MQTT RX Thread (Illustrated by Author) 51
Figure 21 Higher Level Flow Diagram of MQTT Thread (Illustrated by Author) 53
Figure 22 Higher level Flow Diagram of Console Thread (Illustrated by Author) 54
Figure 23 HAL/Common Stacks (Illustrated by Author) 55
Figure 24 AIS Thread Stacks (Illustrated by Author) 55
Figure 25 Console Thread Stacks (Illustrated by Author) 56
Figure 26 MQTT Client instance of MQTT Thread Stacks (Illustrated by Author) 56
Figure 27 Cellular Framework of MQTT Thread Stacks (Illustrated by Author) 57
Figure 28 DHCP and DNS Client Module of MQTT Thread Stacks (Illustrated by Author) 57
Figure 29 SNTP Client Module of MQTT Thread Stack (Illustrated by Author) 58
Figure 30 Supporting Module of MQTT Thread Stacks (Illustrated by Author) 58
Figure 31 Measuring Current Consumption of the Board PK- S5D9 During Runtime with Current Ranger (Illustrated by Author) 59
Figure 32 Measuring Current of the Receiver when LNA is Disabled (Illustrated by Author) 60
Figure 33 Effect of Power Consumption on different run modes [115] 62
Figure 34 Power Consumption Share of RaspAIS Components (Illustrated by Author) . 68
Figure 35 Daily Energy Consumption (Wh) Graph of dAISy Featherwing Receiver (Illustrated by Author) 69
Figure 36 Daily Energy Consumption (Wh) Graph of S5D9 Chip (Illustrated by Author) . 69
Figure 37 Power Consumption Share due to Change in Vessel Traffic (Illustrated by Author) 69
Figure 38 Comparison Bar Chart of Total Energy Consumption (Wh/24) (Illustrated by Author) 70

Figure 39 Wire Connection of the Prototype (Illustrated by Author) 75
Figure 40 Tera Term COM Port Connection (Illustrated by Author) 75
Figure 41 Welcome Page (Illustrated by Author) 76
Figure 42 Help Menu (Illustrated by Author) 76
Figure 43 CWIZ Main Menu (Illustrated by Author) 76
Figure 44 Selecting Network Interface (Illustrated by Author)..... 77
Figure 45 Configuring IP Mode (Illustrated by Author)..... 77
Figure 46 Starting the Demo (Illustrated by Author)..... 78
Figure 47 Receiving AIS data in JSON format (Illustration by Author)..... 78
Figure 48 Subscribing to MQTT broker on AWS IoT Core (Illustrated by Author) 79
Figure 49 Receiving AIS data in AWS IoT Core (Illustrated by Author)..... 79
Figure 50 Publishing Message to turn LNA ON (Illustrated by Author) 79
Figure 51 LNA Status (ON) Reflected in Serial Terminal (Illustrated by Author) 80
Figure 52 Digital Current Spike Observed during Powering the Device at J31 (Illustrated by Author)..... 80
Figure 53 Digital Peak Current Measured During Data Transmission at J31 (Illustrated by Author) 81
Figure 54 Determining Digital Peak Current with Picoscope at J31 (Illustrated by Author) 81
Figure 55 Voltage Measured Across 0.01R at Input during Data Transmission (Illustrated by Author)..... 81
Figure 56 Current Peaks Calculated across 0.01R at Input during Data transmission (Illustrated by Author) 82

List of Tables

Table 1 Types of AIS Receivers 16
Table 2 T-AIS Device Application and it's Available Market Information on Power Consumption 17
Table 3 Cat-M & NB-IoT Capabilities [45] 26
Table 4 Comparison of primary, secondary and hybrid battery technology for IoT devices [53] [54], [56] 29
Table 5 Comparative Analysis of Ambient Energy Harvesting Techniques for IoT Devices [57] [60] [61] 30
Table 6 Description of the RaspAIS Hardware and Antenna Connection 33
Table 7 RaspAIS Power Calculation chart 38
Table 8 Comparison Table of Different Potential AIS Receivers 44
Table 9 Comparison Table for Potential Low Powered Microcontroller Boards 45
Table 10 Different types of High Frequency Antenna 47
Table 11 Power Consumption of the Receiver in Different States 59
Table 12 Daily Power Consumption of the Prototype for AIS Receiver (LNA Enabled) ... 60
Table 13 Daily Power Consumption of the Prototype for AIS Receiver (LNA Disabled) .. 60
Table 14 Effect in Power Consumption due to Change in Vessel Traffic (LNA Disabled) 61
Table 15 Power Consumption of S5D9 in Different Power Modes 62
Table 16 Estimated Daily Power Consumption in Production 63
Table 17 Determining Maximum and Minimum Peak Currents at J31 66
Table 18 Current Peaks Measured Across 0.01R 66
Table 19 Approximate Data Reception Rate/min of the Prototype IoT System (LNA Disabled) 71
Table 20 Pin Connection Diagram of PK-S5D9 & AIS Receiver 74
Table 21 Toggling LNA, Sleep and LED (for debugging) States via publishing MQTT messages 79

List of Abbreviations

AC	Alternating Current
AIS	Automatic Identification System
API	Application Programming Interface
APN	Access Point Name
ARM	Advanced RISC Machine
ASCII	American Standard Code for Information Interchange
BASEPRI	Base Priority Mask Register
BMS	Battery Management System
BNC	Bayonet Neill–Concelman
BLE	Bluetooth Low Energy
BSP	Board Support Package
Cat-M	Category M
CDC-ACM	Communications Device Class – Abstract Control Model
CISC	Complex Instruction Set Computer
CLI	Command Line Interface
CMOS	Complementary Metal–Oxide–Semiconductor
CoAP	Constrained Application Protocol
COM	Common
CPU	Central Processing Unit
CSI	Camera Serial Interface
DC	Direct Current
DER	Distributed Encoding Rules
DHCP	Dynamic Host Configuration Protocol
DNS	Domain Name System
DMA	Direct Memory Access
DSI	Display Serial Interface
DTLS	Datagram Transport Layer Security
DUT	Device Under Test
DVFS	Dynamic Voltage and Frequency Scaling
EMC	Electromagnetic Compatibility
FRAM	Ferroelectric Random Access Memory
GPIO	General-Purpose Input/Output
GT	Gross Tonnage
HAT	Hardware Attached on Top
HDMI	High-Definition Multimedia Interface
HOCO	High-Speed On-Chip Oscillator
HESS	Hybrid Energy Storage System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
ILP	Integer Linear Programming

IoT	Internet of Things
IP	Internet Protocol
LAN	Local Area Network
LDO	Low-Dropout Regulator
LED	Light Emitting Diode
LNA	Low Noise Amplifier
LOCO	Low-Speed On-Chip Oscillator
LoRa	Long Range
LoRaWAN	Long Range Wide Area Network
LPWAN	Low Power Wide Area Network
LTE	Long-Term Evolution
LTE-M	Long-Term Evolution for Machines
MCU	Microcontroller Unit
MPPT	Maximum Power Point Tracking
MQTT	Message Queuing Telemetry Transport
MOCO	Medium-Speed On-Chip Oscillator
NB-IoT	Narrowband Internet of Things
NMEA	National Marine Electronics Association
NVIC	Nested Vectored Interrupt Controller
OLED	Organic Light Emitting Diode
OTA	Over-the-Air
PCB	Printed Circuit Board
PEM	Privacy-Enhanced Mail
PMIC	Power Management Integrated Circuit
PMU	Power Management Unit
PoE	Power over Ethernet
PLL	Phase-Locked Loop
PRIMASK	Primary Interrupt Mask Register
PSRR	Power Supply Rejection Ratio
PV	Photovoltaic
PWM	Pulse-Width Modulation
RF	Radio Frequency
RISC	Reduced Instruction Set Computer
RTC	Real-Time Clock
RTOS	Real-Time Operating System
RX	Receive / Receiver
RXD	Receive Data
RXEV	Receive Event
SBC	Single Board Computer
SC	Supercapacitor
SCR	System Control Register
SD	Secure Digital
SDK	Software Development Kit
SDR	Software-defined Radio

SCI	Serial Communication Interface
SMA	SubMiniature Version A
SNI	Server Name Indication
SNTP	Simple Network Time Protocol
SFTP	Secure File Transfer Protocol
SoC	System on Chip
SOLAS	Safety of life at Sea
SPOT	Sub-threshold Power Optimized Technology
SRAM	Static Random Access Memory
SRPG	State Retention Power Gating
SSL	Secure Sockets Layer
SSH	Secure Shell
SSP	Synergy Software Package
TCP	Transmission Control Protocol
TENG	Triboelectric Nanogenerators
TLS	Transport Layer Security
TXD	Transmit Data
TXEV	Transmit Event Interface
UART	Universal Asynchronous Receiver/Transmitter
U.FL	Ultra Miniature Coaxial
UHF	Ultra-High Frequency
USB	Universal Serial Bus
USBFS	USB Full-Speed
USBHS	USB High-Speed
VDES	VHF Data Exchange System
VHF	Very High Frequency
VTS	Vessel Traffic Services
WEH	Wireless Energy Harvesting
WFE	Wait for Event
WFI	Wait for Interrupt
Wi-Fi	Wireless Fidelity
WIC	Wake-up Interrupt Controller

1 Introduction

Vesseltracker.com, a Wood Mackenzie company, provides a platform for partners or customers to track live traffic of waterways. The company also presents relevant information for each vessel or cargo it is tracking. Data integration is a very crucial part of the company's workflow as many partners of Vesseltracker rely on data, such as the location of the vessel, type of vessel and its historical behavior, what they are carrying etc., for their own business.

To maintain this integrity and have total control over the data collection, Vesseltracker deploys its own T-AIS or terrestrial AIS (Automatic Identification System) trackers in most of the important locations worldwide. AIS tracker includes an AIS receiver along with its processing unit. However, political and geographical barriers restrict access to some areas. Additionally, T-AIS receivers are unable to collect data of the ships that are deep into the oceans. It has its limitations defined by the physical range of the antenna used with it. Therefore, Vesseltracker also receives AIS signals from other methods, for instance satellite and roaming AIS, which help in forming the total tracking system of the maritime vessels. Vesseltracker uses these data in parallel to the terrestrial data to have supplementary layer information.

For a terrestrial AIS tracker to be active, it requires a host that provides the power and internet connectivity with which the AIS tracker can collect and send the data over to remote servers. However, it is not always possible to find a host in remote areas. In addition, maintenance is not easy in remote area. To mitigate these complexities, Vesseltracker needs a reliable, energy-efficient embedded IoT system which will collect and send the AIS data to the remote server.

The goal of this thesis is to provide a generic guideline and aid to the company for deploying standalone, low-powered devices in remote terrestrial locations, which will send real-time raw or valid AIS data to remote AWS IoT Core cloud server when that data is particularly required. Hence, it is a design guideline to a secure, low-powered embedded platform to deploy bare-bones AIS tracker, having a wireless energy harvesting (WEH) system, in remote areas. The AIS data must be real-time as it will represent the ship's live position, speed, heading and other important information that are critical for avoiding collision and vessel traffic services (VTS).

2 Literature Review

A crucial challenge for any off-grid, remote, resource-constrained, low-powered IoT device is designing and managing its power consumption. The device may include a rechargeable or non-rechargeable battery that must be capable of powering the device for a long period of time (typically a decade). This brings the focus on considering different ways of energy harvesting from the surrounding sources and storing it efficiently, giving the device energy efficiency and energy harvesting capabilities. Designing every component efficiently, of the end-to-end system is the key to a successful and functional IoT device.

Energy efficient methods and approaches must be considered in both software and hardware architectures of the IoT device. The system should be analysed and designed to ensure reliability, security and energy efficiency at the component levels. This chapter discusses the key methodologies to be considered as a guideline when designing an energy-efficient AIS tracker, along with an overview of AIS message structures and market players.

2.1 Evolution of AIS

The Automatic Identification System (AIS), initially developed in the 1990s to enhance situational awareness and collision avoidance, is a marine vessel tracking technology that automatically exchanges information between ships and shore-based monitoring stations, providing real-time situational awareness for maritime navigation and operations [1]. It helps to understand trade flows built by the network of receivers that helps monitoring and providing global coverage of fleet movement. AIS messages are sent over two VHF channels (161.975 MHz and 162.025 MHz), with line-of-sight coverage usually between 30 and 80 km, depending on environmental conditions and antenna height [2].

AIS-transmitted data packets are converted to NMEA format by the receiver, with an average message size of about 128 bytes [3]. The AIS message can be categorized in two main classes [4] and it has 27 different types of messages [5]. The different types represent different information of the ships.

Class A: Mandated for all vessels 300 GT (gross tonnage) and above engaged on international voyages as well as all passenger ships
Class B: Provides limited functionality and is intended for non-SOLAS vessels. Primarily used for vessels such as pleasure crafts

A raw Class A or B AIS NMEA message begins with "!AIVDM," (for received messages) or "!AIVDO," (for transmitted messages), followed by comma-separated fields that include message type, quantity of fragments, fragment number, a unique message identifier (for multi-sentence messages), the transmitted data payload, and a checksum, which is crucial for data integrity. The payload data is encoded using binary coded ASCII, where each ASCII character represents 6 bits of information, with an example format being,
“!AIVDM,1,1,,A,400TcdiuiT7VDR>3nIfr6>i00000,0*78” [6].

Vesseltracker is interested in receiving valid AIS messages for both classes which start with “!AIVDM”. A logic in the embedded device is much needed to be set, which will filter out any

invalid AIS message or garbage data. This will ensure that no invalid data is sent to the server and hence will save energy in the cellular communication process.

2.2 Terrestrial AIS tracker available in market

The AIS market is moderately fragmented and characterized by the convergence of conventional maritime electronics companies, satellite operators, and data analytics enterprises.

Table 1 Types of AIS Receivers

Receiver Type	Description	Pros	Cons
T-AIS	Receiver placed of land preferably near coastal areas	- Coverage is constant in the area - It receives 24/7 - It receives all messages within the antenna range	- Covered area is limited - A lot of receivers required for global tracking - Installation is difficult in some countries
S-AIS	Receiver placed on satellites	- Global coverage possible - Also works in countries where receivers cannot be installed	- Does not really work in high density areas or near the coast / in ports - Coverage is moving and limited to number of satellites and their footprints - Not all messages are received - Message processing is often delayed - Expensive
R-AIS	Receiver placed on ships. Data is sent via satellite internet	- Good coverage across main shipping lines if installed on a critical mass of ships - Also works in countries where receivers cannot be installed	- Coverage is moving and not constant - Low traffic areas are sometimes not covered - hard to install and maintain - Requires satellite internet

Table 1, shows an overall categorization of receivers used worldwide to collect AIS data. However, the focus of this section is on T-AIS receiver only. Professional-grade AIS transponder production is dominated by traditional marine electronics companies like Saab, Furuno, and Kongsberg through well-established distribution networks. Comar Systems produces AIS receivers designed for dual-channel AIS decoding with ethernet connectivity such as the SLR450Ni and SLR350Ni [7]. For low-power, community, and distributed network usage, they also manufacture Raspberry Pi–based receivers such as RSK400Ni and RSK450Ni [8]. Hybrid AIS providers such as Spire Global and ORBCOMM integrate terrestrial AIS receivers with their satellite infrastructure.

A clear categorization can be seen in the terrestrial AIS receiver market based on application:

1. Network and shore receivers such as the Comar SLR-series, Comar R400N, Kongsberg BS600/BS610 which are used for VTS and non-stop coastal monitoring
2. Professional Class-A, VDES (VHF Data Exchange System) base stations and transceivers including the Saab R6 Supreme, ComNav Voyager/Mariner series, Furuno FA-40 which are utilized in ports, VTS and defense installations
3. Cost efficient and community units like Raspberry Pi-based (raspAIS) and SDR kits facilitating distributed, volunteer, and research networks

4. Hybrid satellite–terrestrial providers such as Spire, ORBCOMM, ExactEarth, integrating spaceborne AIS with substantial proprietary ground-station infrastructure to enhance worldwide coverage and revisit characteristics

Table 2 gives an overview of the different types of T-AIS receivers and transceivers available in the market for professional and community-based maritime monitoring systems. AIS devices designed for community applications are more conscious about power consumption than base station or VTS applications.

Table 2 T-AIS Device Application and it's Available Market Information on Power Consumption

No.	Manufacturer	Model	Type	Power Supply	Power Consumption	Connectivity
1.	Comar Systems	SLR450Ni [9]	Dual-channel AIS Receiver	5V DC, 2.5A	Not specified	USB-C, Ethernet, Wi-Fi
2.	Comar Systems	SLR350Ni [10]	Dual-channel intelligent AIS Receiver	5.1V DC, >2.5A	Not specified	Ethernet, Wi-Fi, USB, HDMI
3.	Comar Systems	RSK400Ni/ RSK450Ni [11]	Raspberry Pi-based Receiver	7 - 36V DC	24W max	Ethernet / Wi-Fi / USB
4.	Comar Systems	R400N (formerly SLR350N) [12]	Network AIS Receiver (dual channel)	9V - 30V DC, 190mA at 12V	300mA at 12VDC	Ethernet (RJ-45), USB, NMEA output.
5.	Digital Yacht	AIS100Pro [13]	Compact AIS USB / NMEA receiver for PC / plotter	Not Specified	Not Specified	USB, NMEA 0183
6.	Furuno	FA-40 [14]	Commercial AIS receiver (Class A & B)	12-24 V DC, 0.3-0.2 A	Not Specified	NMEA 0183, USB
7.	AMEC/ Alltek Marine	WideLink R150GE [15]	Dual-channel AIS receiver for shore and vessel	12 / 24 V DC	1.5 – 6 W	VHF (SO-239), GPS (TNC), NMEA 0183, USB, Ethernet, Wi-Fi
8.	Garmin	AIS 800 [16]	Class B SO AIS transceiver	9.6 – 31.2 V DC	400 mA at 12 V DC / 250 mA at 24 V DC	NMEA 0183/2000, USB, VHF antenna splitter, GPS input
9.	em-trak	R300 [17]	Small, robust dual-channel AIS receiver	12 or 24V DC	150mA at 12VDC, <2W average	USB, NMEA 2000
10.	Weatherdock	raspAIS A21100 [18]	Raspberry Pi–based dual-channel AIS receiver	5 V DC (via Raspberry Pi 3B)	Low Power ~4W	LAN, WiFi, USB, HDMI, 3.5 mm audio out

11.	Wegmatt	dAISy AIS Receiver [19]	Single-channel AIS receiver (smart channel hopping)	5 V DC (via Mini-USB)	<100 mW in receive mode (<20 mA at 5V)	Mini-USB (data & power), optional TTL serial (3.3V, soldering required)
12.	Saab	R6 Supreme AIS/VDES [20]	Professional AIS/VDES base station & transponder	12-24 VDC Nominal	15W (Transponder), 5 W (CDU)	Dual IEC 61162-450 Ethernet, 4× NMEA 61162-1/2 serial, 3× ECDIS/ARPA/Pilot, GNSS input, VHF antenna
13.	Kongsberg Discovery	AIS BS610 [21]	Professional AIS Base Station for shore/VTS applications	100 – 240 VAC; Optional dual 24/48 VDC	Avg. 9W / Peak 39W	100 Mbps Ethernet, RS-232, USB 2.0, NMEA, GNSS, VHF

2.3 Energy Efficiency in IoT Devices

The Internet of Things (IoT) refers to the pervasive presence of small, interconnected, uniquely detectable devices consisting of various components, protocols, applications, and domains [22]. It is a multidisciplinary paradigm in which many of the objects that surround us are networked and connected to the Internet to provide new services and enhance overall efficiency [23]. A key feature that distinguishes embedded IoT systems from conventional embedded systems is connectivity which can be wired or wireless, and to do this, these devices must be powered. While energy consumption is less crucial when a device is powered directly from the grid, it becomes a major concern for battery-powered IoT devices, where the available energy budget is limited. Remote IoT devices mostly rely on cellular connectivity for communicating to the internet. However, the communication protocol may vary and completely depends on the designer choice.

IoT communication protocols enable devices to exchange data by using physical, network, and application layers, and include standards like MQTT, CoAP, HTTP, Bluetooth, Wi-Fi, and LoRaWAN. Protocols like TLS/SSL and DTLS provide security for data in transit. The selection of an IoT communication protocol depends on the specific requirements of an application, such as bandwidth, power consumption, data volume, and latency.

The main objective of IoT is to maximize the communication of hardware objects with the physical world and to transfer or store the data harvested by these objects into useful information without any human aid. Figure 1 represents the working principle of a standard IoT device. It consists of three elements: hardware, middleware, and presentation [24].

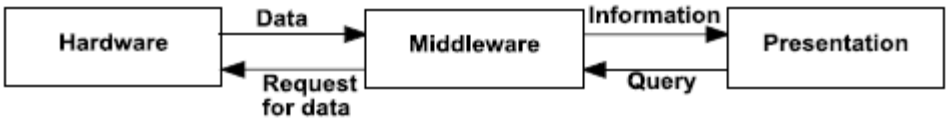

Figure 1 Working Principle of IoT Device [24]

Energy efficiency in embedded wireless systems, particularly for IoT devices, refers to the system's ability to perform sensing, computation, and communication tasks while minimizing energy consumption and maximizing the utilization of available energy, thereby prolonging device lifetime and improving overall network performance [25]. IoT devices are often battery-powered, and energy is primarily consumed during data collection and transmission. While collecting and analyzing more data can improve the accuracy of extracted information, it also increases energy use. Therefore, IoT systems must balance the quality of information with energy consumption. The availability of energy directly impacts the lifetime of resources and the effective operation of the system [24]. This section elaborates the different ways in which energy efficiency and power optimization can be ensured in IoT devices.

2.3.1 Hardware-Level Techniques

Hardware-level techniques to achieve energy efficiency operate at different levels of the hardware stack, from power management modes in microcontrollers to architectural choices in processor design. Efficient off and idle states are crucial for microcontrollers. Sleep-mode current is often the largest contributor to overall power consumption (depending on use case), and wake-up currents also impact the choice of sleep mode. Several methods can be applied to optimize these characteristics, forming the foundation for energy-aware hardware design in IoT devices.

Clock gating is a common technique to reduce dynamic power in microcontrollers by stopping the clock to inactive blocks while retaining their state. It has fine-grained control, allowing individual blocks to enter idle mode independently. Wake-up time depends on whether the clock generator remains active; if so, reactivation is nearly instantaneous, otherwise it varies with the type of clock source. Modern MCUs, such as the STM32L4, offer multiple clock sources and PLLs for frequency scaling, with core clocks turned off during sleep while peripheral clocks can remain active for data reception. The lowest power is achieved when all clocks are off, with wake-up latency determined by the selected clock source [22] [26] [27].

Voltage scaling is combined with clock gating to reduce leakage during idle mode. When the clock is stopped, the supply voltage can be lowered to the retention level of memory elements, saving power at the cost of longer wake-up times due to power supply settling. Additional leakage reduction can be achieved using reverse body biasing, which increases the threshold voltage of transistors, further lowering static power consumption [22] [27].

Power gating is used to achieve lowest leakage in microcontrollers, which turns off the main power supply while keeping a small always-on domain active. This domain typically includes an RTC for timed wakeups and minimal memory elements to retain system state. Users can select which features remain active, providing flexibility for different applications. Wake-up latency is influenced by the startup behavior of analog modules, such as voltage regulators with external capacitors, which may take milliseconds to stabilize. Advanced MCUs, like the Ambiq Apollo, achieve deep sleep currents as low as 100 nA with the RTC active [22] [27].

Dynamic Voltage and Frequency Scaling (DVFS) is a hardware technique that adjusts a microcontrollers or processor's supply voltage, and clock frequency based on workload. During low-demand or idle periods, voltage and frequency are lowered to reduce power consumption, while during high computational demand, they are increased to maintain

performance. DVFS improves energy efficiency, optimizes performance, manages temperature, and ensures system stability. It can be combined with other hardware techniques like clock gating and power gating to further reduce energy consumption. Implementation requires hardware support for voltage/frequency adjustment and software mechanisms to monitor workload and apply real-time changes [26] [27].

Power-down modes allow MCUs to further reduce energy use by entering one of several sleep states. Contemporary devices support up to seven or eight modes, each with different active peripherals and wake-up characteristics. While highly effective in reducing leakage current, programming these modes is complex, requiring verification of peripheral operation, wake-up timing, and transition speed. Selecting an MCU with the appropriate power-down capabilities ensures optimal energy efficiency while meeting application performance requirements [27].

Low-Power Microcontroller Architecture:

A widely adopted hardware-level technique for improving energy efficiency in IoT systems is the use of low-power microcontroller architectures. The performance and power consumption of the device is affected by the design of the CPU core, instruction set, and memory organization.

MCU's with Harvard architecture are called RISC (Reduced Instruction Set Computer) microcontrollers. RISC CPUs, such as the PIC18F, focus on simplicity and efficiency by using a small, fixed instruction set and executing most instructions in a single clock cycle. They employ hardwired control instead of microcode, support pipelining, and allow software to exploit concurrency. RISC CPUs are suitable for embedded applications like image processing, robotics, and instrumentation due to their low power consumption, fast instruction execution, and quick interrupt handling [28]. MCU's with vonNeumann's architecture are called CISC (Complex Instruction Set Computer) microcontrollers. CISC CPUs, such as the Motorola/Freescale HC11, have a large instruction set and many addressing modes. Most instructions can directly access memory, requiring a complex control unit and slower clock cycles [28].

While CISC programs may need fewer instructions per task, RISC can achieve higher performance through faster clocks, efficient pipelining, and compiler optimization. Overall, RISC prioritizes speed and efficiency, whereas CISC focuses on instruction versatility. Hence modern microcontroller manufacturers have widely adapted RISC architecture for production [28].

ARM Cortex-M

A notable example of processors based on RISC architecture is the Cortex-M series processors, such as Cortex-M0, Cortex-M3 and Cortex-M4., manufactured by ARM Ltd. for commercial use. One of the major benefits of these MCUs are their energy efficiency and low power characteristics. The 32-bit ARM Cortex-M4 microcontroller uses a small silicon area and has power-optimized design features, enabling energy consumption comparable to that of 8- or 16-bit devices while delivering higher computational performance. It achieves its energy efficiency through: (i) high performance, by allowing tasks to be completed quickly at lower clock frequencies causing the processor to remain in sleep states longer; (ii) high code density through the thumb instruction set, reducing memory requirements and associated power; and (iii) low-power modes that minimize active energy consumption [29].

The Cortex-M4 supports Sleep and Deep Sleep modes, which can stop clocks or partially turn off the processor, with further reduction possible via State Retention Power Gating (SRPG) in certain implementations. These modes are controlled via the System Control Register (SCR), which also provides programmable options such as Sleep-on-Exit (automatically returning to sleep after interrupt handling) and SEVONPEND (generating wake-up events on new interrupt requests). Entry into low-power states is managed through the WFI (Wait for Interrupt) and WFE (Wait for Event) instructions, while wake-up conditions are determined by interrupt priority and masking mechanisms (PRIMASK, BASEPRI). To ensure responsiveness, later revisions include a Wake-up Interrupt Controller (WIC) that enables interrupt detection even when the NVIC is inactive during Deep Sleep [29].

An additional efficiency mechanism is the event communication interface (RXEV/TXEV), which supports processor wake-up on event signaling. This feature reduces energy wasted in polling loops and enables efficient synchronization in scenarios such as DMA transfers, multi-core communication, and semaphore operations [29]. Cortex-M4 in IoT devices is, therefore, widely used in scenarios where energy efficiency, responsiveness, performance and low-cost integration are critical design requirements.

Use of Low-Power Sensors & Peripherals:

The use of low-powered sensors or peripherals, designed for energy efficiency, needs to be considered to save overall energy of the system. Mostly, the sensors or the peripheral attached to the main microcontroller is also powered from it. So, it is important to select peripherals which are smart and has low power operating modes or can be configured to run in different duty cycles. It is also crucial to be aware of the leakage power induced in CMOS devices during sleep or standby mode. A consumer doesn't have much choice in this matter, as this leakage power is directly related to the threshold voltage of the controller defined during its fabrication phase. The threshold voltage has a direct relationship with both power and performance.

The total energy consumption is the sum of the dynamic and the leakage power. The dynamic power is dominant during the operational mode, and the leakage power gets dominant over time during standby or sleep mode. As a designer, it is important to note that lower threshold voltage means faster switching of the logic gates (better performance) but higher leakage current. High threshold voltage means low leakage (better for sleep modes) but slower operation [30].

IoT devices are typically designed for spending a major portion in sleep modes, which means the leakage power gets dominated. So, peripherals with higher threshold voltage or multi-threshold voltage region are usually more energy efficient in that case. However, it highly depends on the use case. Modern ultra-low-power microcontrollers often use multi-threshold CMOS technology, which has transistors with both high and low threshold voltage. A designer can make use of that along with clock/ power gating and DVFS to save energy in individual peripheral or sensors. As a designed it is important to reach a balance to have the perfect spot to save the most energy ensuring reliable functionality.

2.3.2 Software and system Level Techniques

Energy-efficient IoT devices are built on a foundation of hardware-level solutions, but software and system-level strategies are just as important for extending battery life and enhancing system performance. These techniques function at higher abstraction layers, have control over the usage of hardware resources, and are flexible as they can often be implemented or changed without modifying the existing hardware platform. This section highlights several software and system level techniques for energy-efficiency in IoT devices.

Duty Cycling

This technique involves periodically switching off power-hungry components like radios, sensors, and CPUs when they are not actively required. MCUs in IoT devices periodically collect environmental data from sensors, process it, and wirelessly transmit it, but remain idle for most of the time. For power efficiency, the hardware should be activated or remain in the ON state only for the short time required to read, process, and transmit information, and can be switched off for the rest of the time [26]. This avoids the hardware from idle listening and continuously sensing the channel even when there is no transmission leading to significant energy consumption [31]. Figure 2 illustrates this duty cycling approach in the power consumption pattern of a microcontroller. The MCU remains in a deep sleep state for most of the time, awaiting an internal or external trigger from any sensor or timer to wake up. After capturing the event, the device enters the wake-up phase where it restarts clocks, restores the power supply, and recovers the CPU state including data and stack. The system can then fetch new data from peripherals, process it, and transmit the results. After data processing, the state is saved and the system returns to sleep mode. [26]

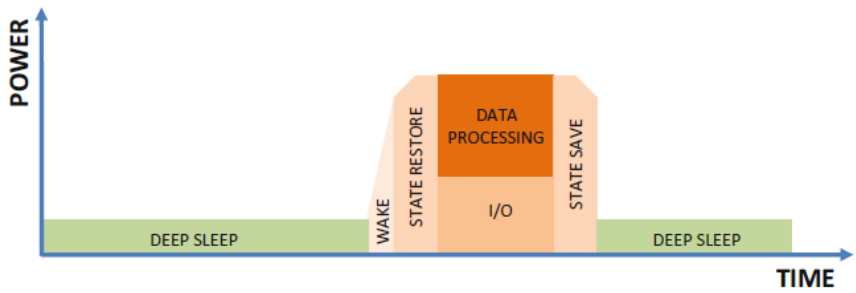

Figure 2 Typical power profile of IoT applications [26].

Lightweight Operating Systems:

These operating systems are especially designed for the small on-chip memory size of the IoT devices for complex application. They occupy a low memory footprint while efficiently utilizing hardware sleep modes to save energy and elevate battery life. Among the many types of IoT operating systems, three main approaches are talked about in [32]: Real-Time Operating Systems (RTOS), protothread-based systems like Contiki, and event-driven systems like TinyOS.

RTOS platforms, including FreeRTOS, Nucleus RTOS, TI-RTOS, ThreadX RTOS, and RIOT, provide power management techniques such as peripheral state control, sleep modes, and tick

suppression. Tick suppression, particularly, stops the periodic tick interrupt during idle time and corrects the RTOS tick count value when the tick interrupt is restarted [33]. This feature causes the system to stay in low-power states for extended periods without the overhead of periodic timer interrupts, increasing the battery lifetime by up to 35% [32]. Alternatively, Contiki offers power management via APIs with power-down functions to put the radio and CPU in low-power states. When application processes are idle, the system automatically switches into sleep mode. However, compared to complete RTOS implementations, Contiki's energy-saving capabilities are limited because it only partially supports tick suppression [34]. TinyOS, another alternative, uses split-phase and event-driven mechanisms to save power automatically by avoiding spinlocks and threads. It has a scheduling model designed to maximize the low-power CPU mode time period [35]. However, TinyOS does not provide tick suppression and instead achieves energy efficiency through its architecture rather than explicit power management APIs. Hence, RTOS platforms can be considered as a comprehensive solution for applications requiring both deterministic real-time behavior and advanced power management [32].

Energy-aware scheduling:

It is another software-level technique for power consumption management in IoT devices. The energy-aware schedulers smartly plan and prioritize when to carry out certain tasks depending on the available or harvested energy and amount of energy needed to execute them. This is done by calculating the required voltage threshold for each task based on parameters like execution time, current consumption, harvesting rate, and capacitor size. When the voltage measured at predetermined intervals stays below the threshold, the device goes into sleep mode. When the voltage reaches or surpasses the threshold and the task can be finished by the deadline, the scheduler executes the task. The scheduler optimizes the number of successful task executions based on predefined priorities, parent-child dependencies, and deadlines, while maintaining forward progress by storing task instances and output data in nonvolatile memory after each execution [36].

Firmware optimization is also an important aspect as this can save memory and can be very effective during OTA process. A small compact firmware will take less data and power to be transferred over the air and is a must for memory constrained embedded device.

2.3.3 Circuit and Architecture-Level Techniques

Low-level circuit and architecture design optimizations can significantly impact overall system power consumption as the choice of memory architecture and processing core configuration directly influences both active and standby power consumption. Two critical considerations in this part are the selection of appropriate memory technologies and the implementation of heterogeneous processing architectures that balance performance requirements with energy constraints.

Energy-efficient memory architecture:

Memory subsystems can consume up to 40% of the system power in general-purpose processors, making them a significant contributor to the overall energy consumption of IoT devices [37]. Optimizing the memory architecture is essential for increasing operational lifetime since IoT nodes typically have limited energy budgets and workloads that shift

between active computing and long idle periods. The selection of memory technology should consider balancing power consumption, access speed, data retention, write endurance, and cost. Energy-efficient IoT memory architectures consist of two key technologies, Static Random Access Memory (SRAM) and Ferroelectric Random Access Memory (FRAM), each addressing different aspects of the power–performance trade-off.

SRAM:

SRAM is a standard volatile memory with fast access times and low active power making it suitable for on-chip caches and working memory in IoT MCUs. Ideal for compute-intensive or instruction-fetch-heavy tasks, SRAM stores data in bistable latching circuits with six transistors per cell, providing access latencies below 10 ms for 13 KB and active power consumption around 60 µA/MHz [38]. Energy-efficient SRAM design, at the circuit level, aims to reduce the minimum operating voltage (V_min), decreasing leakage, and improving reliability. Reducing active and standby power can be achieved through techniques like read/write assist circuits, dynamic voltage scaling, multi-V_{DD} domains and retention gating [39]. However, SRAM's volatility causes static leakage even in idle states due to its need for constant power supply to maintain data, leading to inefficiency in IoT nodes that remain mostly in sleep or low-power modes. The structure of SRAM's six-transistor (6T) cell also limits density and scalability in compact SoCs [38]. For intermittent operation or energy-harvesting power profile scenarios, non-volatile alternatives like FRAM provide substantial benefits.

FRAM:

FRAM is a non-volatile memory technology that stores data through electric polarization in a ferroelectric material. FRAM doesn't cause standby leakage as data is stored as electrical polarization states using a ferroelectric layer that requires no power supply [38], [40]. This mechanism allows fast and low voltage (1.5 V) writes without requiring charge pumps. This results in write energy up to 250 times lower than flash and read energy about twice lower, still supporting near-infinite endurance ($>10^{15}$ cycles) and fast access (8 MHz) comparable to SRAM [40]. In FRAM-based microcontrollers, FRAM serves as both RAM and storage functioning as unified memory, enabling flexible memory mapping strategies where program code, data, and stack sections can all reside in FRAM [38]. This unified memory architecture allows task sections to be mapped to the most energy-efficient memory based on access patterns and operational characteristics.

Energy efficiency in IoT systems heavily relies on memory mapping decisions, particularly the allocation of code, data, and stack segments between volatile and non-volatile memory. SRAM mapping is more efficient for text-intensive tasks due to faster instruction fetches saving up to 30% energy, whereas FRAM mapping is more efficient for data-logging tasks due to its low write energy and non-volatility [38]. The choice between SRAM and FRAM mapping is made more difficult by data migration overhead during power management operations. In deep sleep modes, all SRAM contents must be transferred to FRAM before entering sleep and restored upon wake-up, introducing additional energy consumption [38]. Intermittently powered IoT systems research introduces the memory mapping technique called integer linear programming (ILP) which reduces migration overheads by selectively placing data in SRAM or FRAM based on access frequency and volatility [41]. ILPs can result in average energy savings up to 50% under variable power conditions [41]. Hybrid and non-volatile logic architectures like ILP that utilize both technologies, supported by dynamic memory mapping and hierarchical power

gating, show the energy-efficient circuit and architecture-level design advances for next-generation IoT microcontrollers.

Heterogeneous architectures:

Heterogeneous processing architectures incorporate multiple processing cores with different performance-power characteristics, enabling the system to dynamically allocate tasks to the most appropriate processor based on workload demands [42], [43]. Homogeneous architectures, alternatively, utilize identical cores regardless of task complexity, causing high-performance cores to execute simple tasks leading to energy waste.

A prominent implementation is the ARM big.LITTLE architecture, which includes both high-performance "big" cores and energy-efficient "LITTLE" cores within a single SoC. Big cores, such as Cortex-A15, are optimized for compute-intensive tasks like image processing, encryption and signal analysis, whereas LITTLE cores like Cortex-A7 are designed for lightweight operations like sensor data acquisition and peripheral management [42] [43]. Using this dynamic workload allocation, big.LITTLE architectures significantly lower average power consumption without sacrificing responsiveness. This is particularly useful in IoT and edge computing contexts, which require low energy use balanced with real-time performance requirements [42], [43]. Big.LITTLE systems hold high resilience according to fault-injection studies, where big cores have fault detection rates of about 94% and LITTLE cores about 90%. They are well-suited for diverse IoT workloads [42].

Heterogenous multi-core configurations allow parallel execution of sensing, computation, and communication tasks, lowering total active time and permitting quicker low-power state transitions [42]. Moreover, intelligent task mapping which assigns workloads to cores based on computational complexity, timing constraints, and energy budgets, can result in about 20% additional energy savings while maintaining real-time performance [43]. So, through task-aware resource allocation and adaptive power management, heterogeneous processing architectures minimize energy consumption significantly while maintaining performance and reliability in IoT systems [42], [43].

2.3.4 Communication-Level Techniques

Remote terrestrial AIS receivers need to be deployed worldwide along the coast or at sea, where cellular connectivity is typically the only viable communication option. The choice of cellular modem is always a critical aspect of any remote IoT device, as reliable data transmission to the server depends heavily on the underlying communication technology. In low-power systems, the cellular modem is also commonly the most energy-demanding component. To achieve energy efficiency without compromising connectivity, NB-IoT and LTE-M have become the preferred network technologies for modern IoT designs due to their low power consumption and wide coverage [44], [45].

Cellular networks can consume more battery, particularly in areas with poor signal strength. The modem needs to work harder to maintain a connection, trying reconnection loops, and thus leading to increased power consumption. In mobile devices, 4G and 5G networks can be more efficient than older technologies, like 3G. In general, cellular connections should consume more energy compared to Wi-Fi or Bluetooth connection. However, the actual consumption can vary based on signal quality, usage patterns and device settings.

On average, each AIS message is of 128 bytes [3], which is completely suitable to be handled by NB-IoT or Cat-M technology. A brief comparison between the technologies is provided below in Table 3.

Table 3 Cat-M & NB-IoT Capabilities [45]

Features	**Cat-M**	**NB-IoT**
Bandwidth	1.4 MHz	180 KHz
Latency	Low (100-150 ms)	High (1.5-10 s)
Data Rate (uplink)	Upto ~1Mbps	~200 kbps
Mobility	Full mobility	Limited (best for fixed nodes)
Power Efficiency	Slightly lower than NB-IoT	Excellent

No single provider supports worldwide NB-IoT or Cat-M technology coverage. However, it is developing and advancing with time. Mobile carriers have heavily invested in both 3G and 4G and are now focusing on 5G. Thus they have far better coverage over the world compared to NB-IoT or Cat-M. Global IoT connectivity providers like Verlos IoT [46], EMnify [47], Onomondo [48], Datablaze [49] or Truphone [50] give a detailed network coverage map for companies working with IoT devices to choose a single sim and avoid the hassle of changing carriers across the world. However, as mentioned, they do not have full coverage, and it could be tricky to make a decision on a single provider.

2.3.5 Power Management and Energy Harvesting

Effective power management is crucial for prolonging the operational lifetime of IoT devices, which can be further maximized by the utilization of harvested energy. Power management systems must efficiently regulate, store, and distribute energy to ensure continuous device operation while minimizing losses. In any energy harvesting system, two key aspects of power management involve efficient power regulators and advanced battery technologies (power storage device) to minimize energy losses and maximize its conversion.

Efficient Power Regulators:

Power regulators handle the dynamic character of energy sources and load demands by transforming and conditioning unregulated DC or AC power from energy harvesting sources or batteries into regulated voltage outputs for diverse IoT device elements [51], [52]. This regulator performance is calculated through line regulation and load regulation. Line regulation is the ratio of output voltage change to input voltage change. Load regulation is the ratio of output voltage change to load current change [51]. Additionally, power management systems should ensure safety and tackle key factors like sleep and shutdown modes with current drop to nanoamperes or microamperes to increase battery life, voltage accuracy for strict tolerance specifications for microprocessors, memory, and peripherals, and aging effects where heat dissipation raises ambient temperature, quickening semiconductor degradation over the operational time of IoT devices [52].

DC-DC converters present the most efficient power regulation for managing the dynamic current demands of IoT applications. This requires substantial conversion of voltage or power levels. These converters transfer energy with minimal loss by periodically switching power transistors to charge and discharge the energy storage elements like inductors. Through

switching converter techniques like buck (V_out <V_in), boost (V_out >V_in), and buck-boost converters (|Vout| ≷ |Vin|), and output voltage regulated by pulse-width modulation (PWM), 80-95% efficiency can be achieved due to transistors functioning as switches and not resistive elements [51], [52]. The control methodology includes voltage-mode control which uses output voltage as feedback signal; current-mode control, which uses inductor current as feedback signal; or hysteretic control where output voltage variations directly decide switching actions. Each of these have different trade-offs between transient response, stability, and implementation complexity. For solar energy harvesting applications, maximum power point tracking (MPPT) techniques are used by switching converters that optimize the system while handling variable harvested power and load current demands [51], [52]. Main limitations are controlling switching noise in the output voltage and managing efficient operation despite high fluctuation in inductance values and component degradation due to aging.

Low-Dropout (LDO) regulators provide clean, ripple-free voltage regulation for noise-sensitive circuits while managing dynamic current demands. Unlike converters, LDOs maintain constant output voltage, thus converting unstable or fluctuating voltage from energy source to stable voltage required by the load. Advantages of LDOs are switching noise absence, no bulky inductors, fast transient response from high loop bandwidth, excellent power supply rejection ratio (PSRR), and high voltage accuracy. Modern ultra-low-power LDOs achieve nanoampere to microampere quiescent currents, minimizing overhead during sleep modes [51], [52].

Power regulator selection is guided by factors like input-output voltage differential, peak-to-average load current, efficiency requirements, noise sensitivity, transient response, cost and physical constraints. DC-DC converters are suitable for high efficiency across wide voltages, loads crossing several milliwatts, and significant delivery of peak current. On the contrary, LDOs are suitable for low-dropout cases, noise-sensitive loads and fast transient response. Modern IoT systems include hybrid architectures where switching converters tackle bulk conversion with high efficiency and peak current, and LDOs final regulation with clean output for sensitive analog and RF circuits. This approach optimizes the overall system performance by ensuring high efficiency from switching conversion and low noise from LDOs and also decreases the constraints of battery-operated IoT devices such as limited energy capacity and extended operational lifetime needs in potentially inaccessible deployment locations [51] [52].

Battery Technology:

The performance of IoT devices is heavily dependent on the choice of storage type because various applications have unique energy needs, operating conditions, and lifecycle expectations. Some challenges that IoT devices, especially remote inaccessible ones, face include high self-discharge rates, inefficient power management that lead to shortened operational lifespans and higher maintenance costs incurred by frequent battery replacements. Battery performance is affected by temperature extremes, humidity, dust, ultraviolet radiation, and altitude, making them unsuitable for harsh environments. For example, the capacity of Lithium-ion batteries can decrease by 50% at -20°C, and above 45°C rapid degradation can occur. Batteries also have risks of leakage, fire, or explosion, along with sensitivity to shock, vibration, poor thermal management which leads to cycle life degradation and reduced efficiency. The usability and miniaturization of IoT devices can also be limited by the design and user-centric constraints of its power storage device. The growth of the IoT network creates new concerns with battery scalability, energy security, and integration with energy harvesting

technologies. Energy harvesting technology brings constraints in power management and charging synchronization, requiring robust hybrid power architectures [53].

There are two broad categories of modern battery technologies available for IoT devices: primary or non-rechargeable and secondary or rechargeable batteries. They each provide unique benefits and drawbacks across some performance parameters including energy density, cycle life, operating temperature range, cost, and safety [54].

Primary or non-rechargeable batteries like Lithium Manganese Dioxide (Li-MnO_2) cells are low cost and have moderate energy density and structural simplicity. These are well-suited for low-power IoT devices, like indoor sensors and smart home applications, which allow periodic maintenance or replacement. On the contrary, for remote, low-maintenance devices Lithium Thionyl Chloride (Li-$SOCl_2$) batteries are perfect due to their exceptional longevity, very low self-discharge, and excellent thermal stability [53] [54].

Secondary or rechargeable battery technologies like Nickel Manganese Cobalt (NMC), Nickel Cobalt Aluminum (NCA) are dominantly used for high-performance IoT applications like industrial automation and smart cities due to their high energy density and fast discharge rates. Lithium Iron Phosphate (LFP) batteries offer safety, thermal stability, and longevity, making them favorable for cost-sensitive and safety-critical IoT systems [53]. However, rechargeable batteries need balanced charging, temperature monitoring, and protection against over-discharge and thermal runaway, making them reliant on advanced Battery Management Systems (BMS) [54].

Supercapacitors (SCs) or ultracapacitors have increasing demand in the hybrid energy storage systems (HESS) for IoT applications. Supercapacitors have a rapid rate of charging and discharging, high power density, and long operational lifespans as they store energy electrostatically and not chemically [54]. Although supercapacitors are rarely the sole energy source for IoT devices due to their lower energy density compared to batteries, they dominate in mitigating power gaps during peak demand or intermittent energy harvesting from vibration or solar sources. SCs are ideal for short-duty IoT tasks, wireless sensor nodes, and energy harvesting circuits that require quick bursts of power or stable voltage regulation. SCs, when used in combination with batteries, also increase overall battery life, enhance power stability, and decrease chemical degradation caused by frequent cycling [54].

Additionally, the Betavolt BV100 nuclear battery [55] is a recent innovation that needs long-term, maintenance-free energy sources. The BV100 converts the radioactive decay energy of Nickel-63 into electricity using beta-voltaic technology through a semiconductor junction. This allows it to function without recharging continuously for approximately fifty years. This nuclear battery is suitable for IoT applications deployed in harsh or inaccessible environments, such as remote sensors, deep-sea monitoring systems, and defense or space technologies because of its compact size of 15×15×5 mm, durability, and extreme temperatures endurance from -60°C to 120°C. The BV100 battery's 100-microwatt, 3-volt output lowers the need for battery replacement and maintenance, sustaining low-power IoT devices indefinitely and increases reliability in critical systems. However, enhancing efficiency of energy conversion, ensuring radiation safety, and cost-management and disposal of radioactive materials are key challenges of this technology [56].

Table 4 shows a comparison of some properties of the different types of battery technologies available in the market and their typical usage in IoT application which relies heavily on the

technology's properties. Trade-offs between energy density, cost, safety, and environmental impact need to be considered while choosing the energy storage in IoT systems. Primary batteries like Li-$SOCl_2$ are non-rechargeable but have low-cost maintenance; rechargeable batteries like LFP are more sustainable but need advanced BMS infrastructure. Hybrid systems integrating both batteries and supercapacitors are becoming popular as they enable self-sustaining, energy-harvesting IoT nodes by combining the prolonged storage of batteries with the fast responsiveness of capacitors [54]. The future entails developments focused on solid-state batteries, improved BMS algorithms, and eco-friendly recycling and second-life applications to tackle e-waste and increase the sustainability of large-scale IoT applications [53].

Table 4 Comparison of primary, secondary and hybrid battery technology for IoT devices [53] [54], [56]

Battery Type	**Category**	**Energy Density (Wh/kg)**	**Self-Discharge Rate (%/month)**	**Cycle Life**	**Typical IoT Applications**
Li-MnO_2	Primary	~280	1–2	Non-rechargeable	Low-power IoT nodes, consumer devices
Li-$SOCl_2$	Primary	~500	<1	Non-rechargeable	IoT in Remote/harsh env., smart metering, industrial sensors
NiMH	Secondary	~60–120	20–30	500–1000	Basic smart home devices, low-demand IoT sensors
LFP ($LiFePO_4$)	Secondary	~100–160	2–3	2000–4000	Healthcare, smart homes, environmental monitoring
NMC ($LiNiMnCoO_2$)	Secondary	~150–220	2–3	1000–2000	Smart city, industrial IoT
NCA ($LiNiCoAlO_2$)	Secondary	~200–260	2–3	1000–2000	Smart transport, heavy-duty IoT
Supercapacitor	Hybrid	5–10	~100	>500,000	Energy-harvesting nodes, wireless sensor networks
BV100 (Betavolt)	Nuclear	Not specified ("~10× Li-ion" [55])	~0 Negligible	Non-rechargeable (~50 years continuous life)	Remote IoT nodes, deep-sea/space/defense sensors, long-lifetime infrastructure monitoring

Energy Harvesting:

Energy harvesting has emerged as a promising solution for powering IoT devices to alleviate the limitations of these battery-powered systems. Despite advances in communication protocols, operating systems, and power management techniques, batteries eventually deplete, requiring costly and often impractical maintenance or replacement, especially when sensors are deployed in large-scale or inaccessible environments [57]. Energy harvesting can be defined as the process of capturing and converting ambient energy from the surrounding environment into electrical energy that can be used to power the IoT devices. The source of the harvested energy can be categorized as ambient, mechanical, human, organic and hybrid [57]. Here, only the ambient sources will be elaborated due to their constant presence in the environment making them suitable for designing self-powered IoT devices.

Solar energy is the most abundant energy source on earth and solar cells offer the highest power density, of approximately 15 mW/cm2, as compared to various other energy harvesting techniques, making them the most common mechanism used for prolonging the lifetime of the power supply in IoT devices [58]. Despite variations due to the day-night cycle, weather, and temperature, solar energy can be predicted and paired with storage elements like rechargeable batteries or supercapacitors to ensure continuous operation. Advances in PV design now enable both outdoor and indoor use, supporting not only low-power sensors but also moderate-power applications like edge computing [57].

Radio frequency (RF) energy harvesting provides key benefits in terms of being wireless, readily available in the form of transmitted energy, low cost, and small form factor implementation. RF signals from Wi-Fi, cellular networks, radio, and satellite broadcasts (3 kHz–300 GHz), which are available 24/7, can be captured using an antenna and rectifier circuit to generate power [59]. RF energy suffers from low power density, distance-dependent performance, and conversion efficiencies of only 10–30% making them suitable for extremely low-power IoT devices or as a complementary energy source in hybrid systems [60].

Wind energy harvesting uses turbines or piezoelectric membranes to convert air movement into electrical energy. Wind energy harvesting supplies power densities varying from 0.065 to 28.5 mW/cm², making it suitable for remote sensing and environmental monitoring applications [60].

Wave energy harvesters use oscillating water columns, point absorbers, or nodding duck structures to convert ocean wave motion into electrical power, producing power outputs of 0.4 to 1 kW annually or 4 W/m³ instantaneously. Wave energy is a good choice for marine IoT applications like ocean buoys and sea-crossing bridge monitoring because of its extremely high energy density and dual functionality (energy harvesting and environmental monitoring). Wave energy harvesting, nevertheless, is only applicable to coastal and marine locations where wave action is continuously accessible, which limits its use in terrestrial IoT deployments [61].

Table 5 Comparative Analysis of Ambient Energy Harvesting Techniques for IoT Devices [57] [60] [61]

Energy Source	**Technology**	**Power Density**	**Examples of Applications**
Solar	PV cells	10 - 100 mW/cm² (outdoor) 100 µW/cm² (indoor)	Smart homes, healthcare, agriculture, environmental monitoring
RF	Antenna and rectifier circuits	0.01 – 0.1 µW/cm² 1 - 10 mW/cm² (near source)	Low-power IoT devices, healthcare monitoring, smart homes
Wind	Piezoelectric, turbines	0.065 - 28.5 mW/cm²	Remote sensing, environmental monitoring, structural health monitoring
Wave	TENG, electromagnetic, hybrid	4 W/m³ (from a multi-track independent triboelectric nanogenerator wave energy harvester)	Marine monitoring, sea-crossing bridges, ocean buoys, coastal infrastructure

Based on the comparative analysis in Table 5, solar and wind energy harvesting are the most suitable options for terrestrial IoT device deployment as they both have high power densities and widespread availability. Solar energy has a mature technology offering the highest power density, but its nighttime unavailability requires energy storage solutions. Similarly, wind energy has power densities regardless of its variability and unpredictability. Wave energy offers exceptional power density but is dependent on hybrid harvesting and restricted to marine

environments and often has a high implementation cost. RF energy is only suitable for ultra-low-power devices as it has very low power densities regardless of its availability.

3 RaspAIS Kit

The current IoT device, RaspAIS kit, is the terrestrial AIS tracker used by Vesseltracker to collect AIS data and send to its remote servers. It collects raw data and sends it to the server for further processing. The device is also equipped with a health check software running to notify that the device is alive.

A detailed description of the system architecture, hardware and software will be provided in this section. It is important to understand each component and analyse its power consumption to have a clear idea of the overall power distribution. Thus, a power analysis is conducted as a part of the thesis work to help decide on which sectors it needs optimizations to build an energy efficient AIS tracker.

3.1 System Architecture Overview

The overview of the RaspAIS is best described with a flow chart. Figure 3, graphically describes the end-end flow diagram of data collection process.

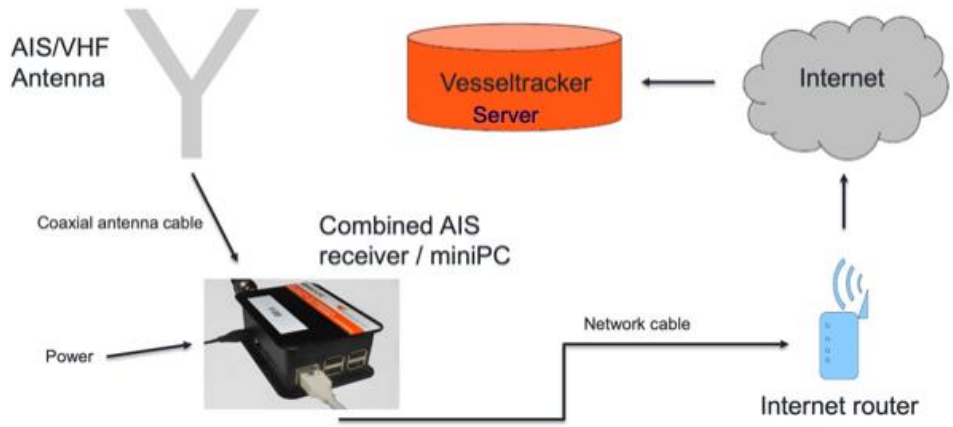

Figure 3 System Architecture Overview of RaspAIS (Illustrated by Author)

The RaspAIS receives the raw AIS message transmitted from ships or base stations at sea with the help of a VHF antenna and sends these messages to the Vesseltracker's own server over the internet. An omni-directional antenna is generally used with the frequency range of 156 - 162.5 MHz (International Maritime VHF-band). For special case, directional antennas are also used for optimizing antenna coverage range. However, due to its installation complexities it is avoided by the company for clients without technical knowledge. The IoT device creates two SSH connections to Vesseltracker's main and backup servers and forwards the data there in their designated IP and port. It also sends a health-check update to a third server for the technical team to diagnose in case of device failure. For this purpose, the host needs to allow certain IPs and ports in the router or gateway to have unlimited access to Vesseltracker servers.

The raw data is further processed in the backend and later displayed on the website for interactive live tracking of vessels.

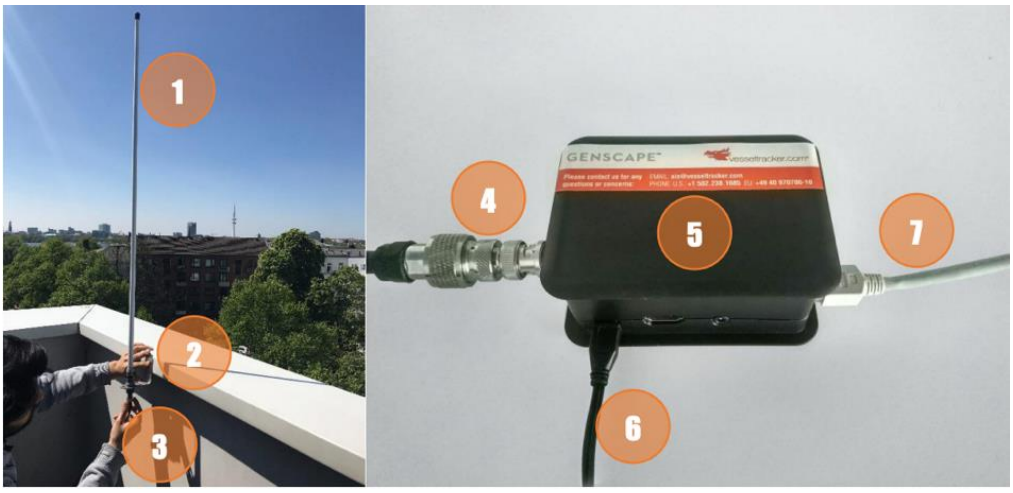

Figure 4 Connection Diagram and Antenna Erection of RaspAIS [38]

The connection diagram *Figure 4*, shows how the RaspAIS is connected to have a functioning AIS tracker system. The marked number are described below in Table 6.

Table 6 Description of the RaspAIS Hardware and Antenna Connection

No.	Description
1	Placement of antenna with an open view to the sea / river
2	Mast or Wall Mount, to hold the antenna upright
3	Connection to coaxial cable with the antenna
4	Connection of the coaxial cable with a BNC adaptor which connects to the receiver
5	RaspAIS, which contains the Raspberry Pi 3B+ and Weatherdock AIS receiver
6	Power Supply for Raspberry Pi (5V, 3A)
7	Ethernet cable for internet connection

3.2 Hardware Details

The microprocessor-based single board computer (SBC), Raspberry Pi 3B+, is the processing unit of this AIS receiver. It powers the Weatherdock AIS receiver connected as a HAT from its 5V supply pin. The receiver is connected through a BNC adaptor and coaxial cable to an antenna with a required frequency range of 161.97-162.02 MHZ to receive the AIS signal from the vessels. The device is connected to the internet via ethernet port or cellular modem. However, ethernet connectivity is the preferred choice by the company. According to the customers' preference, it is also configured to Wi-Fi which is supported in Raspberry Pi 3B+. For a host who is unable to provide internet over ethernet/Wi-Fi, a Soracom USB dongle modem is configured and provided to supply LTE connection. Figure 5 shows the hardware block diagram of the device.

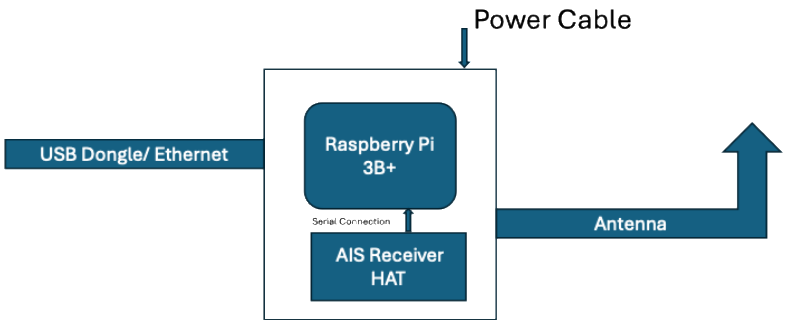

Figure 5 Block Diagram RaspAIS (Illustrated by Author)

3.2.1 Technical Specifications

A brief description for each of the hardware used will be provided in this section along with the power consumption of each component.

3.2.1.1 Raspberry Pi 3B+

Raspberry Pi 3B+ [62] is the final version of the Pi 3 range. This SBC can be setup as a desktop or used headless. It has a 1.4GHz clock speed and 64-bit quad-core processor namely Broadcom BCM2837B0. The processor is Cortex-A53, designed by ARM. It uses ARMv8 instruction set architecture, which supports 64-bit processing.

Moreover, it has dual-band wireless LAN, Bluetooth 4.2/BLE, Fast ethernet port over USB 2.0 which has a max throughput of 300 MBps. It also supports PoE, HDMI, DSI display port, CSI camera port and micro-SD port. The power supply needed for the device is 5V, 2.5A DC. For more detailed information on the product refer to [63]. The mechanical drawing, schematic diagram and detailed information on Raspberry Pi hardware can also be checked in the following references [64], [65],[66].

Figure 6 Raspberry Pi 3B+ (Front Side) [62]

3.2.1.2 Weather Dock AIS receiver

The weather dock AIS receiver is a HAT ready to be used with different models of Raspberry Pi. The RaspAIS receiver board is equipped with the AIS baseband processor CML CMX7032I9 and has a 2-channel parallel AIS receiver for best performance. The receiver is

easy to mount/install and has a programmable RX- frequency with an LED indicator. As it receives AIS data, an LED blinks and the user can read its activity. The receiver also comes with a U.FL-female connector for mounting VHF antenna. It is also designed to be powered from Raspberry without any voltage conversion [67]. According to the datasheet, this receiver communicates via UART at a baud rate of 38.4 k. Moreover, it consumes on average 200 mA [67]. The board doesn't facilitate a sleep or power saving mode. The product link can be accessed in [68].

Figure 7 RaspAIS Receiver Board (Front Side) (Illustrated by Author)

Figure 8 RaspAIS Receiver Board (Back Side) (Illustrated by Author)

Figure 7 and Figure 8 show the front and back side of the AIS receiver HAT that comes with the pins designed to fit with Raspberry Pi 3, 4 & 5 out of the box.

3.2.1.3 Soracom Onyx LTE USB Modem

RaspAIS Kit is provided with a USB modem where the host is unable to provide internet connection via ethernet or Wi-Fi. It is mostly the scenario in Japan and thus the company provides this Soracom USB modem [69] along with Soracom sim card.

Figure 9 Soracom Onyx LTE USB Modem [69]

The modem is configured to the system using a script provided by Soracom that is further modified by Vesseltrcker technical team to its need. The correct APN and password need to be provided to have a successful internet connection. The underlying modem is the Quectel EG25-G. This dongle only works on Linux operating systems, making it suitable and easy to configure on the Raspberry Pi. The device is USB powered and works with a range of 4.2 V – 6.0 V supply voltage. According to the datasheet the current consumption is 35 mA typically in idle state and 600 mA during communication. However, practically measuring the current of the device with Current Ranger during data communication shows a very fluctuating current ranging between 110-180 mA and some peak currents of 544 mA.

3.3 Software Details

The Raspberry Pi SBC is configured to have a Raspbian OS running in headless setup. The serial ports are configured to accept data over UART, and respective internet configuration is also ensured to have a connection to remote servers. A Go-lang binary code is running which ensures that the device collects the AIS data over through the serial ports and fans out to 4 different ports. They are TCP server and client, file and a web socket. Users can connect to this web socket port to see live data received by the device only. The TCP client is then responsible for creating the SSH connections and send data to the servers.

Two different SSH daemons are running in the background to ensure two secure connects: to a main server and another backup server. These servers are Vesseltracker's own and are located remotely. A health-check server is also running in the background which gathers certain information via cronjob and sends them to the third server for technical analysis of the device. The servers are responsible for further processing and storing of the data. Finally, this is also represented graphically and via a database on the website in real time for customers.

3.4 Power Analysis of RaspAIS Kit

The power graph of RaspAIS Kit was unknown to the company. Hence, a thorough analysis and measurement was needed to understand the system power consumption and to reach a decision on which part to optimise if needed.

As a part of the experiment, two ways were considered to measure the current consumption of the device. One was to use a UNI-T digital USB multi-meter [70], which can be connected between the device and the power supply to get the current reading in amps. The other method used was to measure the current with the help of a Current ranger R3 MK6 multi-meter [71]. It has a SAMD21 Cortex M0+ chip and the option to change or update firmware. The device can switch to 3 modes automatically and measure current in milli-amps, micro-amps and nano-amps. It also has the provision for connecting a digital OLED display with data logging options. The later was found suitable for measuring the components with high precision and low uncertainty percentage as it is designed with high resolution.

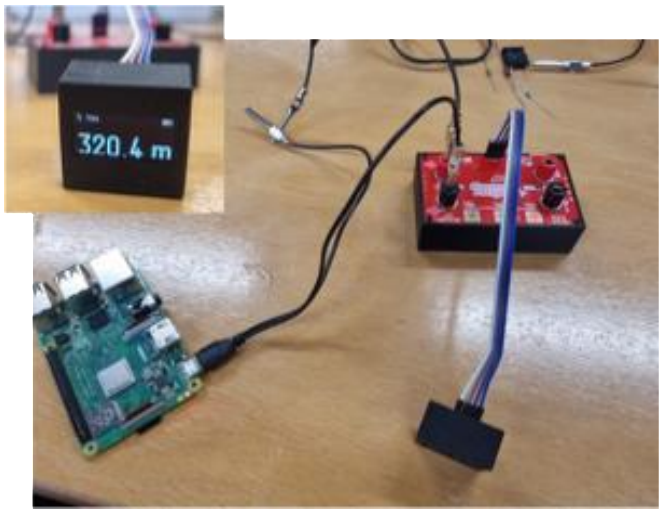

Figure 10 Measuring Current of Raspberry Pi 3B+ at Run Time (Illustrated by Author)

Figure 11 Measuring Current of RaspAIS Kit at Run Time with no Antenna (Illustrated by Author)

The average current of each component of the RaspAIS kit is calculated with the OS/code running with a whip antenna connected and measured by the Current Ranger to be represented in Table 7. Later, the total power over 24 hours is calculated from it and cross checked via USB multimeter over a week. There is a percentage of uncertainty for the USB multimeter which

need to be considered. For more details on the uncertainty calculation, please refer to appendix 11.1. Please note that the power consumption of the Soracom USB dongle is not considered in the chart below, rather the power is calculated considering the internet connectivity over ethernet port.

Table 7 RaspAIS Power Calculation chart

Part	Components	Voltage (V)	Current (mA)	Watt	WH (24h)
A	Raspberry Pi 3B+	5.10	320	1.63	39.17
B	WD receiver	5.10	233	1.19	28.52
C	Ethernet connectivity overhead	5.10	122	0.62	14.93
D	Data Processing overhead when antenna is connected	5.10	25	0.13	3.06
Total Daily Power					85.68

4 Design Approach to EE AIS Tracker

To build an energy-efficient AIS tracker, all the aspects discussed in the Literature Review needs to be considered. This chapter will discuss different approaches which can be considered to make the existing RaspAIS device more energy efficient. From the previous power consumption table, both Raspberry Pi 3B+ and Weather Dock receivers are the dominant power consuming components in the system. So, a right approach would be to optimize them first. Different techniques can be applied to approach the desired goal.

The requirement from the company is to have a secured working IoT device which is not only energy efficient but also can reliably send the collected AIS data in real time to AWS IoT Core. Currently the company is in the process of migrating to AWS IoT Core from its own servers.

Case 1:
The first case to consider could be replacing the weather dock receiver by a less power consuming board and keep the rest of the system as is. There are available receivers with sleep mode and active mode. There are also options to control LNA (Low noise amplifier) which can save further power in the cost of decreased receiver range.

Observation:
Replacing only the Weatherdock receiver including dAISy Featherwing receiver (used in the prototype) with LNA enabled can lower the total power consumption down to 62.7 Wh from 85.7 Wh, saving almost 23 Wh per day.

Case 2:
Raspberry Pi 3B+ does not support any sleep or power saving mode. So one way to save energy on this board could be to change CPU frequency [72], disable HDMI, Bluetooth and Wi-Fi or ethernet port (when not required). Moreover, the USB ports can also be powered off selectively using different available tools on Linux.

An external power management HAT (e.g., PiJuice, Sleepy Pi) or a microcontroller (Pi Zero) that can cut 5 V power and wake the Pi on schedule or via GPIO can also be considered.

Observation:
The percentage of power saved, with a combination of Case 1, should be quite high. However, this percentage is not as high as modern low-powered devices. Also, using a power management HAT to put the SBC to sleep means that there will be a delay in sending the data to the server. The tradeoff will be to lose some data or send data at intervals.

Case 3:
Considering the requirement from the company, there is no need to use a Raspberry Pi 3B+ as it is designed for heavier workload and there is no actual need of running a Linux system as it adds a considerable power overhead. One idea could be to have a low-powered Raspberry Pi Pico as the main microcontroller along with a low-powered AIS receiver. It doesn't support ethernet connectivity by default. Only the Wi-Fi is supported by Pico-W. From an experiment done by a customer, it is seen that Pico-W can consume around 52 mA while using Wi-Fi [73]. Calculating this over a period of 24 hours gives 6.24 Wh. Thus, by changing only the processing unit the power consumption can be lowered by 32.93 Wh.

Observation:
This can save a great deal of power when considered along with low-powered AIS receiver where Wi-Fi can be used.

Case 4:
A two-step system can also be considered in design. The first part of the system will be responsible for receiving and storing AIS message locally in the receiver. A second part of the system will wake up the main processor or microcontroller via an interrupt or scheduler and send the data to the server. The data can be further processed and optimized before being transmitted to the server. As the task is complete, it will go back to sleep again. For example, a Pi Zero can be used as the main processor. The current consumption of Pi Zero is between 100-120 mA [74].

Therefore, for sending the data, the microcontroller will wake up, set up internet connectivity, send data to AWS IoT Core (4-5 s) and then sleep the rest of the time (56 s). Hence, considering this example, the duty cycle of the main controller will be 6.67%.

Observation:
This method will save a lot of power. However, it will not fulfill the requirement of real-time data by the company.

Case 5:
There are some low-traffic zones, where it is rare to receive an AIS signal. A software logic can be developed which will check and make the microcontroller sleep if no ships are detected in 5 mins. It can, for example, put the whole system to sleep for a fixed amount of time and wake up again in decided intervals to check AIS signals.

Observation:
This approach will certainly save most of the power. However, there is a risk of losing AIS data during the sleep time. The power consumption will vary depending on the active state of the microcontroller.

Case 6:
The most energy-efficient design would be to consider an ultra-low-powered microcontroller with sleep, deep sleep and snooze modes available. It can be configured to run in low clock frequency as well as saving more power. In addition, choosing an energy-efficient AIS receiver with the option to configure power-saving modes will further lower the power consumption.

Observation:
This is the best approach to reduce power consumption. However, the decision relies on the company for choosing different parameters here. The prototype is built on this concept and discussed in detail in Chapter 5.

4.1 System-Level AIS tracker Device Architecture

The system-level architecture for designing an energy-efficient AIS tracker is defined by its requirement, location and use case. The company requires a self-powered, secure, reliable and remote AIS tracker which can send data directly to AWS IoT core. It is meant to be designed for remote, world-wide deployment.

As the device is designed for remote location, the internet connectivity is narrowed down to cellular connectivity. However, the cellular coverage over the world is not the same and there is no lone provider who can ensure NB-IoT or 4G LTE connectivity all over the globe. 4G LTE /CAT-1 or 4 are widely available over the globe as opposed to other technologies but they also consume slightly higher power compared to LTE-M or NB-IoT. Selection of cellular modem, thus, depends on the connectivity technology, power and financial budget. It is discussed in depth in Chapter 2.3.4.

It is often observed that cellular connectivity of a device is the most power consuming component in a low-powered IoT device. The power can be saved by reducing the overall amount of radio signals that the device must transmit and receive. Less data means less time the cellular radio is active, and the device can use lower transmission power for shorter bursts, decreasing battery consumption compared to sending unnecessary or redundant information. Thus, a logic to send only valid data to the servers and a logic to compress the AIS message before sending can be used to save energy.

For a device to be self-powered, it must be low powered and needs a system for energy harvesting capability to increase battery life. The terrestrial AIS devices usually need to be located near the coastal areas for gaining max coverage range of the sea traffic by its antenna. However, it can also be in the middle of the sea if a waterproof system is provided. So, depending on the location of the device, an energy-harvesting system needs to be determined, obtaining the max energy density. The most common, cost-efficient and feasible energy harvesting system available in the market is the solar harvesting system. It is shown in the block diagram below. However, hybrid, wind or wave energy can be considered depending on the location of the device. Different green energy harvesting systems and their characteristics have been discussed in Chapter 2.3.5.

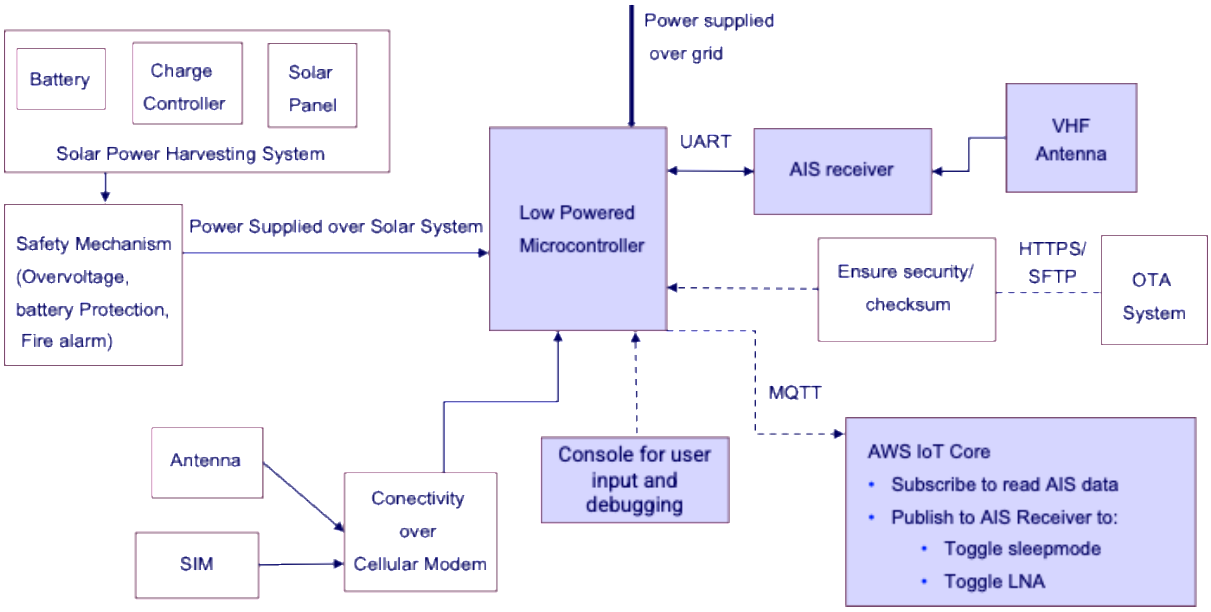

Figure 12 System-Level Architecture Block Diagram (Illustrated by Author)

Figure 12, gives a generic block diagram for the design of an energy-efficient AIS tracking device based on the Case-6, as discussed. It shows that a low-powered microcontroller can be connected to an AIS receiver over UART, which is further connected to a very high frequency

antenna to receive AIS messages. The device needs to be connected to the internet for sending data to the AWS IoT core. Thus, a TCP/IP stack is required to be implemented in the microcontroller with TLS (required for connecting to AWS IoT Core). Some cellular modems also come with their own TCP/IP stack.

A console terminal is also required for the user to set up the credentials needed to connect to AWS IoT core, allowing the device to connect to different endpoints using different certificates, without changing the code. Connection to AWS IoT needs an endpoint, a unique private key and root certificate for device connection. Therefore, the console will take input from the user and provide flexibility. Otherwise, the credentials need to be hard coded in each IoT device. The console can be also used for debugging purpose and setting up network preferences.

To ensure proper firmware and security updates, an OTA system needs to be considered in the design as well. For an OTA system, it is more common to use HTTPS or SFTP [75] to transfer files greater than 512 KB rather than via MQTT. MQTT is not designed to transfer large data files, and HTTPS allows to resume interrupted file transfer, making HTTPS a preferred method in OTA process. AWS IoT core provides services for OTA transfer over both MQTT and HTTPS. If HTTPS is chosen over MQTT, one disadvantage might be that it will require a separate TCP connection with a different port in addition to the TCP connection to the AWS IoT Core MQTT broker. Bootloader option needs to be considered when designing the OTA system and proper space in the memory needs to be kept free for the firmware download. A checksum is also very crucial to ensure the firmware download is unaltered.

Designing the OTA system can be critical and depends on the designer's preference. A common practice is to check the size of the program code and keep the same amount of memory free for the new firmware to be downloaded over the air [76]. The system can select the new firmware at boot via bootloader.

4.2 Current Trends and Key Players in the Low-Power Components Market

The market for low-powered IoT components is growing fast, with current market analyses showing multi-billion-dollar rise in the usage of ultra-low-power microcontrollers and LPWAN components as everlasting, maintenance-free sensors become more widely used in smart city and industrial settings [77]. Similarly, there is a growing trend of radio technologies, such as LoRa/LoRaWAN, continuing due to them combining long range with low duty cycles, whereas specialist vendors like Ambiq seek process-level gains with subthreshold operation or SPOT technology [78] to push sleep currents into the nanoampere range. The focus has shifted from single-chip benchmarks to system-level energy and lifetime metrics, due to the competition between prominent semiconductor vendors such as Nordic Semiconductor, TI, STMicroelectronics, NXP, Renesas, Semtech, and others over a stack of MCU, radio, and PMIC (Power Management Integrated Circuit) solutions.

Most semiconductor suppliers offer their own series of low-powered chips and IDE for programing and debugging, each with a unique user interface and set of features which allows users to decide depending on the comfort, technical support and use case.

5 Prototype Implementation

The power analysis of RaspAIS kit clearly shows that the main power-hungry components are its SBC processor and the AIS receiver itself. So, the prototype is aimed at showing how the energy consumption can be reduced just by choosing the right hardware and applying energy-saving techniques to it based on the approach discussed in Case-6 of Chapter 0. The overall data collection system flow is described in the block diagram Figure 13. This chapter will discuss the hardware selection process and software development details followed by a detailed power analysis of the prototype device.

Figure 13 Data Collection System Block Diagram of the Prototype (Illustrated by Author)

5.1 System Architecture Design

The prototype focuses on the basic functionality of an energy-efficient AIS tracker. It collects AIS signals, checks if the message is valid and then sends it to AWS IoT Core. The RaspAIS kit does not have a logic to check if the received AIS message is valid or not. It sends all the message received directly to the server for further processing and parsing. However, in the prototype a function logic is developed to check if the message received is valid before sending it to AWS IoT Core thus reducing data packets to save power. The prototype uses ethernet as the physical layer instead of cellular modem for internet connectivity. Cellular connectivity is kept for further work-scope, although the framework has already been implemented in the prototype.

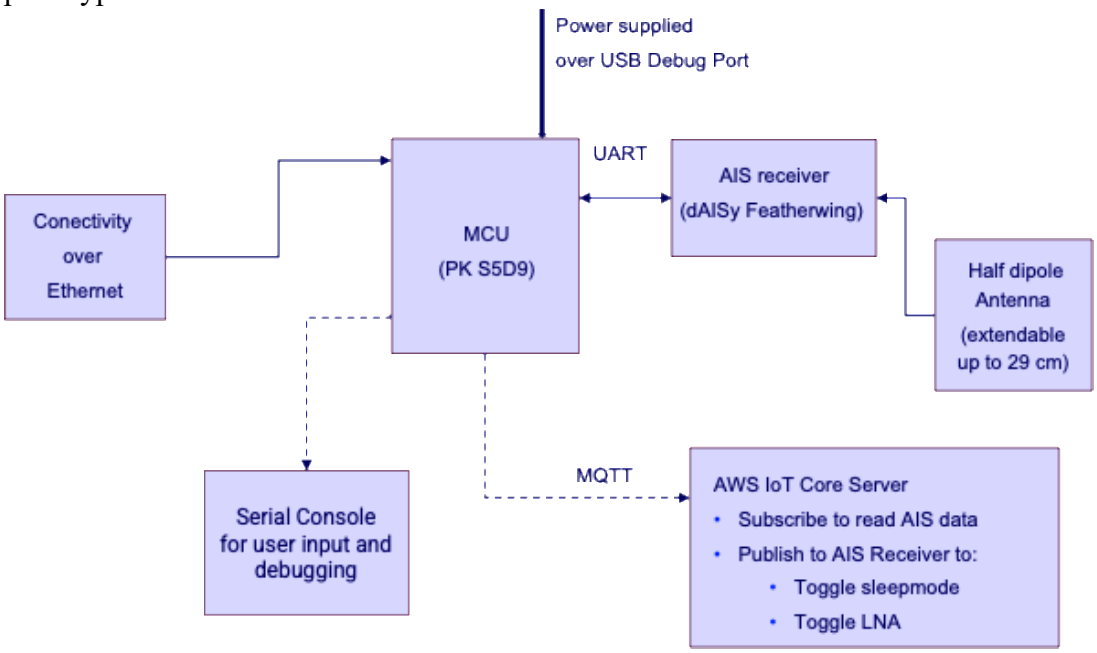

Figure 14 System-level Architecture of Implemented Prototype (Illustrated by Author)

Figure 14, shows that the prototype is powered via USB debug port of the microcontroller, and it is communicating with the AWS server via ethernet. The MCU is connected to an AIS receiver and communicating via UART protocol. The receiver is again connected to a half dipole antenna for test purpose and portability. The prototype also has serial console support and logic to control sleep mode and LNA of the AIS receiver.

5.2 Hardware Selection

With the energy-efficient design in mind, an analysis on the suitable low-powered MCU/ dev kit and AIS receiver available in market is done to build a prototype project. This section talks about the different products available in market to be readily used for developing the project. The selected hardware is used only for demonstration and not production. Nevertheless, the microcontroller chip can be used for PCB design later.

Table 8 Comparison Table of Different Potential AIS Receivers

No.	Boards	Data Sheet/ Product link	Power Consumption	Input Power + Serial interface	AIS Channel	Notes
1.	dAISy FeatherWing	Product Page [79] Manual [80]	Low power: 70 mA w/ LNA enabled, 35 mA w/ LNA disabled, 0.35 mA sleep mode	3.3V power & logic, I2C/ UART interface for AIS output	Dual channel AIS receiver with two independent radios	Sleep Mode available, Integrated with Low noise amplifier
2.	dAISy Mini AIS Receiver	Product Page [81] Manual [82]	Low power: 70 mA w/ LNA enabled, 36 mA w/ LNA disabled, 0.4 mA sleep mode			
3.	AMEC CYPHO150 S AIS Receiver	Product Page [83]	Supply Voltage: 12 / 24 V DC. Power consumption: < 1.5 Watt.	USB Type A	Two parallel AIS receivers in one box	Not suitable choice for ultra-low powered micro-controllers
4.	AIS100 Receiver	Product Page [84]	No information about power provided. Powered via USB interface	USB Type A	Dual-channel AIS receiver for use with AIS compatible PC navigation software	Not suitable choice for ultra-low powered micro-controllers
5.	RTL-SDR (software defined radio)	Product Page [85] Data sheet [86]	Powered by 5V USB interface	USB Type A	Works as a receiver along with software like AIS-catcher or rtl_ais	Used as versatile and inexpensive radio receiver

Table 8 describes different AIS receiver that can be used with a dev board to create the prototype. The receivers have both USB and serial interface. However, the most common type of receiver comes with USB interface. USB connectivity consumes more power than serial

interface like I2C or UART. Also, USB connectivity it is more vulnerable to EMC issue. Therefore, dAISy Featherwing receiver is selected as it supports UART interface and, most importantly, it can be switched between active and sleep mode. It also supports toggling its on-board Low Noise Amplifier (LNA), which can further reduce its current consumption by half. The LNA is toggled by an IO pin which is set to Input, pull-down state. Thus, supplying low input keeps the LNA turned off (by default) and giving a high input turns the LNA on.

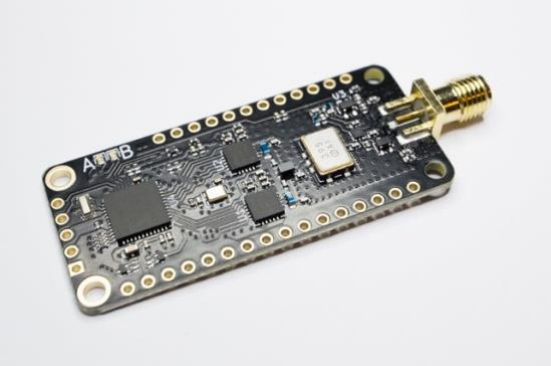

Figure 15 dAISy Featherwing AIS receiver [79]

Table 9 Comparison Table for Potential Low Powered Microcontroller Boards

No.	Boards	Data Sheet/ Product link	Processor and Features	Notes
1	B-L462E-CELL1 (stm32l462RE)	Product Page [87] Data Brief [88]	Arm Cortex-M4 core with 512 Kbytes of Flash memory and 160 Kbytes of RAM, 64 Mbytes of onboard Quad-SPI Flash memory	On-board ST-LINK/V2-1 debugger, Comes with an eSIM (ST4SIM-200M), Ultra Lower powered microcontroller
2	B-L475E-IOT01A	Product Page [89]	Arm Cortex-M4 core with 1 Mbyte of Flash memory and 128 Kbytes of SRAM, 64-Mbit Quad-SPI Flash memory	On-board ST-LINK/V2-1 debugger, Designed for low powered IoT applications And has no integrated eSIM
3	NUCLEO-L476RG	Product Page [90] Data Brief [91]	Arm Cortex M4, 32.768 kHz crystal oscillator	Ultra low power
4	ESP32	Datasheet [92]	Dual core Tensilica Xtensa LX6, 448 KB ROM, 520 KB SRAM, 16 KB SRAM in RTC and QSPI supports multiple flash/SRAM chips	Power consumption in Deep-sleep mode is 10 µA, Has Five power mode: Active, Modem-sleep, Light-sleep, Deep-sleep, Hibernation
5	Renesas PK-SD59	Product Page [93]	Arm Cortex-M4 core, 2MB Code Flash, 64KB Data Flash, 640 KB SRAM	Supports low power modes and comes with on-board debugger. Company is already familiar with Renesas platform
6	MSP-EXP430FR2355	Product Page [94]	16-bit, RISC-based, mixed-signal processors	Designed with energy efficiency as priority
7	LP-MSPM0L1306	Product Page [95]	Amr cortex M0+	Designed with energy efficiency as priority

Table 9, shows a list of low-powered microcontroller boards that comes with sufficient memory, different operating power modes and changeable clock frequency. Almost all the MCUs have an ARM Cortex processor and thus an energy-efficient architecture. This market analysis was done to check the probable options available. However, PK- S5D9 has been chosen for the prototype development as the company already has engineers working on the Renesas platform and have already integrated the interface. So, it made sense to stick to the same platform for future maintenance of the device in the company.

Figure 16 Renesas Synergy PK- S5D9 Kit [93]

Renesas has an excellent customer support and provides different application projects along with codes to use as a starting point for developing one's own prototype. Additionally, S5D9 is designed to prioritize energy efficiency. The chip supports four modes of operation namely, normal, sleep, software standby and deep software standby. It also has the option to run in multiple clock modes like LOCO (low-speed on-chip oscillator), MOCO (Middle-speed on-chip oscillator), HOCO (High-speed on-chip oscillator) or PLL (Phase-Locked Loop) [96], ranging from 32.78 kHz – 120 MHz. With the help of PLL logic circuit it can be configure to 240 MHz clock frequency. Thus, it provides the flexibility to be designed for running the program in different clock speed, ultimately saving energy when applied with duty cycles.

Another important hardware component is the antenna selection. This is the passive component of the IoT device and, therefore, does not consume power itself. However, the effectiveness of the antenna has a slight contribution in the overall energy consumption. For example, an antenna with a better range has a higher probability of receiving AIS data and thus has a direct relation with the energy consumption. AIS data is transmitted at frequency 161.975 MHz and 162.025 MHz. Thus, any antenna covering this frequency range will effectively receive AIS data. Most of the AIS receivers now come with a bandpass filter that filters out any signal outside the AIS frequency band, allowing users to choose any antenna they prefer. However, there are varies suitable antennas tuned for AIS frequency in the market. Another aspect to be aware of is the type of antenna used, that is, omnidirectional or directional. This plays a critical role in positioning the antenna to obtain the highest data reception. Additionally, antenna connectors also need to match the receiver, or else, an adaptor might be needed for connection. It is worth mentioning that any adaptor or connector in the cable will add data loss to some extent.

Table 10 Different types of High Frequency Antenna

No.	Antenna Name	Product link	Specification	Notes
1	Scan-Antenna VHF23 (Whip Antenna)	Product Page [97]	Frequency: 156-162.5 MHz, 3dB gain, Full 1/2 λ dipole, Omni-directional Connector: UHF-female, length – 0.95m	Used in the company as a standard antenna
2	Extendable SMA Antenna	Part of tinySA spectrum Analyzer [98]	Half dipole omnidirectional antenna- extendable up to 29 cm. Cover a wide range of frequency	Comes with the SMA connector, readily connects to the dAISy receiver
3	Sirio Antenna WY 155-2N	Product Page [99]	Directional, 3.05dBd- 5.2dBi gain, Connector: N-female, Frequency: 155-175MHz	Installation is difficult
4	Sirio Antenna GP- 3E	Product Page [100]	5/8 λ ground plane, Omnidirectional, Frequency: 135-175 MHz, Connector: UHF-female	Installation is difficult
5	Scan-Antenna VHF74	Product Page [101]	Frequency: 156-162.5 MHz, 4dB gain, Full 1/2 λ dipole, Omni-directional Connector: UHF-female, Full 1/2 λ dipole - no ground plane needed, length- 1.36m	Shipment is critical

Figure 17 Omni-directional Half Dipole SMA Antenna (Illustrated by Author)

The prototype has been tested with Scan-Antenna VHF23 and VHF74 at the Vesseltracker Hamburg office. A short adaptor which converts SMA to UHF was needed for this. However, for the daily debugging and code development, an extendable SMA antenna is used, which connects to the AIS receiver without any adaptor for feasibility (shown in Figure 17).

5.3 Software Design

The code has been developed in C language with the E2Studio (version 2025-01) environment available for Renesas MCUs. Renesas provides a Synergy Software package (SSP) for the easy development of prototypes using API call [102]. Moreover, SSP is fully integrated with ThreadX RTOS. In the project, SSP version 2.7.0 and Board support package (BSP) for the S5D9 board is used. To compile, build and link, E2Studio uses the GCC ARM Embedded toolchain.

In this project, the ThreadX RTOS was selected instead of a bare-metal approach to provide a more structured, efficient, and scalable software design. ThreadX offers deterministic real-time scheduling with minimal context-switch overhead, ensuring that time-critical tasks such as MQTT communication are executed reliably and without delay. Its lightweight footprint and integration within the SSP make it well-suited for embedded IoT applications on the S5D9 microcontroller. By leveraging ThreadX, the system can run multiple concurrent threads while maintaining synchronization, memory protection, and inter-thread communication through built-in mechanisms like semaphores, mutex and message queues. In contrast, a bare-metal implementation would require manual task scheduling and synchronization, increasing complexity and the likelihood of timing-related bugs, making it difficult to integrate AWS IoT Core SDK to the project. Overall, using ThreadX enhances modularity, simplifies maintenance, and improves system reliability, which are all essential for developing robust and scalable IoT solutions connected to AWS IoT Core.

The project has been developed on top of the application projects demonstrating AWS cloud connectivity provided by Renesas platform [103].

5.3.1 ThreadX RTOS Threads

In this section, the threads used in the project will be discussed. The project uses four threads which runs in parallel to execute the data sensing, processing, handling user input and communicating with AWS IoT Core. An overview of the threads and their principal responsibilities are described by the blocks in Figure 18. Along with the described threads, there is also a HAL/common thread which is activated by default by the SSP configurator to support different module and drivers.

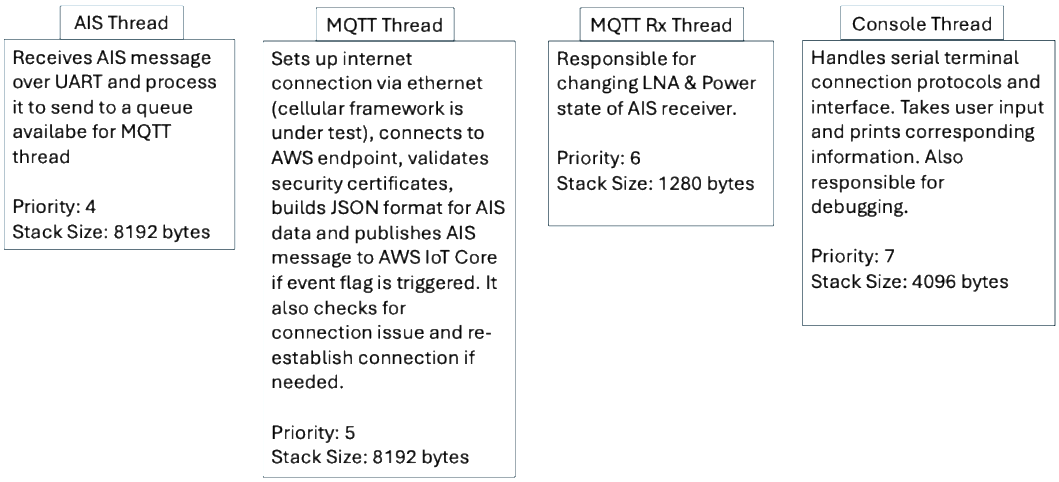

Figure 18 Overview of Threads Used in Prototype (Illustrated by Author)

5.3.1.1 AIS Thread

The AIS Thread is responsible for managing the reception, queueing, and processing of AIS messages received through a UART interface. The baud rate for the receiver is 38400, data bits is 8 and parity is set to none, with a single stop bit. This is set in the properties section under the UART driver in the configurator file. The SCI channel 2 is also selected for the data

transmission. The pins for SCI are set to P302 for TXD and P301 for RXD. The section 5.3.2 provides further information on the stack configuration. Its operation begins with thread initialization, where debug-monitor handles are configured to enable detailed logging for optional debugging and performance analysis. The thread waits for a semaphore (g_AIS_uart_semaphore) signaled from MQTT Thread to proceed. The UART interface is then opened to establish communication with external AIS receiver, and a message queue (uart_queue) is created to temporarily store pointers to received AIS message buffers. Once initialization is complete, the thread initiates UART receive operation to start listening to the incoming AIS data.

When new data is received, the UART callback function is triggered to handle the completed receive event. The received message buffer is sent to the uart_queue for processing, and the buffer index is advanced in a circular manner to efficiently manage memory usage. The UART receive operation is then re-armed to ensure continuous data reception without interruption.

The main thread loop runs indefinitely, continuously waiting for new AIS message pointers from the uart_queue. Upon receiving a message, the thread invokes the process_received_data function to interpret and handle the data. This function validates incoming AIS data, checking for correctly formatted messages (e.g., those beginning with "!AIVDM"). Valid messages are stored in a circular buffer (g_ais_msg_buffers) and queued for the MQTT thread via the g_ais_mqtt_queue, which facilitates communication with other threads. The MQTT thread is then notified through the SYS_EV_AIS_UPDATE event flag, enabling real-time message propagation. Invalid or incomplete messages are managed gracefully by resetting or realigning the buffer to the next valid AIS frame, ensuring data integrity. This method ensures an energy and resource efficiency by avoiding invalid data transmission. A small delay of 5 ticks (50 ms) is applied for allowing other low priority threads to complete their tasks.

Error handling mechanisms are integrated to maintain system reliability. UART read failures are logged for debugging (optional), queue overflow conditions trigger the discarding of the oldest messages to free space, and buffer overflows are prevented by resetting the buffer before capacity is exceeded. Buffer or queue overflow might occur if there is a high rate of data reception in very a short period, giving no time for handling the data to MQTT thread. Long, invalid message strings are discarded to keep the system synchronized with valid data.
This entire process operates in a continuous loop, allowing the AIS thread to receive, validate, and forward AIS data efficiently and reliably. Through this design, the thread ensures seamless communication and consistent data flow, supporting the overall stability. Figure 19, gives a high-level workflow of the thread.

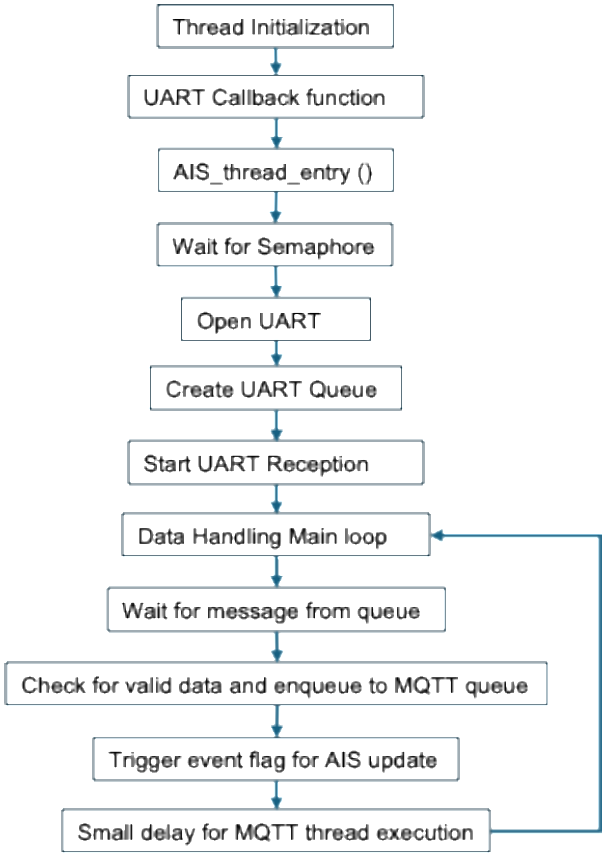

Figure 19 Higher Level Flow Diagram of AIS Thread (Illustrated by Author)

5.3.1.2 MQTT RX Thread

The MQTT RX Thread is a critical component responsible for receiving, parsing, and acting upon MQTT messages within the system's IoT communication framework. Its operation begins with thread initialization, where all necessary resources are prepared for message reception and processing. During this phase, the thread retrieves the IoT service instance (p_iots) from the global message queue (g_msg_queue) to configure the MQTT client. The framework is kept unchanged and has been followed from the application project [103]. It also establishes synchronization mechanisms, such as the g_mqtt_notify_sem semaphore, to coordinate message handling efficiently and ensure the MQTT client is ready to receive data.

Once initialized, the thread enters the subscription and notification handling phase. It waits for notifications through the g_mqtt_notify_sem semaphore, which signals that an MQTT message has arrived. Upon receiving a notification, the thread refers to the num_messages field in the MQTT client control structure (p_ctrl) to determine how many messages are pending for processing.

Incoming messages are then passed to the message parsing function (parse_message), which examines the topic and payload to determine the appropriate action. The thread supports a set of predefined commands that control system hardware, including LEDs (optionally kept for debugging) and receiver modes. Based on the message content, the thread executes the corresponding control logic to manipulate hardware states, basically toggling GPIOs state.

The thread handles AIS receiver sleep and LNA mode management. It interprets MQTT messages to toggle these modes through dedicated GPIO pins (IOPORT_PORT_06_PIN_01 for sleep mode and IOPORT_PORT_06_PIN_02 for LNA mode). The updated receiver states are also sent to the g_mqtt_rx_payload queue for system-wide visibility.

Following successful message parsing and execution, the thread triggers event flag signaling by setting the SYS_EV_RX_UPDATE flag using tx_event_flags_set API call of ThreadX. This mechanism notifies other threads that the command has been received and processed, allowing them to perform any dependent operations accordingly. This is then printed in the user terminal.

Thus, like the AIS thread, a robust error handling mechanism ensures reliable operation even under abnormal conditions. The thread detects and logs issues such as null pointers encountered during message parsing, invalid or unrecognized MQTT topics, and unsupported payloads. All such errors are reported to the console to aid in debugging and system monitoring.

Finally, the thread operates in an infinite loop, continuously waiting for new MQTT messages from the broker and processing them as they arrive. It ensures that every message is fully handled before awaiting the next notification, maintaining real-time responsiveness and stable communication between the MQTT client and the embedded system components. Figure 20, gives a high-level workflow of the thread.

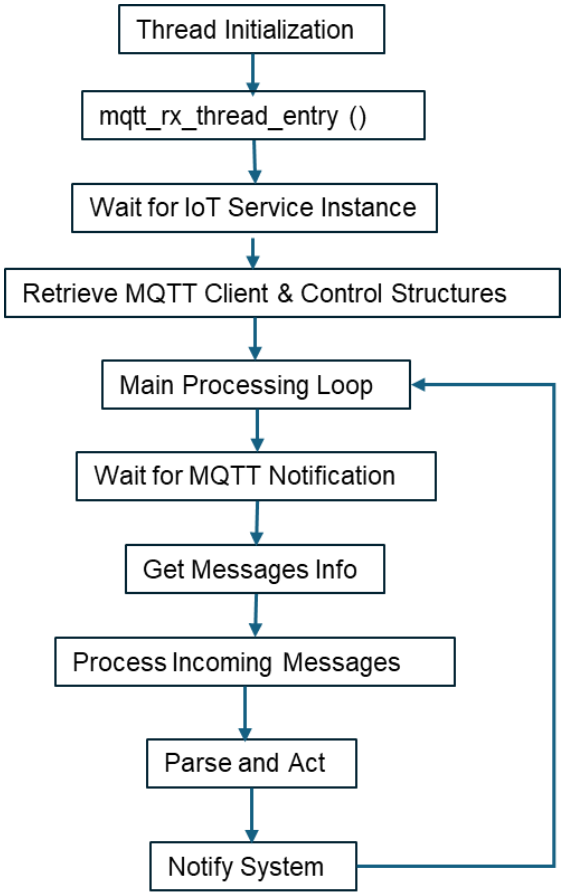

Figure 20 Higher Level Flow Diagram of MQTT RX Thread (Illustrated by Author)

5.3.1.3 MQTT Thread

The MQTT Thread serves as the central communication and coordination module that manages the system's MQTT-based connectivity, event handling, and data exchange with the IoT service. Its operation begins with thread initialization, during which the MQTT client, IoT service, and network configurations are properly set up to establish reliable communication channels. Additionally, timers are configured to manage periodic MQTT updates and implement retry mechanisms for IoT service or network initialization, ensuring that the thread remains resilient and responsive even in cases of temporary connection failures. Through this setup, the thread becomes fully prepared to handle MQTT communication and responds to system events effectively. The code has been developed for AIS reception on the framework provided by the application project.

Following initialization, the thread transitions into starting demo handling, where it awaits the SYS_EV_DEMO_START event flag that signals the commencement of the demo operation. Once this event is detected, the thread loads network, IoT service, and security certificate configurations from internal storage, then initializes both the network interface and the IoT service to establish a stable connection with the IoT platform. At this point, the thread also releases the g_AIS_uart_semaphore, allowing AIS Thread to proceed with data reception. The MQTT system timer is also activated to manage scheduled updates, and relevant IoT service information is transmitted to the MQTT RX thread for continued message processing and synchronization.

The thread then enters a continuous event handling loop, which operates indefinitely to monitor and process incoming system events. This loop ensures that the MQTT thread remains continuously responsive to various operational signals and can dynamically adapt to system state changes in real time.

Within this loop, event processing plays a key role in managing system behavior based on specific event flags. When the SYS_EV_MQTT_DISCONN event occurs, the thread safely handles MQTT disconnection by stopping active timers, de-initializing the IoT service, and initiating reconnection attempts. The SYS_EV_IOT_NW_RETRY event triggers retry operations for the IoT service or network initialization, ensuring automatic recovery from communication failures. The SYS_EV_AIS_UPDATE event allows the thread to process AIS data received from the AIS thread and forwards it to the IoT service for external transmission. Likewise, the SYS_EV_RX_UPDATE and SYS_EV_MQTT_UPDATE events handle data synchronization, updating the IoT service with the latest MQTT payloads. When a SYS_EV_DEMO_STOP event is detected, the thread gracefully stops all timers, de-initializes services, and shuts down the network interface, ensuring an orderly system halt.

The thread's timers and retry mechanisms are fundamental to maintaining continuous operation and network stability. Periodic timers schedule MQTT system updates, while retry counters monitor connection attempts to recover automatically from temporary service disruptions. This approach enhances robustness and ensures minimal downtime.

Finally, during cleanup and exit, the MQTT thread deactivates all active timers, releases allocated resources and performs orderly deinitialization of network and IoT service components. This ensures that the system shuts down gracefully without leaving residual processes or memory leaks. Overall, the MQTT thread functions as the backbone of the IoT communication architecture maintaining connectivity, synchronizing data between threads,

and ensuring reliable, real-time interaction with the cloud-based IoT platform. Figure 21, gives a high-level workflow of the thread.

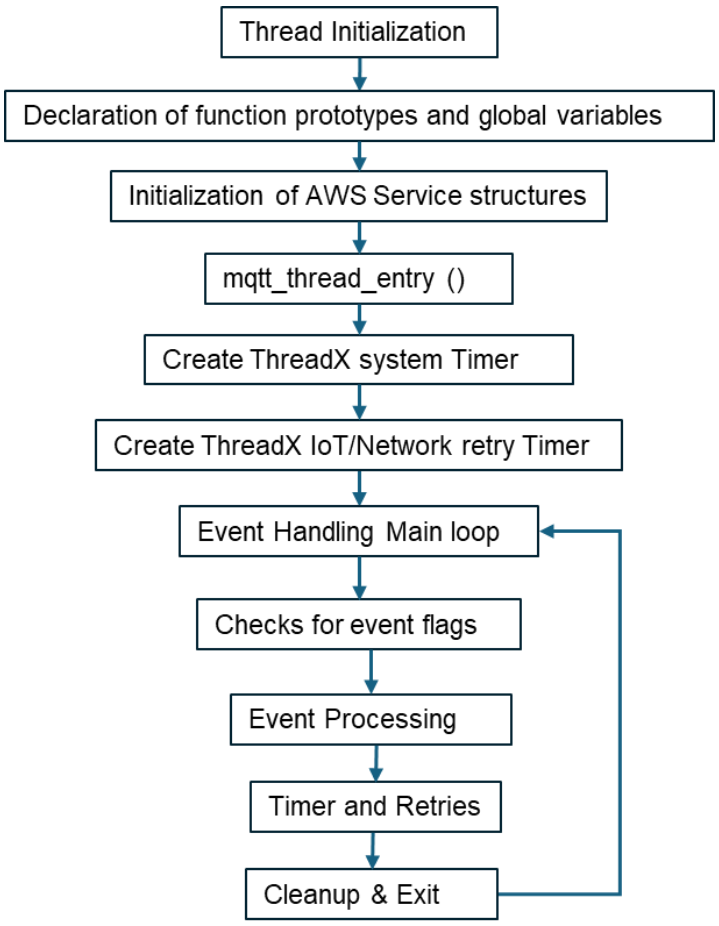

Figure 21 Higher Level Flow Diagram of MQTT Thread (Illustrated by Author)

5.3.1.4 Console Thread

The Console Thread serves as the primary user interface component of the IoT device, enabling direct interaction between the user and the application through the micro-USB port of the board. During thread initialization, it configures the console framework and prepares the system to receive user commands. Upon startup, the thread displays essential application information, such as the firmware version and software package details, providing users with contextual awareness of the running system. The original framework is kept from the application project and has been developed for the prototype use case.

Once initialized, the thread transitions into its main loop**,** where it continuously monitors user commands using the console framework's prompt functionality. This loop ensures that the console always remains active and responsive, allowing real-time command execution without interruption. During production phase this logic needs to be changed and activate the framework only through external interrupt.

Incoming commands are managed through the command input handling phase. The thread listens for user input and employs the console framework's callback mechanism to interpret

and route commands to their designated functions. This design provides a modular and efficient way to extend console functionality without modifying the core logic.

During command parsing, the thread supports a set of predefined commands designed for system configuration and control. For instance, the “cwiz” command launches the network and cloud configuration menu, allowing users to set up network interfaces, IoT services, and certificates interactively. The “demo” command enables the user to start or stop the demo application depending on the argument provided (start or stop). Each recognized command is mapped to a dedicated callback function, ensuring organized and predictable system behavior.

Just like the other threads, it also incorporates robust error handling mechanisms to maintain reliability. When an invalid or unrecognized command is entered, the console provides a clear error message without disrupting the main loop. This ensures that the console remains stable and responsive even under incorrect user input or unexpected conditions.

Effective resource management is another key aspect of the console thread’s operation. Shared resources, such as the console mutex, are managed carefully to ensure thread-safe operations and prevent conflicts with other system components. After each operation, resources are properly released to avoid potential deadlocks or memory contention.

Finally, the console thread includes an optional demo auto-start feature, allowing the application to begin execution automatically at system boot. This functionality is achieved by setting the SYS_EV_DEMO_START event flag during the thread’s initial execution. Figure 22 gives a high-level workflow of the thread.

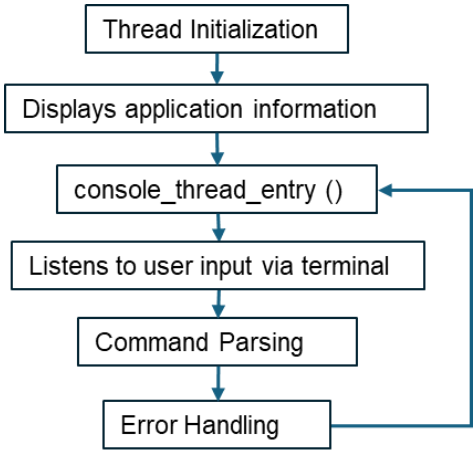

Figure 22 Higher level Flow Diagram of Console Thread (Illustrated by Author)

5.3.2 Thread Stacks

The SSP configurator of E2Studio helps build the project by adding lower-level drivers needed for different modules and frameworks. This is very helpful for developing a project with the

help of abstraction layers and API calls by-passing the low level bare-metal coding. SSP configurator thus adds portability of code across different Renesas microcontroller platforms.

Figure 23 HAL/Common Stacks (Illustrated by Author)

On creating a new project for a specific board, HAL/Common stacks are added automatically with different modules for providing high level API for clock control, connecting different peripherals for autonomous operation, I/O pins control, and flash interface. Detailed resource guide for all the module and framework discussed in this section is available in the Renesas support page under knowledge base [104].

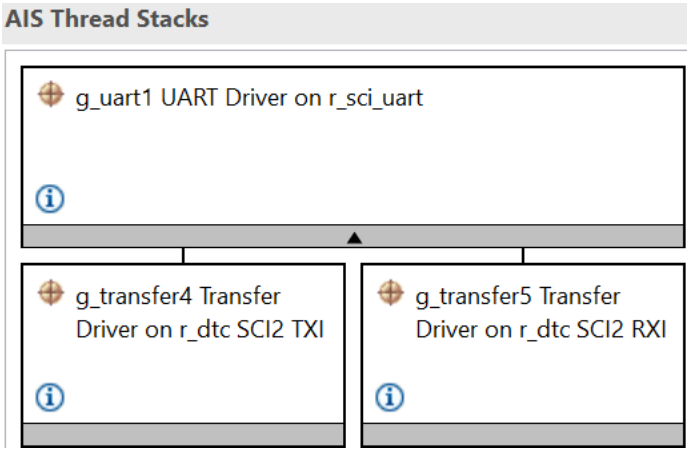

Figure 24 AIS Thread Stacks (Illustrated by Author)

As mentioned in 5.3.1, AIS thread is responsible for AIS data reception, processing and forwarding to MQTT thread. Figure 24 shows the use of UART module which allows high-level API for serial communication application. Here, it is using the SCI peripheral on channel 2. Some drivers require additional configuration information by the system. The SSP configurator asks for them using different colors like red, pink or blue. A gray color indicates individual modules which are stand alone. Blue color means shared modules. Modules with pink color band might need lower-level drivers and red band indicates a need for extra configuration information.

Figure 25 shows the console framework that provides a complete API implementation for a menu-based command line interface (CLI) using the ThreadX RTOS. This can be implemented either by UART, USB or Ethernet Telnet connectivity. Here, the framework is set to use USBX device class CDC-ACM module, and it uses USB Full Speed (USBFS) connectivity. It can be also configured for USB High Speed (USBHS).

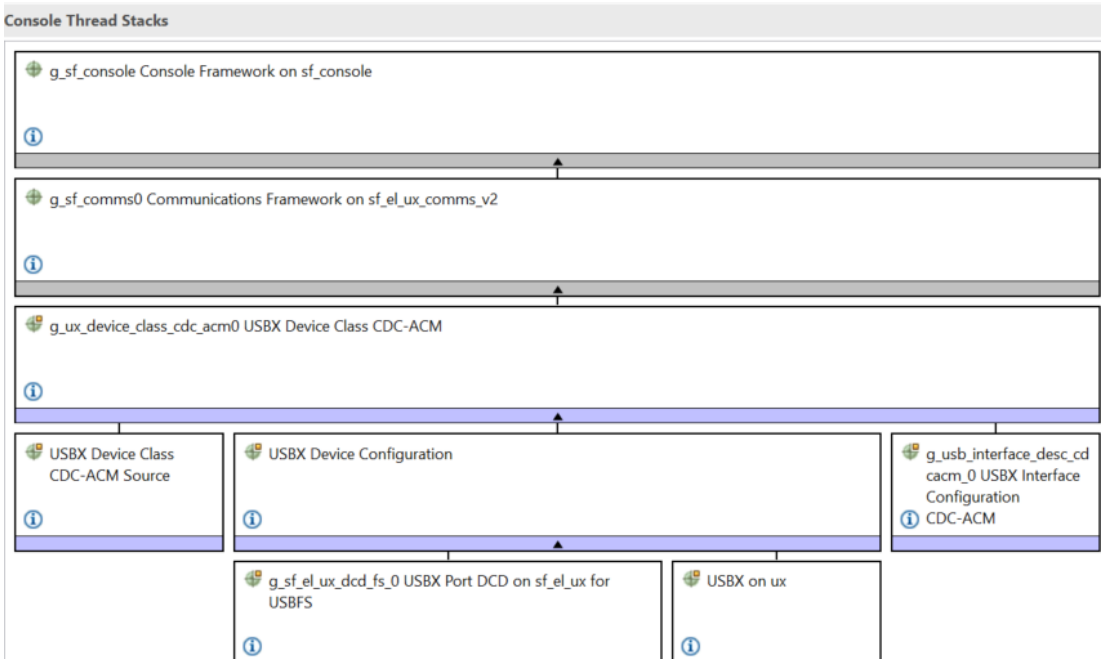

Figure 25 Console Thread Stacks (Illustrated by Author)

MQTT Thread is the heart of the prototype project. It handles the network connectivity, AWS connection, data publishing and so on. The rest of the section will discuss each module or framework used under MQTT thread stacks. Figure 26 shows the implementation of MQTT client provided by the NetX Duo TCP/IP network stack from X-Ware or also known as the Synergy Express Logic. This client instance requires an IP instance, packet pool and TLS support package. The ethernet module is integrated to this framework as the project is built to support internet connectivity over ethernet. The guide [105] for communications framework on NX module provided by Renesas Corporation makes it easier to implement. The MQTT client is needed to communicate to the AWS IoT core, which here acts as the MQTT broker.

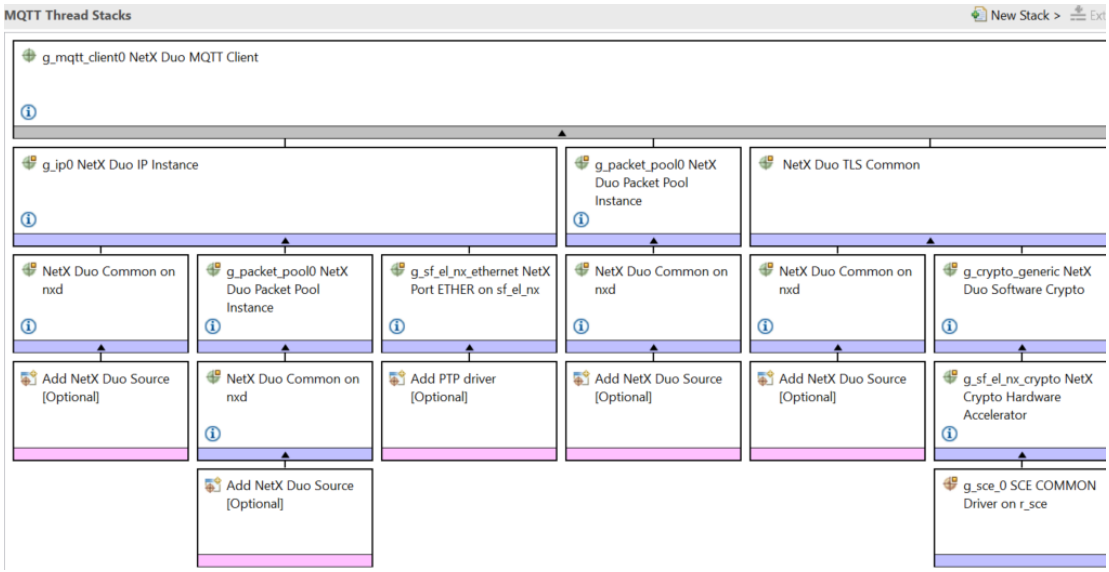

Figure 26 MQTT Client instance of MQTT Thread Stacks (Illustrated by Author)

Figure 27 shows the implementation of cellular framework via SSP configurator. This part of the implementation is incomplete; however, the framework is set following the application note

from Renesas Corporation [106]. Natively, SSP supports RYZ014A CAT M1 and Quectel BG96 Modems. The software needs to be integrated separately for any other modem.

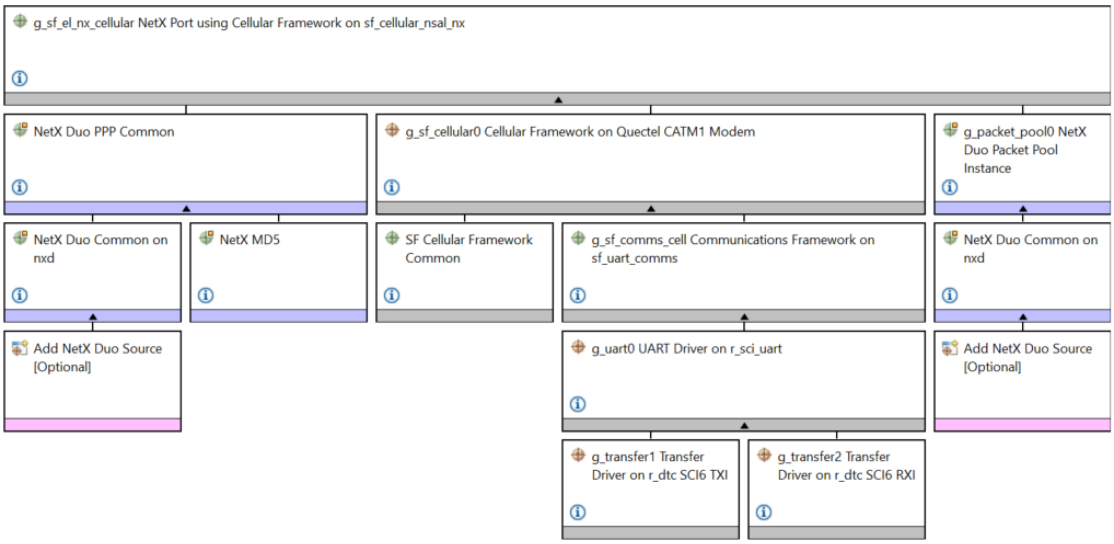

Figure 27 Cellular Framework of MQTT Thread Stacks (Illustrated by Author)

Figure 28 shows the implementation of DHCP and DNS client module in the MQTT thread stack. This is important to dynamically assign IP address for the device for connecting to the internet and translating AWS endpoint name to an IP address respectively.

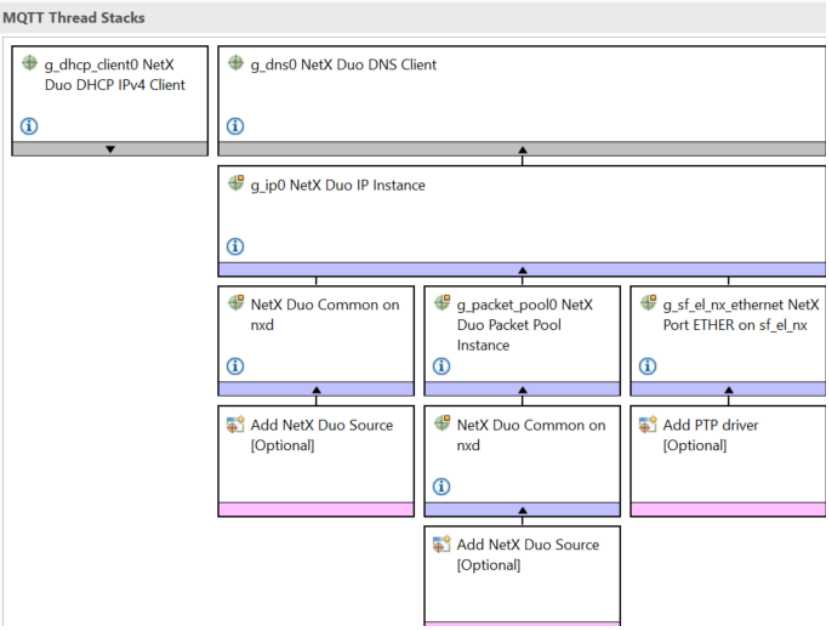

Figure 28 DHCP and DNS Client Module of MQTT Thread Stacks (Illustrated by Author)

Figure 29 shows the implementation of SNTP client module that is also important for connecting to AWS IoT Core. This client delivers precise timing information over an IP network. Simple Network time Protocol is used for clock synchronization and is needed for establishing a connection to AWS IoT.

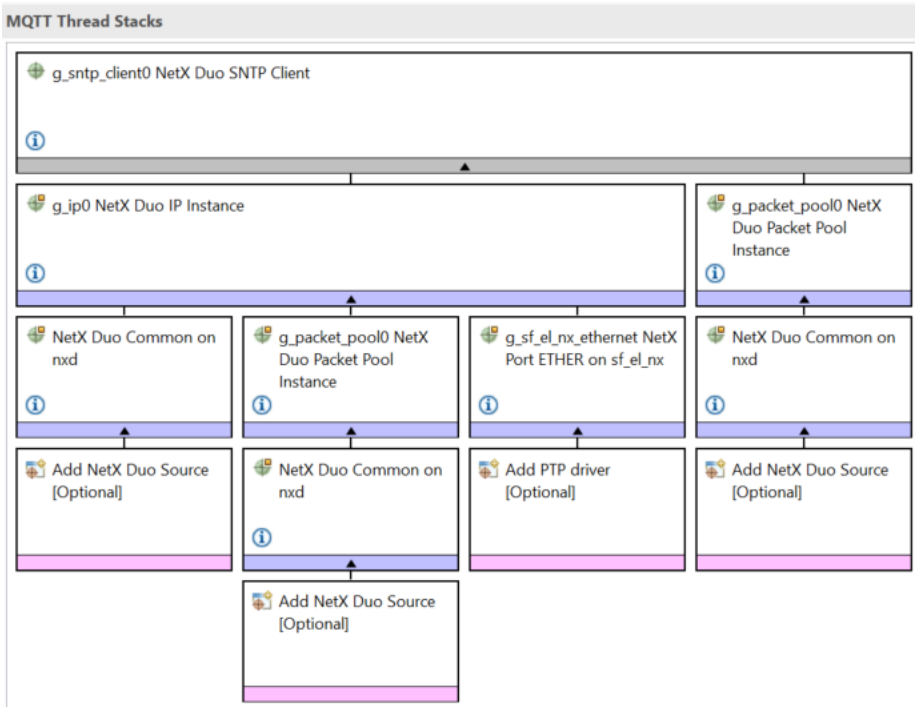

Figure 29 SNTP Client Module of MQTT Thread Stack (Illustrated by Author)

Figure 30 shows a supporting module of the MQTT thread stack. The flash driver is essential for managing non-volatile storage operations needed to access different information within the threads.

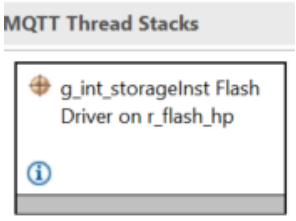

Figure 30 Supporting Module of MQTT Thread Stacks (Illustrated by Author)

5.4 Power Analysis

The power consumption calculation for the prototype has been done in two ways. First it has been measured through the J19 USB debug port of the PK-S5D9 board. Details on the boards jumper port can be accessed in the user manual [107]. In the second method, it is measured via its J31 jumper port. This lets the user to directly get the voltage reading across a 50 m ohm (+/- 1%) resistor [108]. It gives the digital current consumption of the chip itself. As the board is using the UART protocol to communicate with the receiver, measuring the digital current consumption should give an idea of the power profile, which is needed for designing PCB in production phase. However, data transmission via ethernet port requires current from the analog rail, which is discussed later in this section.

Figure 31 Measuring Current Consumption of the Board PK- S5D9 During Runtime with Current Ranger (Illustrated by Author)

Figure 31, demonstrates the first case where the Current Ranger device has been placed in series connection between the connectors to get the overall current consumption of the whole board. This includes the total consumption of a bunch of unnecessary stuff, for example, user LED's, on-board debugger, etc., that are connected to the development board. The board also comes with LCD, capacitive touch and a lot of GPIOs. Before measuring the current consumption, these features have been disabled and the GPIOs were put to input/output state with pull-up/down resistors. Floating GPIOs induce leakage current and oscillate due to noise and hence adds up to power consumption [109]. During production phase, the power consumption of the chip will be of interest only. In this chapter detailed power consumption of the prototype and S5D9 chip will be analyzed and compared to the values presented in its datasheet [110] using both the measurement methods.

The receiver dAISy Featherwing, can operate in three states so has three different current consumption [82]. In the lab, the receiver's current consumption is calculated using Current Ranger in all the states and the average values are quite close to the datasheet value. The measurements are taken by setting the current meter in series between the power supply of the receiver only.

Table 11 Power Consumption of the Receiver in Different States

Part	Receiver States	Voltage (V)	Current (mA)	Watt	Wh (24h)
A	LNA Enabled (Receiving Data)	3.3	70	0.231	5.544
B	LNA Disabled (Receiving Data)	3.3	36	0.1188	2.8512
C	LNA Disabled (Not in Receiving Mode / In settings Mode)	3.3	12.7	0.042	1.01
D	Sleep	3.3	0.35	0.001155	0.02772

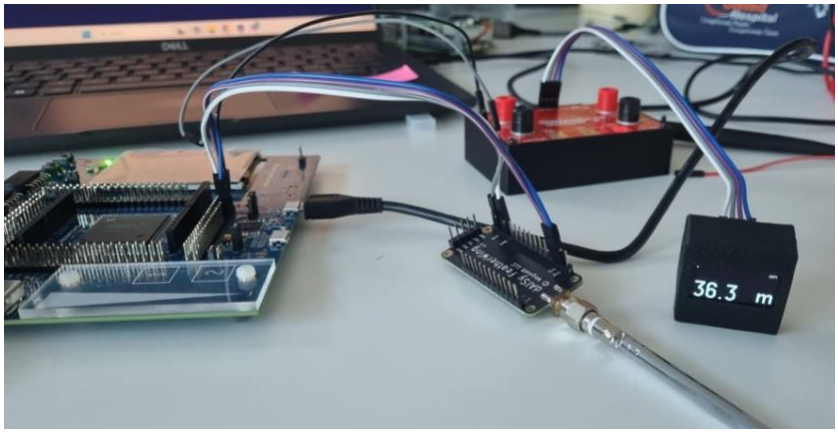

Figure 32 Measuring Current of the Receiver when LNA is Disabled (Illustrated by Author)

The need for AIS tracker restricts itself from sleeping as that will risk losing valuable data coming from the sea traffic. However, this sleep mode can be used by the BMS in emergency case, when power backup is almost non-existent. It will help the IoT device from going dark.

Table 12 Daily Power Consumption of the Prototype for AIS Receiver (LNA Enabled)

Part	Components	Voltage (V)	Current (mA)	Watt	Wh (24h)
A	PK- S5D9 dev board	5.0	72	0.36	8.64
B	dAISy receiver (LNA enabled)	3.3	70	0.231	5.544
C	Ethernet connectivity overhead	5.0	11.77	0.058	1.412
D	Data Processing overhead when antenna is connected	5.0	3.9	0.019	0.468
Total					16.064

Table 13 Daily Power Consumption of the Prototype for AIS Receiver (LNA Disabled)

Part	Components	Voltage (V)	Current (mA)	Watt	WH (24h)
A	PK- S5D9 dev board	5.0	72	0.36	8.64
B	dAISy receiver (LNA disabled)	3.3	36	0.1188	2.8512
C	Ethernet connectivity overhead	5.0	11.77	0.058	1.412
D	Data Processing overhead when antenna is connected	5.0	3.9	0.019	0.468
Total					13.372

In Table 12 & Table 13, Voltage is considered 5V for dev board as this is the supply voltage at the USB debug port J19. The current is measured in different states by Current Ranger as shown in Figure 31 and Figure 32. The receiver current values are considered from Table 11. The difference between the total power consumption demonstrates that an approximate power of 2.7W can be saved daily just by considering the different operational states of the Featherwing receiver. Data processing overhead has been considered the same here as it does not fluctuate much, depending on the vessel traffic.

The power consumption tables above consider the power of the whole development board which does not give the actual picture of the power flow diagram, as during production the chip will be used instead of the dev board. Therefore, actual picture is obtained by considering the power of the chip and the peripherals needed for AIS tracking.

In PK-S5D9, a step-down buck converter [111] transforms the supplied 5V voltage to 3.3V, which is fed to the main MCU chip. Most of the digital peripherals along with the chip itself is powered from 3.3V rail. The board come with Micrel KSZ8081 10/100 Ethernet physical interface which requires power from both digital and analog rail. The ethernet interface core logic runs on a 1.2V digital power rail, while the analog portion of the circuit, which includes the transmit and receive functions for the physical medium, requires a separate analog power rail. This can be more clearly understood from the schematic diagram [112] of the board. The datasheet of KSZ8081 [113] shows a typical total power consumption of the chip for 10-100 base varies in between 82.5-152 mW for TX link-up (no traffic or activity).

Table 14 Effect in Power Consumption due to Change in Vessel Traffic (LNA Disabled)

Part	Cases	Voltage (V)	Current (mA)	Avg. Current (mA)	Watt	Wh (24h)
A	No Traffic (No Antenna connected)	5.0	107.8-109	108.4	0.542	13.008
B	With typical traffic (~1200 message/min using VHF-74 placed in rooftop)	5.0	112-116	114	0.57	13.68
C	With less traffic (VHF-23 Antenna placed inside office, ~260 message/min)	5.0	111.2-114	112.6	0.563	13.512
D	With less traffic (Extendable SMA antenna placed inside office, ~50 messages/min)	5.0	109.8-110.4	110.1	0.5505	13.212

Table 14 shows the difference in power consumption of the total system measured at the debug node of PK–S5D9 with a Current Ranger while considering different traffic condition using different antennas from the Hamburg office, which is located near to the Elbe River. The office is on the seventh floor, and the rooftop antenna is place at two levels above the office floor, which gives the sufficient clearing and height (approximately 31m above the ground) for the antenna coverage. The experiment was done in a gloomy day, around mid-day. From the experiment, it is observed that the prototype is not good with handling high data reception like from VHF-74 and the power consumption doesn't differ much due to change in vessel traffic. It is noted that only the board connected to the current ranger gave a measurement around ~72 mA, while further powering the receiver from the board gave an additional ~26mA and the remaining was due to ethernet connectivity and data transfer overhead when measured at the USB debug input port.

Table 15 Power Consumption of S5D9 in Different Power Modes

Part	Cases	Voltage across resistor R113 (mV)	Resistor R113 (m Ohm)	Calculated / measured Current (mA)	Supply voltage (V)	Watt	Wh (24h)
A	High Speed Mode	0.8	50	16	3.3	0.0528	1.2672
B	Low Speed Mode	0.4		8	3.3	0.0264	0.6336
C	Software Standby Mode	0.1		2	3.3	0.0066	0.1584

Renesas synergy S5D9 chip supports different power modes. The modes are namely: Normal Mode, Sleep Mode, Software Standby Mode, Snooze Mode and Deep Software Standby Mode [114].

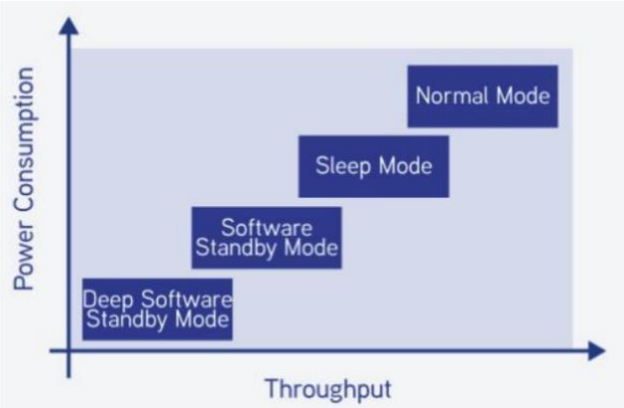

Figure 33 Effect of Power Consumption on different run modes [115]

Figure 33 shows the trade-off between throughput and power consumption of the S5D9 chip. Renesas provides a power profile V2 (PPv2) framework for the S5D9 chip which allows it to run in high clock speed and low clock speed with an option to stall in standby mode. During sleep and software standby mode, the clock gating technique is applied and in deep software standby mode it is powered off. This framework helps to demonstrate the power profile and allows to measure the current consumption of the chip through the J31 port of PK-S5D9 while running in different clock speeds. In the high-speed mode, the system runs at 120MHz using PLL clock source and keeping different peripherals and GPIOs active. In Low-speed mode it runs at 32.768 KHz using the LOCO system clock source with most of the peripherals and GPIOs inactive. The application note [115] explains the project and provides code for better understanding.

Table 15 shows the average power consumption of the chip calculated following Ohm's law in different modes, while running the PPv2 application project. This power analysis is done to understand the actual digital power consumption of the chip itself to get an exact picture of the power graph, which will help to understand the power demand in production phase. This PPv2 framework is not integrated in the prototype but can be included in future scope.

Table 16 Estimated Daily Power Consumption in Production

Part	Components	Voltage across resistor R113 (mV)	Resistor R113 (m Ohm)	Calculated / measured Current (mA)	Supply voltage (V)	Watt	Wh (24h)
A	S5D9 chip (Digital current at runtime)	1.5	50	30	3.3	0.099	2.376
B	Estimated data transfer overhead (Analog current)	-	-	-	-	0.029	0,696
C	dAISy receiver (LNA disabled)	-	-	36	3.3	0.1188	2.8512
Total							5.9232

Table 16 shows the average power consumption of the chip along with the receiver. Part A of the table does not include the analog current consumed by the ethernet port during data transmission. As the network is attached, an increase of 0.2-0.3 mV is observed across the R113 resistor which is calculated to 6 mA or 19.8 mW. Therefore, let's consider ~20mW as the digital power consumption of the ethernet port connectivity. Now as the analog power is consumed during data transmission only, it can be calculated from the Table 14. It can also be directly calculated by removing the R114 resistor and measuring current across it [107]. Calculating the energy difference during no traffic (no data transfer) and high traffic (data transfer) scenario, analog power consumption can be estimated. The calculation gives ~29 mW (estimated) of power consumption. Thus, the total energy consumption at production phase can be estimated around ~5.8 Wh/day when LNA is disabled or ~8.62 Wh/day when LNA is enabled.

6 AWS IoT Core Integration

Data collection, storage and presentation are important parts of an IoT system. The company has its own backend, which processes the data and provides an interactive platform for the users to visualize data in real time. Currently, RaspAIS stations sends the raw data directly to Vesseltracker servers and from there is it forwarded and processed further.

The company is in the process of migrating to AWS IoT Core for data storage and handling. Thus, one of requirement of the prototype was to send the collected data to AWS IoT Core. AWS provides detailed guideline for starters [116] and also provides support engineers for companies migrating to their services.

Unlike other servers, AWS IoT Core requires security clearance and authentications for the device to communicate to MQTT broker. It doesn't allow unsecured communication to its servers. The easiest way to integrate an IoT device is to use the SDK provided by AWS IoT. Device connection to AWS IoT use X.509 client certificates and AWS signature V4 for authentication. Device communications are secured by TLS version 1.3 (also supports TLS 1.2) and AWS IoT requires devices to send the Server Name Indication (SNI) extension when they connect. Therefore, for the IoT device to connect, it requires AWS root certificate, private key and client certificate. These credentials are provided in PEM format by AWS IoT Core when a new Thing is registered, and corresponding policies are attached to it. However, for RTOS like ThreadX, DER formatted certificates are also accepted. H-Converted certificates can also be used for embedded devices using C programing. Thus, a developer needs to be aware of the right format while using the credentials. The credentials tell the AWS server which device will be connecting to it for communication. Each registered Thing has a unique private and client certificate. AWS provides a developers guide to integrate the SDK and troubleshoot different issues in the following website [117].

While designing a resource-constrained, low-powered IoT device, any overhead adds a challenge. The overhead of this TLS connection required for AWS is not negligible. If the IoT device is designed to reconnect to AWS IoT Core after sleep, it means that it must go through the authentication process again and confirm the handshake before data transfer. The total overhead to establish a new TLS session comes to about 6.5k bytes on average and the total overhead to resume an existing TLS session comes to about 330 bytes on average [118]. This can quickly result in a significant number of data packets, thereby increasing overall data consumption and associated costs.

To sum up, the IoT device needs AWS IoT Core SDK integrated in the microcontroller and the Thing (IoT device) needs to be registered along with its policies in the AWS IoT Core service management console. For the prototype, the policy is set to allow any IoT device to connect with the right credentials. AWS provides multiple services for device management and data handling (e.g. OTA service) which can be really complicated to clearly understand even though AWS made a good effort with the documentation process.

7 Energy Harvesting System

Energy harvesting system is a crucial part of the design for any battery powered IoT device. The battery will run out of juice at a certain time, and a harvesting system is designed to recharge the battery so that the remote IoT device remains alive. There are a lot of different sources for green energy and some of them are also discussed in section 2.3.5. In this section observed current peaks of the prototype will be discussed at run time.

IoT devices have drastically varying profiles, where in active state they demand tens to hundreds of milliamperes during transmission or processing, and in idle state, only nanoamperes to microamperes. This high peak-to-average ratio can be a challenge, as insufficient current capability can cause voltage droop during transmission bursts, potentially leading to communication failures or system crashes, whereas oversizing can raise cost, size, and quiescent power consumption. Hence, it is crucial to consider current peaks while selecting components throughout the power management chain. It is a critical design challenge as instantaneous harvested power may be lower than peak demands. Energy storage devices like rechargeable batteries or supercapacitors must, therefore, be able to handle both total energy capacity and ability to supply peak current pulses without voltage droop leading to system brownout. Energy buffers should store excess harvested power when generation exceeds demand and supply deficit power during peak consumption [52] [51].

Modern solar harvesting systems incorporate advanced power management units and high-efficient voltage converters that optimize energy transfer and storage. Moreover, recent advancements in PV technology have pushed solar cell efficiencies making solar harvesting a leading choice for sustainable power generation in IoT sector.

For designing an energy harvesting system for AIS tracker, either solar or wind can be chosen. A hybrid approach can also be decided. However, the energy conversion efficiency is comparatively poor in hybrid systems. The implementation of the energy harvesting is kept as a future scope of work.

Depending on the choice for green energy source, a power supply unit (PSU) or a management unit (PMU) needs to be designed. Solar power harvesting typically includes selecting the right solar panel, implementing a MPPT charge controller, a DC-DC convertor and an energy storage system like a battery or capacitor or both. Designing any PSU, the maximum and minimum peak currents of the IoT device needs to be considered and the selected components should be able to handle the current demands to avoid overheating, brownout or burnouts.

To calculate the digital max and min peak currents, Picoscope 2207B MSO [119] has been used to measure the voltage across the R113 resistor of J31. Moreover, the overall peak current of PK-S5D9 has also been measured using BI MHP 140 0R01, a high-power resistor of 10 m Ohms. The negative current values represents current flowing in the opposite direction, which is mainly due to the discharge current from the capacitors used on the promotional kit and also might include some back flow.

Table 17 Determining Maximum and Minimum Peak Currents at J31

No.	Cases	State of the MCU	Max Current (mA)	Min Current (mA)	Avg. Current (mA)
1	No Traffic	Powering the board	187.67	-66.48	5.79
		Interacting in serial terminal	224. 26	-130.82	30.07
		Starting demo	224.26	-119.6	30.43
		Attaching ethernet interface	190.63	-123.34	30.36
		Waiting to receive data	171.94	-100.91	30.45
2	No Traffic	During Flash Download by JTAG	149.50	-119.59	11.28
		Attaching COM terminal	183.14	-119.59	25.96
		During Starting Demo	190.63	-108.34	30.52
		Stopping Demo	153.24	-97.17	27.04
		Debugger to flashing code	171.94	-127.1	14.48
		Playing Run Mode in Debugging	156.97	-97.17	26.14
3	With Data Traffic from Extendable SMA Antenna	During Flash Download by JTAG	112.91	-70.23	10.44
		After Flash Download Complete	101.69	-88.91	6.15
		Playing Resume in Debugging	109.17	-73.96	18.58
		Playing Resume from Main Function Debugging	127.86	-59.018	26.85
		Interacting via serial terminal (Pressing Enter)	116.64	-66.48	26.68
		Interacting via Serial terminal	120.38	-66.48	27.03
		Interacting via Serial terminal (Providing Demo Command)	120.38	-59.02	26.94
		Demo started	127.86	-62.74	29.91
		Network Attached	139.07	-62.74	30.2
		Data Reception + Transmission	124.13	-55.27	29.82
		Data Reception + Transmission	116.64	-62.74	30.24
		Data Reception + Transmission	131.59	-47.79	29.88
4	With data Traffic from Whip Antenna	Data Reception + Transmission	160.71	-89.702	31.147
		Data Reception + Transmission	168.196	-93.44	31.09
		Data Reception + Transmission + Toggling LNA	142.027	-71.013	31.24

Table 17 shows the peak currents calculated from the voltage measured by the oscilloscope at different states from port J31. The observed max current is 224.26mA and min current is -130.82mA. The energy harvesting components must be chosen in a way so that it can handle the current peaks. Section 11.3 shows oscilloscope graphs and explains current calculation process from the graph.

Table 18 Current Peaks Measured Across 0.01R

No.	Cases	State of PK-S5D9	Max Current (mA)	Min Current (mA)	Avg. Current (mA)
1	No Traffic	Power on	4111.40	-560.56	8.73
2	No Traffic	Demo Running	853.162	-548.47	121.38
3	With Data Traffic	Demo Running	1208.23	-1015.68	122.31
4	With Data Traffic	Demo Running	1189.53	-1015.68	118.56

Table 18, shows the current peaks observed during data transmission by the board PK-S5D9. Section 11.3 displays the signal change as the board receives and sends the data to AWS IoT Core.

Considering solar harvesting system, a design guide will be suggested here based on some calculation assumption. The results will vary depending on the location, design consideration taken by the company and the actual efficiency of the components. Let's assume the daily energy consumption is 16.06 Wh for the prototype (using PK-S5D9) with receiver's LNA enabled. Additionally, assuming a 20% solar system loss or 80% system efficiency without considering the battery:

Total Energy Required by Panel = 16.06 Wh / 0.8 = 20.075 Wh.

To determine solar panel sizing, solar irradiance for the intended geographical location is important. The following sites, [120] and [121], provide helpful information about day light and solar irradiance. Assuming 4 hours of sunlight on average for Hamburg:

Panel Wattage Required = Total Daily Energy Required / Sunlight Hours
= 20.075 Wh / 4 h = 5.01875 W = ~6W

Coming to the storage device, considering 3.2V LiFePO4 battery with ~90% efficiency and 3 days of autonomy:

Battery Energy Requirement = 16.06 Wh × 3 = 48.18 Wh
Adjusted Capacity Considering Efficiency = 48.18 Wh / 0.9 = 53.53 Wh
Battery Capacity (Ah) = Adjusted battery Capacity / Nominal Voltage
= 53.53 / 3.2 = 16.73 Ah = ~ 18Ah

The battery-side charge current at the controller's output can theoretically be calculated as:
I_Panel = Panel power / battery voltage
= 6W / 3.2V
= 1.87 = ~2A

To summarize, a solar panel of 6W, 6-12 V nominal output voltage would be sufficient for the system and a safer choice would be 10W. A 3.2V, 18 Ah LiFePO4 battery with BMS would be recommended along with MPPT or PWM charge controller that can support both panel and battery topology. It is required to note that the charge controller must be compatible with the LiFePO4 battery. MPPT has more efficiency compared to PWM. However, it is also comparatively costly.

All protective parts must handle the total peak currents. Total peak current would be the sum of I_Panel and I_Peakload (~4.11 A) observed from Table 18. Fuse and wiring need to be chosen to handle the total current peak of ~6 A. Additionally, considering a 25% safety margin, a wire rated to handle around 7.5A continuous current would be recommended to reduce heat loss and a 7.5A battery fuse can be added to ensure safety. A charge controller >8A capacity should be chosen with a built in USB output port to power the device, otherwise it might need a DC-DC convertor depending on the voltage difference. Capacitors can also be integrated to handle the inrush current peaks.

8 Performance Analysis and Results

In this section, detailed performance analysis with respect to power consumption will be graphically provided. Moreover, data handling capability of the prototype will also be discussed based on the data logs from Tera Term terminal and AWS MQTT test client console.

Designing an energy-efficient AIS tracker first involved market analysis to know if any hardware vendor provides efficient low-powered AIS receiver or transceiver, as discussed in detail in Chapter 2.2. Secondly, it was important to know the power graph of RaspAIS kit. It helped to understand the power-hungry components of the system and facilitate the decision-making process on prioritizing where to optimize.

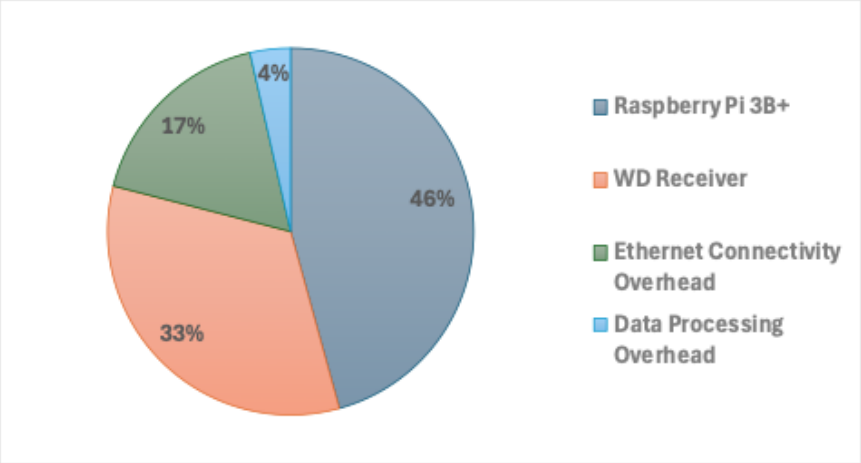

Figure 34 Power Consumption Share of RaspAIS Components (Illustrated by Author)

Figure 34 presents the power consumption percentage share of the components of RaspAIS based on the results of Table 7. The most power-hungry component of the RaspAIS is its processing unit, Raspberry Pi 3b+, followed by the Weatherdock receiver and the ethernet connectivity. Raspberry Pi 3B+ runs on an embedded Linux system, which is quite unnecessary for the use case. It also has a Gigabit Ethernet port which has a throughput of around 300 Mbps. This powerful ethernet port adds up to the connectivity overhead. Moreover, the receive does not support smart power handling modes and thus is not the best choice for a standalone, off-grid AIS tracking system. Therefore, to optimize energy consumption of the device, it made sense to consider optimizing both the processing unit and the receiver.

Figure 35 shows the daily power consumption of dAISy Featherwing receiver in different operating modes. The designer has the choice to select the different operating modes depending on the condition of the IoT device. For instance, at low to no sunlight on a gloomy day, the IoT device can be smartly configured to operate in LNA disabled state, saving almost half the power consumption of the receiver at the cost of reduced data reception. The device can also fall to sleep state if there is not enough battery backup.

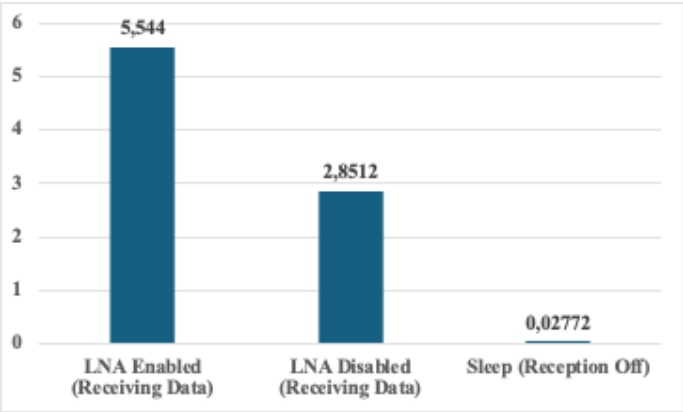

Figure 35 Daily Energy Consumption (Wh) Graph of dAISy Featherwing Receiver (Illustrated by Author)

The processing chip S5D9 of the prototype is designed with energy efficient architecture, allowing it to operate in different power modes. Figure 36 shows how a designer's choice can save energy per day with respect to the processing unit used in the prototype. The chip also supports other power modes which is discussed in its datasheet.

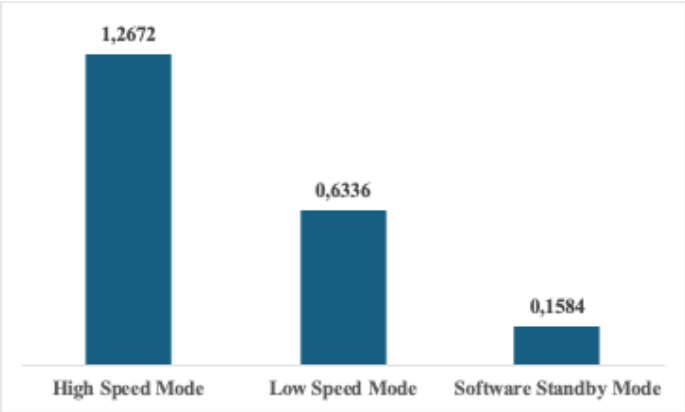

Figure 36 Daily Energy Consumption (Wh) Graph of S5D9 Chip (Illustrated by Author)

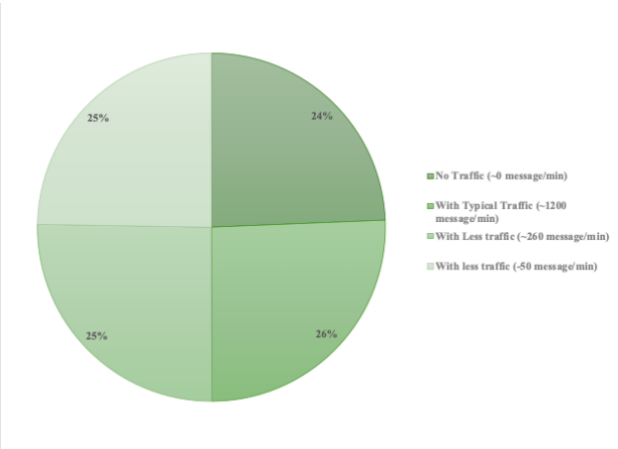

Figure 37 Power Consumption Share due to Change in Vessel Traffic (Illustrated by Author)

Figure 37 shows that there is very minimal effect in power consumption due to change in data processing overhead by the prototype. This is because all the stacks have already been

allocated, and network is already attached no matter if there is data to transmit or not. It would have an impact if the MCU is put to sleep and needed repeated network attachment process.

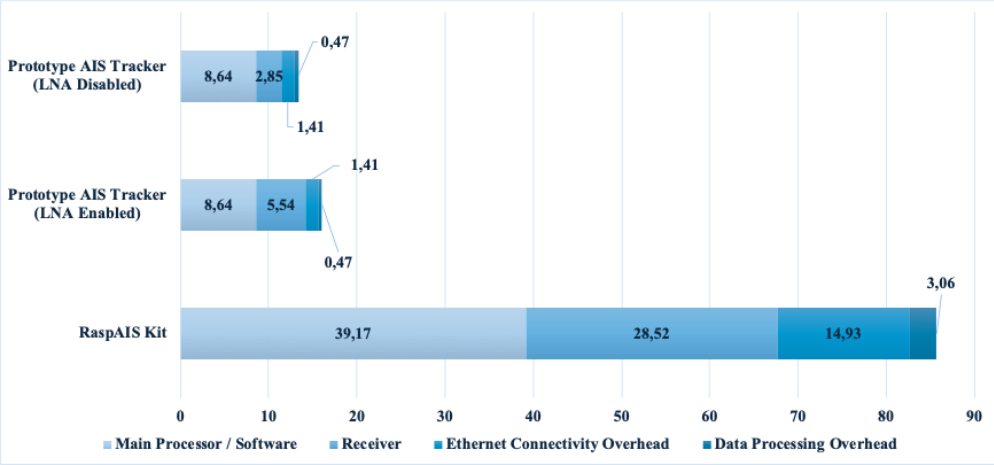

Figure 38 Comparison Bar Chart of Total Energy Consumption (Wh/24) (Illustrated by Author)

Figure 38 shows the comparison between total energy consumption of the prototype device with different receiver states and that of the RaspAIS kit. Through the proposed power optimization hardware and software techniques, the system's daily energy consumption decreased from 85.7 Wh to 16.06 Wh or 13.37 Wh, depending on receiver state selection. This results in an energy reduction of 81-84% and an overall efficiency improvement of approximately fivefold. Energy consumption can further be reduced to ~6Wh at production phase by using only the S5D9 chip instead of the promotion kit.

Another key factor to consider is the performance of the prototype AIS tracker, that is to determine the data reception rate of the device. Table 19 shows experiments done by changing the antenna and the UART receive buffer size (UART_RX_BUFFER_SIZE) in the code for different time periods. It is observed that at high data traffic rate (~1200/min), the prototype fails to receive all the messages as the buffer size is limited and increasing it makes the performance better. As the code is designed for the AIS thread to stall for around 50ms in every loop and allow other threads to complete tasks, it loses messages during that period when data frequency is high. The bigger the receive buffer size is, the more data it stores and thus also takes more time for MQTT thread to send it to AWS IoT Core. This is completely fine considering the remote use case of the device. However, there is flexibility for the designer to change the logic behind to improve performance even more.

Table 19 Approximate Data Reception Rate/min of the Prototype IoT System (LNA Disabled)

No.	UART Receive Buffer Size (bytes)	Antenna	Date	Start Time	End Time	Time Difference (min)	Message Received	Approx. Data Rate/Min
1	512	Extendable SMA	15.09.25	16:11:51	16:14:29	2,63	100	38
2	512	Extendable SMA	15.09.25	16:16:40	16:37:12	20,53	794	39
3	512	Extendable SMA	15.09.25	16:40:00	17:11:05	31,08	968	31
4	512	Extendable SMA	15.09.25	15:40:57	15:41:07	0,17	10	60
5	512	Extendable SMA	26.10.25	21:14:23	21:14:31	0.13	20	150
6	1028	Extendable SMA	26.10.25	21:53:55	21:54:14	0.32	42	133
7	1536	Extendable SMA	26.10.25	22:00:49	22:01:19	0.50	62	124
8	1536	Whip	27.10.25	14:45:44	14:46:13	0.48	60	125
9	1536	Extendable SMA	27.10.25	14:28:39	14:33:40	5.02	156	31
10	1536	Extendable SMA	27.10.25	14:59:38	15:08:06	8.47	189	23
11	1536 (Time Slice reduced from 10 to 5 in AIS Thread)	Extendable SMA	27.10.25	15:38:40	15:38:40	4.82	188	39

9 Challenges and Solutions

One the biggest challenges in developing a low-powered IoT system is measuring its current at different power states. A multimeter or current meter has an internal shunt resistor across which the voltage is measured. This introduces a burden voltage that cannot be avoided. Since the voltage drop is directly proportional to the value of the shunt resistor, it affects both the current and the supply voltage delivered to the DUT, potentially causing the MCU to enter a brownout state. One solution is to understand the expected current range and use a suitable resistor.

Finding the right network provider platform for stable and low-powered connectivity supporting worldwide coverage is also a difficult task at hand. There are network providers in the market covering important locations around the world and still expanding their coverage. However, there are tradeoffs that need to be considered as flexibility comes with cost. Therefore, depending on the location of the IoT system, a suitable provider needs to be selected which supports the required network technology. Ideally it would be best to get a single provider allowing the AIS tracker to be deployed anywhere around the world without having to consider network technologies used and its coverage.

Moreover, finding the right source of green energy and designing the harvesting system accordingly is also crucial and challenging for a remote IoT device intended to be deployed out of the box. Thus, decisions are required from the company's end on choosing the energy source based on geographical location and designing the system with the ability to handle current peaks.

To achieve the maximum power efficiency, it would be best if the MCU or the receiver could be put to sleep state. However, this involves the risk of data loss. This challenge cannot be solved considering the importance of AIS to be real time in VTS.

AWS offers various features to support and maintain IoT devices. Understanding the need and implementing the SDK on a resource-constrained device can be critical. Best approach is to understand the need and go through the documentation provided by AWS.

10 Conclusion & Future Work

The thesis presents a guideline for designing and developing an energy-efficient, off-grid AIS tracker with worldwide remote functionality. Using the Renesas PK-S5D9 microcontroller and dAISy Featherwing receiver, a compact, low-powered prototype was presented, which is capable of reliably acquiring and transmitting vessel data while optimizing power consumption for long term deployment in resource constrained environments. The goal of the prototype was to give quantified idea on the energy consumption for a low-powered AIS tracking system.

A detailed system-level design approach was proposed, integrating hardware and software optimizations to minimize energy usage. Hardware selection focused on low-power components and efficient interfaces, while the software architecture was implemented using the ThreadX RTOS to enable precise task scheduling and event driven parallel task handling, avoiding unnecessary polling to optimize CPU processing time and energy. Logic was implemented to toggle receiver's operating mode which can reduce almost half of its current consumption. Overall, fivefold energy reduction is achieved in the prototype, marking a significant step towards sustainable IoT-based remote maritime tracking solution.

Furthermore, the developed prototype was successfully integrated with AWS IoT Core, enabling a secure and scalable cloud connectivity for real-time vessel data monitoring and analytics. The cloud integration validated the potential of combining energy-efficient embedded design with modern IoT infrastructure for next-generation AIS systems.

Although energy harvesting system was not implemented in the current prototype, a comprehensive power analysis was conducted to understand the peak and average currents, which provides a valuable foundation for designing any energy-harvesting system. Moreover, a detailed, step by step calculation process was mentioned for designing solar harvesting system with a power backup of up to three days in Hamburg using the PK-S5D9 prototype, which awaits implementation and testing on field along with a hazard protection system as a future scope.

The future scope of work also demands an implementation of a cellular modem and its integration with the existing cellular framework along with suitable network technology and provider replacing the ethernet connectivity of the prototype.

Moreover, apart from optimizing the code, PPv2 can also be included in the software architecture to control it from AWS IoT Core with the help of MQTT publish message feature and trigger events or interrupt registers to toggle the clock frequency of the MCU, thus saving more power. OTA features and its implementation is also very important for a successful system in production.

Furthermore, the prototype needs to be developed to a custom single board PCB design for production. It will bring environmental robustness and stability by reducing the size, weight and system loss using the S5D9 chip. Probable EMC issues need addressing and testing during this design phase.

11 Appendix

11.1 Determining Uncertainty of USB Multimeter

Model: Uni-T (UT658DUAL USB Tester)

Measuring the current consumption of an ESP8266 (DUT) by current ranger gives 76.5 mA current and doing the same with the Uni-T USB multimeter gives 70mA (0.07A). The MCU was programmed to run a loop to blink LEDs and go to sleep for few ms.

Current Ranger has a resolution of ~0.1µA (depending on mode). It uses 10m Ohm shunt for measuring mA range, 10 Ohm for measuring µA range and 10k Ohm for measuring nA range. It measures the voltage across the shunt resistor and displays the calculated current across it. via its OLED display. Oscilloscope output can also be measured at the BNC output jack of the device. However, the output provides voltage proportional to the current. It automatically switches between the shunts by sensing the input voltage [122].

On the other hand, USB multimeter (UT658Dual) has a resolution: ~0.01 A (10 mA)

So, there's a discrepancy of:

$\Delta I = 76.5 \text{ mA} - 70 \text{ mA} = 6.5 \text{ mA}$

Percentage Error Current Calculation

$$\begin{aligned}\text{Percentage Error} &= (\Delta I / \text{Reference Current}) \times 100 \\ &= (6.5 \text{ mA} / 76.5 \text{ mA}) \times 100 \\ &\approx 8.5\ \%\end{aligned}$$

11.2 How to Operate the Prototype

To run the prototype, first in AWS IoT Core management console, the Thing needs to be registered and attached to a policy. Once it is done, the console generates private and public key along with client certificate. It also provides the root certificate and endpoint for the device to connect. These credentials need to be saved for later use.

For the hardware setup, the AIS receiver needs to be connected to the PK-S5D9 board for UART communication. Six cables need to be connected from the AIS receiver to the development board. The receiver needs to be connected to an antenna as well for AIS data reception. The pin connection diagram is as follows:

Table 20 Pin Connection Diagram of PK-S5D9 & AIS Receiver

No.	PIN for PK-S5D9	PIN for dAISy Featherwing	Purpose
1	VCC (3.3V)	3V3	Supply Power
2	GND	GND	Grounding
3	P31 (RXD)	TX1	Data Transmission
4	P32 (TXD)	RX1	Data Transmission
5	P61	IO2	Toggling Sleep Mode
6	P62	IO1	Toggling LNA Mode

Two Micro-USB to USB type A/C cable needs to be connected from the port J19 and J5 to the computer for debugging and USB console connection. Additionally, an ethernet cable is also connected for internet connectivity. Port J19 is responsible for both debugging and supply power in the prototype version.

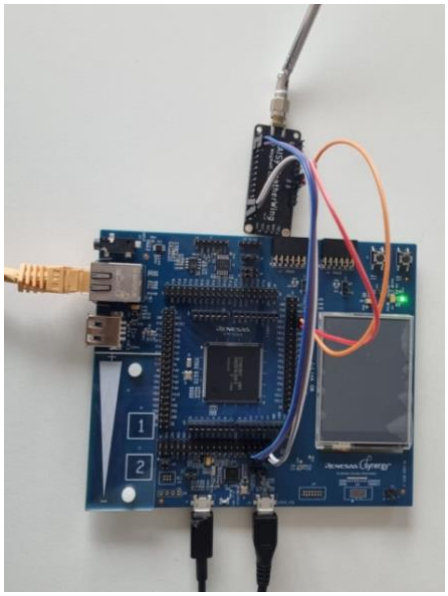

Figure 39 Wire Connection of the Prototype (Illustrated by Author)

As the code is built successfully in E2studio, it is flashed and ran with the help of the on-board J-Link debugger. The computer will detect a new COM port as the USB interface is set. Open any serial terminal software like Tera Term and connect to the COM port. The baud rate needs to be set at 9600, data bits at 8 and no parity with 1 stop bit for the serial terminal.

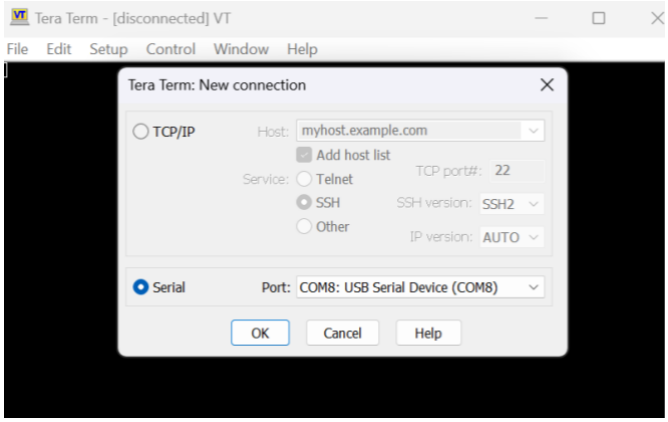

Figure 40 Tera Term COM Port Connection (Illustrated by Author)

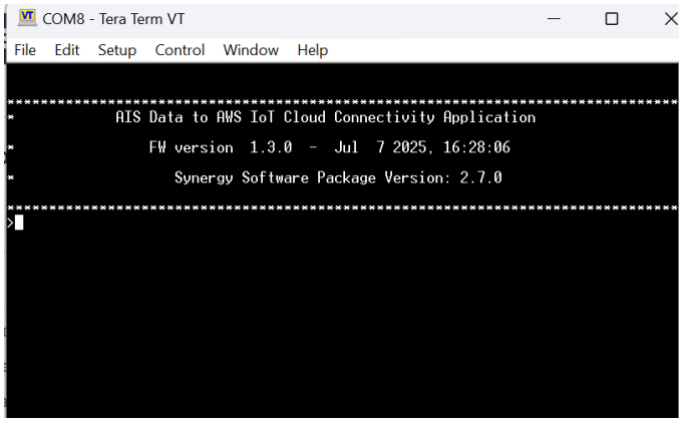

Figure 41 Welcome Page (Illustrated by Author)

After successful connection to the USB serial device, the user needs to press "Enter". This will display the welcome page, as shown in Figure 41, with information related to firmware version, date, SSP version etc.

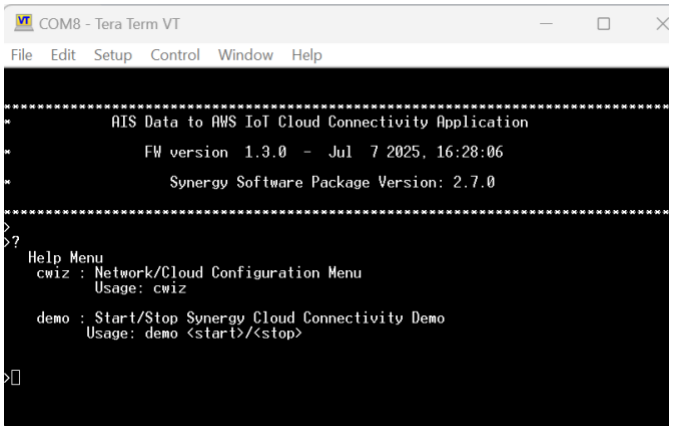

Figure 42 Help Menu (Illustrated by Author)

Figure 42 shows that pressing "?" displays the keywords accepted by the console as user commands. "cwiz" lets the user select the network configuration and interface options. "demo" lets the user star or stop the prototype demonstration.

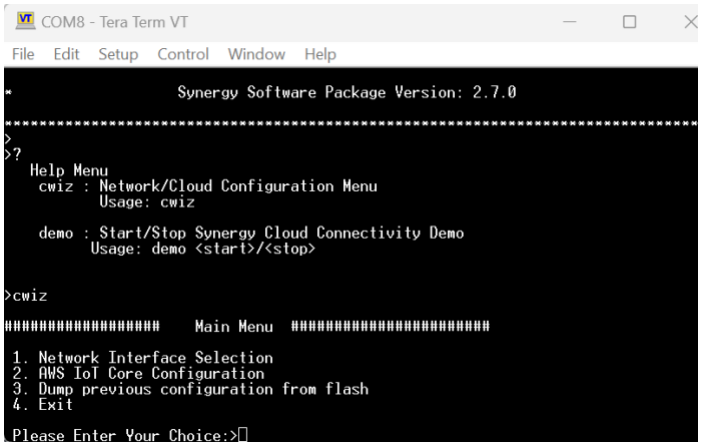

Figure 43 CWIZ Main Menu (Illustrated by Author)

The AWS IoT Core configuration option lets the user to provide the cloud settings like endpoint, rootCA, device private key and client certificate in PEM format.

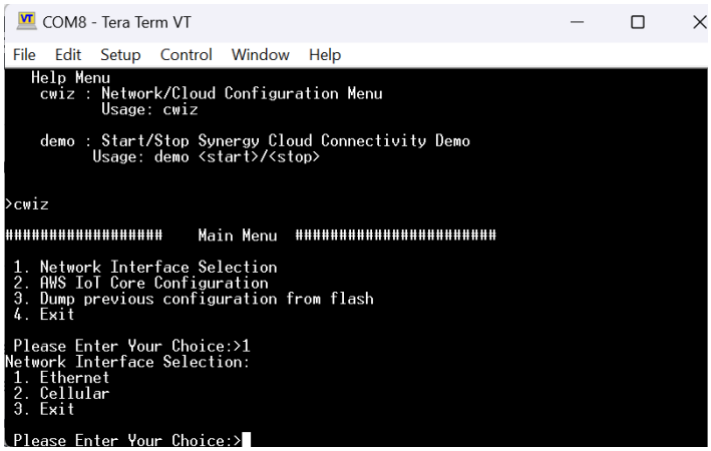

Figure 44 Selecting Network Interface (Illustrated by Author)

Figure 44 shows that from the network interface option, the user can select Ethernet or Cellular. However, the cellular interface has not yet been implemented.

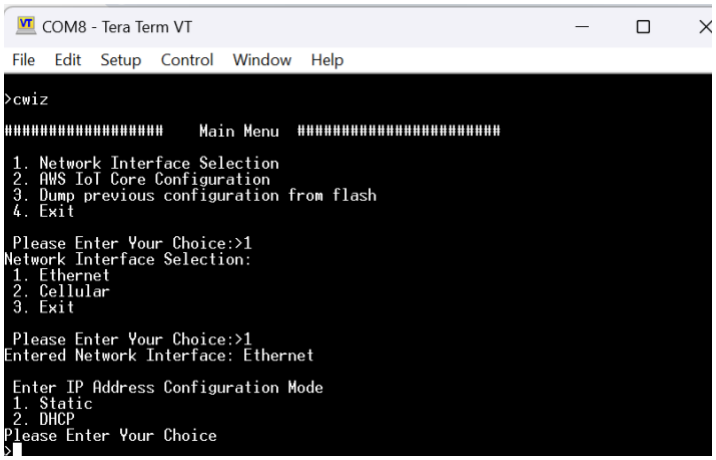

Figure 45 Configuring IP Mode (Illustrated by Author)

Figure 45 shows that the user has the flexibility to choose between static or DHCP connection. Selecting static option will be required to provide IP, gateway, subnet and DNS.

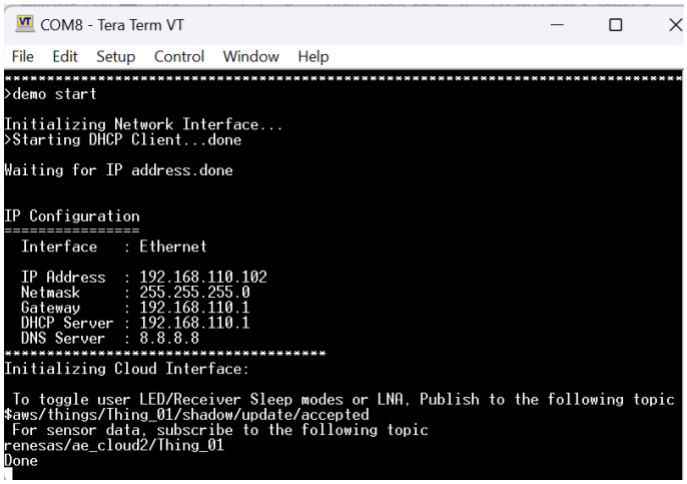

Figure 46 Starting the Demo (Illustrated by Author)

Figure 46 shows that the prototype starts the program as it gets the "demo start" command from the user. This initializes the network interface, gets the IP and displays it in the terminal. Also, it provides information on MQTT topic names for publishing or subscribing. Subscribing to "renesas/ae_cloud2/Thing_01" will give the AIS data in AWS IoT console site.

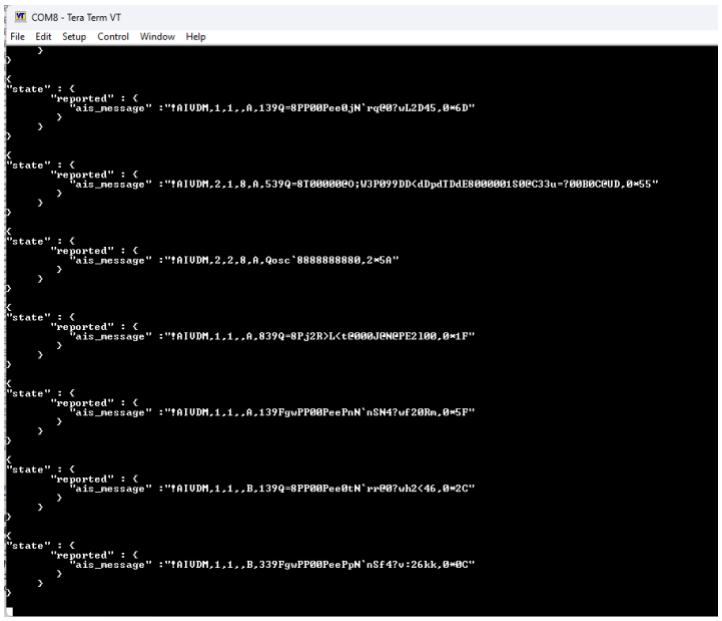

Figure 47 Receiving AIS data in JSON format (Illustration by Author)

The receiver AIS data is displayed in the serial terminal (for debugging) and sent to the AWS IoT Core. Figure 47 and Figure 49 displays the AIS data received by the prototype.

Figure 48 shows a MQTT test client feature of AWS IoT Core to test data reception from the prototype IoT device. The following table shows the options available for publishing message to the topic "$aws/things/Thing_01/shadow/update/accepted"

Table 21 Toggling LNA, Sleep and LED (for debugging) States via publishing MQTT messages

GPIO States	Message
LNA ON	{"state":{"desired":{"Receiver_LNA":"ON"}}}
LNA OFF	{"state":{"desired":{"Receiver_LNA":"OFF"}}}
SLEEP ON	{"state":{"desired":{"Receiver_sleep":"ON"}}}
SLEEP OFF	{"state":{"desired":{"Receiver_sleep":"OFF"}}}
RED LED ON	{"state":{"desired":{"Red_LED":"ON"}}}
RED LED OFF	{"state":{"desired":{"Red_LED":"OFF"}}}

Table 21 shows the expected message from the user by AWS MQTT broker for toggling different GPIO states.

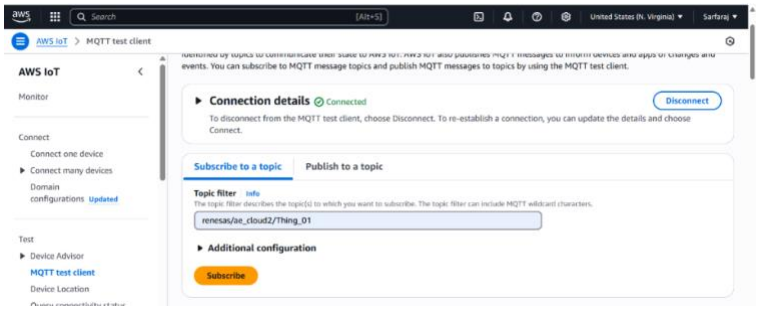

Figure 48 Subscribing to MQTT broker on AWS IoT Core (Illustrated by Author)

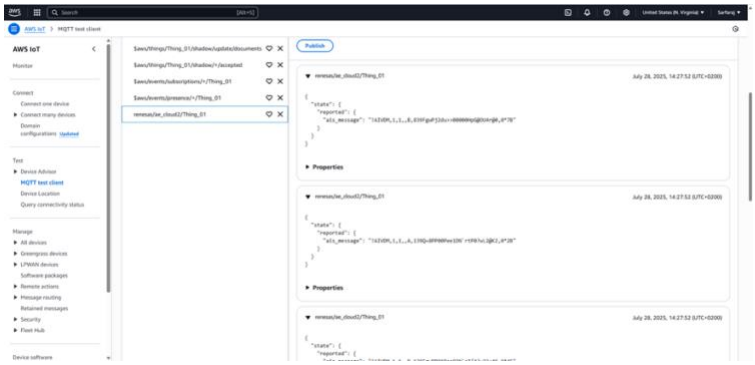

Figure 49 Receiving AIS data in AWS IoT Core (Illustrated by Author)

Figure 48 and Figure 49, shows Subscribing to defined topic and getting AIS data in the MQTT test client on AWS IoT console.

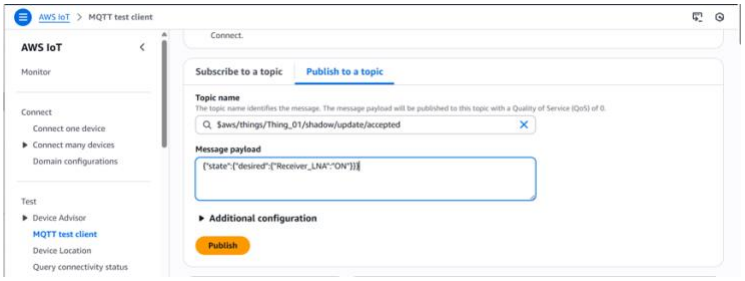

Figure 50 Publishing Message to turn LNA ON (Illustrated by Author)

Figure 51 LNA Status (ON) Reflected in Serial Terminal (Illustrated by Author)

Figure 50 and Figure 51 shows publishing message in the AWS MQTT test client and status reflection of receiver in serial terminal.

11.3 Picoscope Graphs

This section shows some images taken from the Picoscope oscilloscope software during different experiment scenarios. In Figure 52, Figure 53 and Figure 54 only one oscilloscope channel has been used to measure the voltage across the R113 resistor (50m ohm) at J31. The blue signal represents the voltage in millivolt measured across the resistor. A math function is introduced to calculate the current through the resistor, and it is represented by the red signal in mA. Moreover, the measurement feature has been used to display the maximum, minimum peak current along with their mean value with some offsets to display both the signals on the same graph. Table 17 gives all the different value observed at different stages while measuring voltage with this Picoscope.

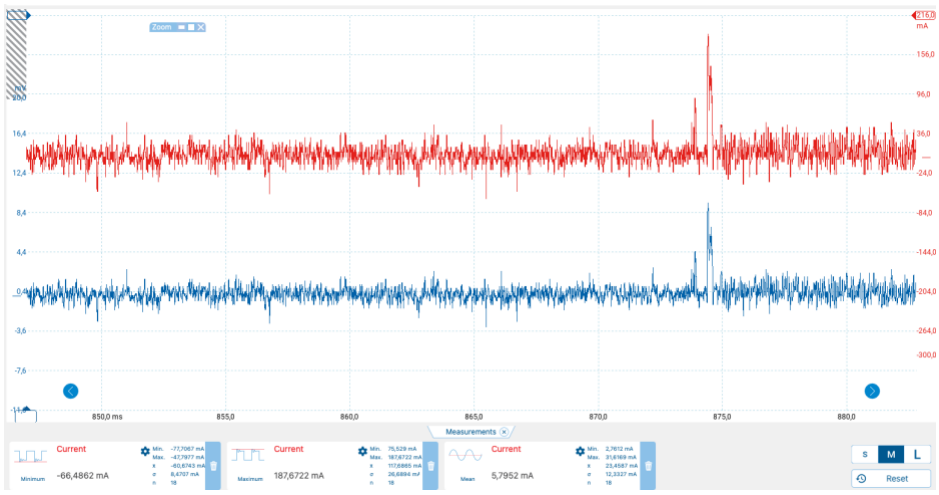

Figure 52 Digital Current Spike Observed during Powering the Device at J31 (Illustrated by Author)

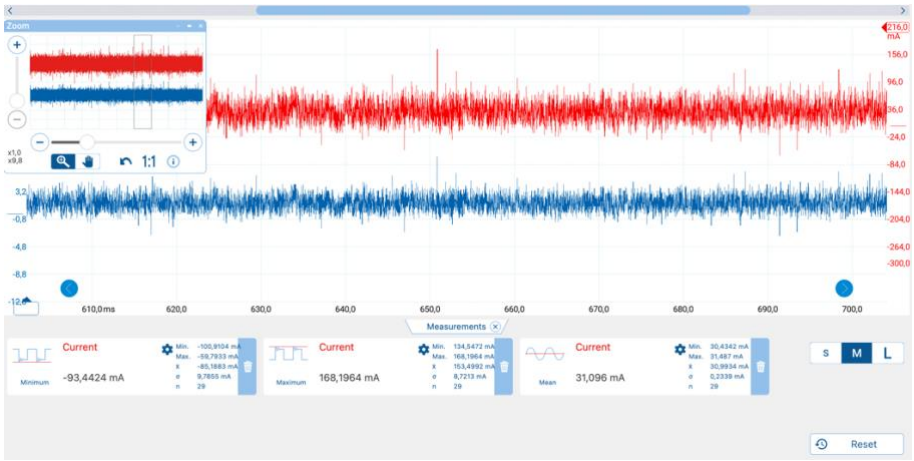

Figure 53 Digital Peak Current Measured During Data Transmission at J31 (Illustrated by Author)

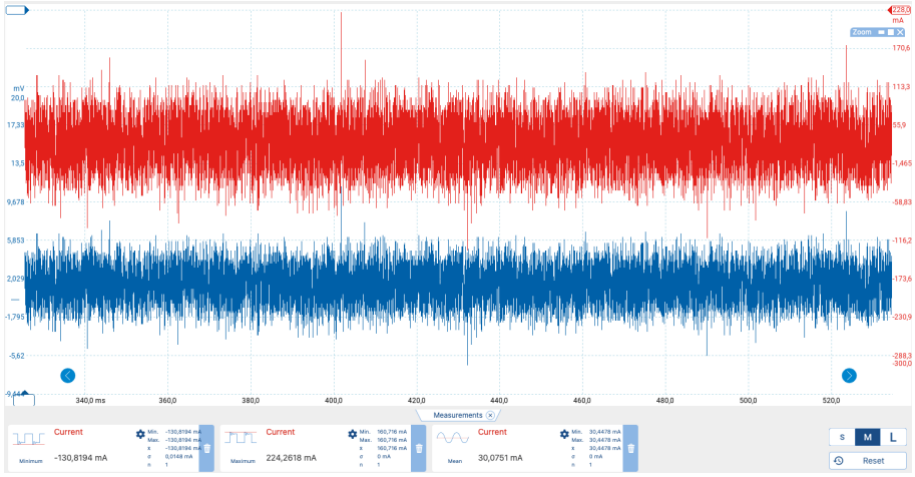

Figure 54 Determining Digital Peak Current with Picoscope at J31 (Illustrated by Author)

To observer the graph during data transmission and to get an idea about the current peaks a 0.01 Ohm resistor is connected in series at the input cable which supplies 5V to the USB debug port of the board PK-S5D9. Voltage across the resistor is measured and the current is calculated.

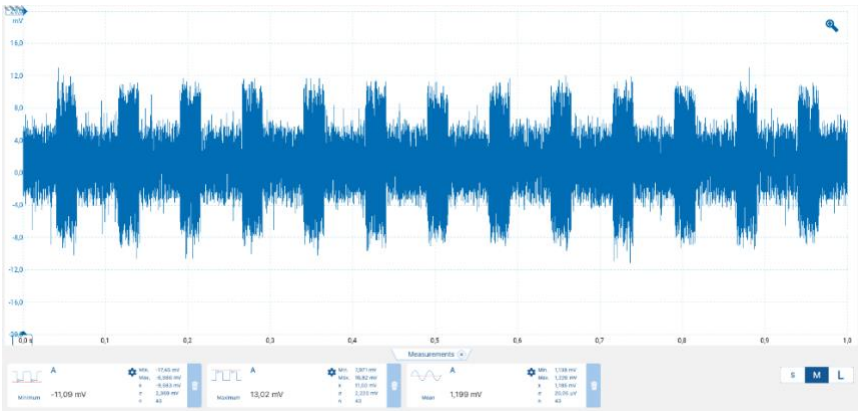

Figure 55 Voltage Measured Across 0.01R at Input during Data Transmission (Illustrated by Author)

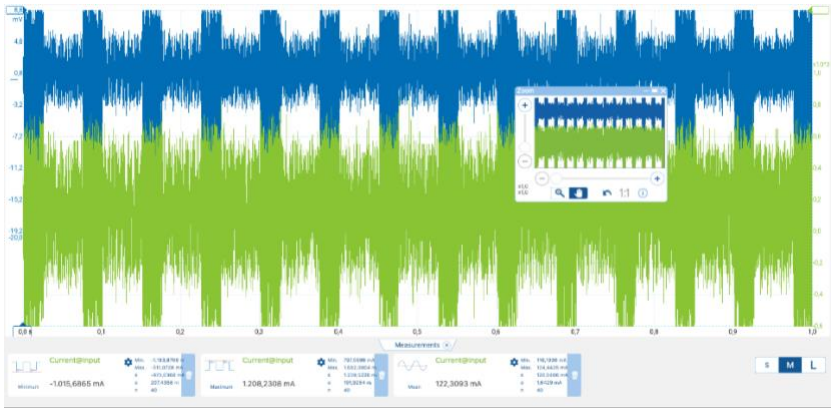

Figure 56 Current Peaks Calculated across 0.01R at Input during Data transmission (Illustrated by Author)

Figure 55 shows the voltage signal observed across the resistor and Figure 56 shows the calculated current across it with the help of math function in the Picoscope software.

12 Bibliography

[1] K. Schwehr, “Vessel Tracking Using the Automatic Identification System (AIS) During Emergency Response: Lessons from the Deepwater Horizon Incident,” 2011, [Online]. Available: chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://vislab-ccom.unh.edu/~schwehr/papers/2011-schwehr-ushydro-dwh.pdf

[2] I. T. U. (ITU)., “Technical characteristics for an automatic identification system using time division multiple access in the VHF maritime mobile frequency band1,” 2014. [Online]. Available: https://www.itu.int/dms_pubrec/itu-r/rec/m/R-REC-M.1371-5-201402-I!!PDF-E.pdf

[3] Touros, Anastassis, “Will sharing AIS data affect my internet connection? | MarineTraffic.” Accessed: Sept. 30, 2025. [Online]. Available: https://support.marinetraffic.com/en/articles/9552975-will-sharing-ais-data-affect-my-internet-connection

[4] NATO SHIPPING CENTRE, “AIS (Automatic Identification System) Overview,” shipping.nato.int. Accessed: Sept. 30, 2025. [Online]. Available: https://shipping.nato.int/nsc/operations/news/2021/ais-automatic-identification-system-overview.aspx

[5] “What is AIS,” VT Explorer. Accessed: Sept. 30, 2025. [Online]. Available: https://www.vtexplorer.com/what-is-ais/

[6] “AIS Message Parsing,” OpenAIS. Accessed: Sept. 30, 2025. [Online]. Available: https://open-ais.org/services/2/

[7] A. Touros, “Comar R400N (SLR350N) | MarineTraffic.” Accessed: Oct. 10, 2025. [Online]. Available: https://support.marinetraffic.com/en/articles/9552938-comar-r400n-slr350n

[8] “MarineTraffic provided AIS Receivers | MarineTraffic.” Accessed: Oct. 10, 2025. [Online]. Available: https://support.marinetraffic.com/en/articles/9552934-marinetraffic-provided-ais-receivers

[9] A. Touros, “Comar SLR450Ni | MarineTraffic.” Accessed: Oct. 11, 2025. [Online]. Available: https://support.marinetraffic.com/en/articles/9552935-comar-slr450ni

[10] A. Touros, “Comar SLR350Ni | MarineTraffic.” Accessed: Oct. 11, 2025. [Online]. Available: https://support.marinetraffic.com/en/articles/9552937-comar-slr350ni

[11] A. Touros, “RSK450Ni | Kpler Help Center.” Accessed: Oct. 11, 2025. [Online]. Available: https://help.kpler.com/en/articles/9706675-rsk450ni

[12] Marine Traffic, “SLR350N User Manual,” 2025. [Online]. Available: https://www.marinetraffic.com/files/SLR350N_User_Manual.pdf

[13] “AIS100PRO (NMEA & USB) - AIS Receiver - Digital Yacht,” Digital Yacht UK. Accessed: Oct. 11, 2025. [Online]. Available: https://digitalyacht.co.uk/product/ais100pro-nmea-usb/

[14] “AIS RECEIVER FA-40 | AIS | Products | FURUNO.” Accessed: Oct. 11, 2025. [Online]. Available: https://www.furuno.com/en/products/ais/FA-40

[15] “ALLTEK MARINE – THE AIS SPECIALISTS | Alltek Marine – the AIS Specialists.” Accessed: Oct. 11, 2025. [Online]. Available: https://www.alltekmarine.com/en-global/products/aisreceiver/WideLink-R150GE

[16] Garmin and G. L. or its subsidiaries, “GARMIN (DE) | AIS 800 | Sende-/Empfangsgerät | Marine | VHF & AIS,” Garmin. Accessed: Oct. 11, 2025. [Online]. Available: https://www.garmin.com/de-DE/p/623924/
[17] “Products - R300,” em-trak. Accessed: Oct. 11, 2025. [Online]. Available: https://em-trak.com/products-r300/
[18] “A21100 raspAIS mobile AIS Receiver | Weatherdock.” Accessed: Oct. 11, 2025. [Online]. Available: https://www.easyais.com/en/products/a21100-raspais/
[19] “dAISy AIS Receiver - The Original,” Wegmatt. Accessed: Oct. 23, 2025. [Online]. Available: https://shop.wegmatt.com/products/daisy-ais-receiver
[20] Saab, “R6 Supreme AIS/VDES,” Start. Accessed: Oct. 11, 2025. [Online]. Available: https://www.saab.com/products/r6-supreme
[21] Kongsberg Discovery, “AIS RX610 Datasheet,” Nov. 2023. [Online]. Available: https://www.kongsberg.com/globalassets/kongsberg-discovery/commerce/surveillance--monitoring/ais-rx610/datasheet_aisrx610.pdf
[22] I. Sultan and M. T. Banday, “Ultra-Low Power Microcontroller Architectures for the Internet of Things (IoT) devices,” in *2023 5th International Conference on Smart Systems and Inventive Technology (ICSSIT)*, Jan. 2023, pp. 482–488. doi: 10.1109/ICSSIT55814.2023.10060949.
[23] F. Samie, L. Bauer, and J. Henkel, “IoT technologies for embedded computing: a survey,” in *Proceedings of the Eleventh IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis*, Pittsburgh Pennsylvania: ACM, Oct. 2016, pp. 1–10. doi: 10.1145/2968456.2974004.
[24] N. Kaur and S. K. Sood, “An Energy-Efficient Architecture for the Internet of Things (IoT),” *IEEE Syst. J.*, vol. 11, no. 2, pp. 796–805, June 2017, doi: 10.1109/JSYST.2015.2469676.
[25] C. Guo, S. Ci, Y. Zhou, and Y. Yang, “A Survey of Energy Consumption Measurement in Embedded Systems,” *IEEE Access*, vol. 9, pp. 60516–60530, 2021, doi: 10.1109/ACCESS.2021.3074070.
[26] D. Rossi, I. Loi, A. Pullini, and L. Benini, “Ultra-Low-Power Digital Architectures for the Internet of Things,” in *Enabling the Internet of Things*, M. Alioto, Ed., Cham: Springer International Publishing, 2017, pp. 69–93. doi: 10.1007/978-3-319-51482-6_3.
[27] S. Jadhav and B. S. Chaudhari, “Embedded systems for low-power applications,” in *TinyML for Edge Intelligence in IoT and LPWAN Networks*, Elsevier, 2024, pp. 13–26. doi: 10.1016/B978-0-44-322202-3.00007-5.
[28] R. M, *Microcontroller Theory and Applications with the PIC18F*, 1st ed. Wiley, 2017. doi: 10.1002/9781119448457.
[29] J. Yiu, “Chapter 9 - Low Power and System Control Features,” in *The Definitive Guide to ARM® CORTEX®-M3 and CORTEX®-M4 Processors (Third Edition)*, J. Yiu, Ed., Oxford: Newnes, 2014, pp. 289–326. doi: 10.1016/B978-0-12-408082-9.00009-9.
[30] M. Alioto, “Near-Threshold Digital Circuits for Nearly-Minimum Energy Processing,” in *Enabling the Internet of Things: From Integrated Circuits to Integrated Systems*, Springer International Publishing, 2017, pp. 95–148. [Online]. Available: https://doi.org/10.1007/978-3-319-51482-6_4
[31] B. Rana and Y. Singh, “Duty-Cycling Techniques in IoT: Energy-Efficiency Perspective,” in *Recent Innovations in Computing*, vol. 832, P. K. Singh, Y. Singh, M.

H. Kolekar, A. K. Kar, and P. J. S. Gonçalves, Eds., in Lecture Notes in Electrical Engineering, vol. 832. , Singapore: Springer Singapore, 2022, pp. 505–512. doi: 10.1007/978-981-16-8248-3_42.

[32] R. H. Randhawa, A. Ahmed, and M. I. Siddiqui, "Power Management Techniques in Popular Operating Systems for IoT Devices," in *2018 International Conference on Frontiers of Information Technology (FIT)*, Islamabad, Pakistan: IEEE, Dec. 2018, pp. 309–314. doi: 10.1109/FIT.2018.00061.

[33] Lethaby, Nick, "RTOS Power Management: Essential for Connected MCU-based IoT Nodes," 2017. [Online]. Available: https://www.ti.com/lit/pdf/spry282

[34] contiki-os, "Add support for the CC13xx CPU · contiki-os/contiki@01e3653," GitHub. Accessed: Oct. 18, 2025. [Online]. Available: https://github.com/contiki-os/contiki/commit/01e36532c2bfd90663a0cd98470c2bb32453fc46

[35] P. Levis *et al.*, "TinyOS: An Operating System for Sensor Networks," in *Ambient Intelligence*, W. Weber, J. M. Rabaey, and E. Aarts, Eds., Berlin/Heidelberg: Springer-Verlag, 2005, pp. 115–148. doi: 10.1007/3-540-27139-2_7.

[36] A. Sabovic, A. K. Sultania, C. Delgado, L. D. Roeck, and J. Famaey, "An Energy-Aware Task Scheduler for Energy-Harvesting Batteryless IoT Devices," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 23097–23114, Nov. 2022, doi: 10.1109/JIOT.2022.3185321.

[37] B. Jahannia, S. A. Ghasemi, and H. Farbeh, "Multi-Retention STT-MRAM Architectures for IoT: Evaluating the Impact of Retention Levels and Memory Mapping Schemes," *IEEE Access*, vol. 12, pp. 26562–26580, 2024, doi: 10.1109/ACCESS.2024.3366074.

[38] M. Kim, J. Lee, Y. Kim, and Y. H. Song, "An analysis of energy consumption under various memory mappings for FRAM-based IoT devices," in *2018 IEEE 4th World Forum on Internet of Things (WF-IoT)*, Singapore: IEEE, Feb. 2018, pp. 574–579. doi: 10.1109/WF-IoT.2018.8355212.

[39] J. P. Kulkarni, J. W. Tschanz, and V. K. De, "Energy Efficient Volatile Memory Circuits for the IoT Era," in *Enabling the Internet of Things*, M. Alioto, Ed., Cham: Springer International Publishing, 2017, pp. 149–170. doi: 10.1007/978-3-319-51482-6_5.

[40] S. Khanna, M. Jung, M. Zwerg, and S. Bartling, "Ferro-Electric RAM Based Microcontrollers: Ultra-Low Power Intelligence for the Internet of Things," in *Enabling the Internet of Things*, M. Alioto, Ed., Cham: Springer International Publishing, 2017, pp. 503–520. doi: 10.1007/978-3-319-51482-6_18.

[41] S. J. Badri, M. Saini, and N. Goel, "Mapi-Pro: An Energy Efficient Memory Mapping Technique for Intermittent Computing," *ACM Trans. Archit. Code Optim.*, vol. 20, no. 4, pp. 1–25, Dec. 2023, doi: 10.1145/3629524.

[42] M. Moradiyan and Y. Sedaghat, "Reliability analysis in ARM big.LITTLE heterogeneous multi-core processor for IoT," in *2024 8th International Conference on Smart Cities, Internet of Things and Applications (SCIoT)*, Mashhad, Iran, Islamic Republic of: IEEE, May 2024, pp. 31–36. doi: 10.1109/SCIoT62588.2024.10570143.

[43] Y. Liu, H. Qu, S. Chen, and X. Feng, "Energy efficient task scheduling for heterogeneous multicore processors in edge computing," *Sci. Rep.*, vol. 15, no. 1, p. 11819, Apr. 2025, doi: 10.1038/s41598-025-92604-6.

[44] GSMA, "Mobile IoT in a 5G Future," Oct. 2024. [Online]. Available: https://www.gsma.com/solutions-and-impact/technologies/internet-of-things/wp-content/uploads/2024/10/Mobile-IoT-in-a-5G-Future-Final.pdf

[45] "Cat 1 vs Cat M vs NB-IoT: Key Differences in Cellular IoT Technologies," Minew. Accessed: Oct. 18, 2025. [Online]. Available: https://www.minew.com/comparison-of-cat-1-cat-m-and-nb-iot/
[46] "Interactive Ultimate IoT Coverage Map -Velos IoT." Accessed: Oct. 18, 2025. [Online]. Available: https://velosiot.com/interactive-ultimate-global-iot-coverage/
[47] "Global LTE-M Connectivity | emnify." Accessed: Oct. 18, 2025. [Online]. Available: https://www.emnify.com/iot-supernetwork/global-iot-coverage/lte-m
[48] "The world's largest LTE-M coverage for IoT | Onomondo." Accessed: Oct. 18, 2025. [Online]. Available: https://onomondo.com/coverage/lte-m/#coverage
[49] "IoT Solutions Provider - IoT Software, Devices, Connectivity | Datablaze." Accessed: Oct. 18, 2025. [Online]. Available: https://datablaze.com/
[50] "Truphone for Things coverage." Accessed: Oct. 18, 2025. [Online]. Available: https://www.truphone.com/things/plans/coverage/
[51] D. B. Ma and Y. Lu, "Power Management Circuit Design for IoT Nodes," in *Enabling the Internet of Things*, M. Alioto, Ed., Cham: Springer International Publishing, 2017, pp. 287–316. doi: 10.1007/978-3-319-51482-6_10.
[52] A. Prasad and P. Chawda, "Power management factors and techniques for IoT design devices," in *2018 19th International Symposium on Quality Electronic Design (ISQED)*, Santa Clara, CA: IEEE, Mar. 2018, pp. 364–369. doi: 10.1109/ISQED.2018.8357314.
[53] S. M. H. Nabavi, S. Hajforoosh, and S. M. A. Nabavi, "Battery Selection for Energy-Efficient IoT Devices: A Comparative Study of Longevity Across Environmental Conditions," *Int. J. Electrochem.*, vol. 2025, no. 1, p. 7783200, Jan. 2025, doi: 10.1155/ijel/7783200.
[54] K. Hasan, N. Tom, and M. R. Yuce, "Navigating Battery Choices in IoT: An Extensive Survey of Technologies and Their Applications," *Batteries*, vol. 9, no. 12, p. 580, Dec. 2023, doi: 10.3390/batteries9120580.
[55] Victoria Atkinson published, "This tiny radioactive battery can last 50 years without recharging — and it's coming in 2025," Live Science. Accessed: Oct. 25, 2025. [Online]. Available: https://www.livescience.com/technology/engineering/betavolt-bv100-radioactive-battery-can-last-50-years-coming-in-2025
[56] K. Gourav, A. M. Wattamwar, Devansh, and R. Rani, "Exploring Future Sustainable Energy Resource - A Review on Diamond Batteries," in *2024 IEEE 11th Power India International Conference (PIICON)*, JAIPUR, India: IEEE, Dec. 2024, pp. 1–6. doi: 10.1109/PIICON63519.2024.10995150.
[57] T. Sanislav, G. D. Mois, S. Zeadally, and S. C. Folea, "Energy Harvesting Techniques for Internet of Things (IoT)," *IEEE Access*, vol. 9, pp. 39530–39549, 2021, doi: 10.1109/ACCESS.2021.3064066.
[58] C. Wang, J. Li, Y. Yang, and F. Ye, "Combining Solar Energy Harvesting with Wireless Charging for Hybrid Wireless Sensor Networks," *IEEE Trans. Mob. Comput.*, vol. 17, no. 3, pp. 560–576, Mar. 2018, doi: 10.1109/TMC.2017.2732979.
[59] P. Kamalinejad, C. Mahapatra, Z. Sheng, S. Mirabbasi, V. C. M. Leung, and Y. L. Guan, "Wireless energy harvesting for the Internet of Things," *IEEE Commun. Mag.*, vol. 53, no. 6, pp. 102–108, June 2015, doi: 10.1109/MCOM.2015.7120024.

[60] A. Ali *et al.*, “Advancements in energy harvesting techniques for sustainable IoT devices,” *Results Eng.*, vol. 26, p. 104820, June 2025, doi: 10.1016/j.rineng.2025.104820.
[61] J. Zhou *et al.*, “A self-powered and self-sensing wave energy harvesting system for the sea-crossing bridge,” *Mater. Today Nano*, vol. 27, p. 100500, Aug. 2024, doi: 10.1016/j.mtnano.2024.100500.
[62] Raspberry Pi Ltd, “Buy a Raspberry Pi 3 Model B+,” Raspberry Pi. Accessed: Sept. 30, 2025. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/
[63] Raspberry Pi Ltd., “Raspberry Pi 3 Model B+ Product Brief,” 2025. [Online]. Available: https://datasheets.raspberrypi.com/rpi3/raspberry-pi-3-b-plus-product-brief.pdf
[64] Raspberry Pi Foundation, “Raspberry Pi 3B+ Mechanical Drawing,” Raspberry Pi 3B+ Mechanical Drawing.
[65] R. Thornton, “Raspberry Pi 3 Model B+ (Reduced Schematics).” [Online]. Available: https://datasheets.raspberrypi.com/rpi3/raspberry-pi-3-b-plus-reduced-schematics.pdf
[66] “Raspberry Pi computer hardware - Raspberry Pi Documentation.” Accessed: Sept. 30, 2025. [Online]. Available: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html
[67] weather dock, “raspAIS Specification Artnr.:A211,” July 2024.
[68] Weather Dock, “OEM AIS Empfänger Board,” High quality by Weatherdock. Accessed: Oct. 11, 2025. [Online]. Available: https://www.easyais.com/produkte/oem-ais-produkte/oem-ais-empfaenger-board
[69] “Soracom Onyx LTE™ USB Dongle | Soracom IoT Hardware,” Soracom. Accessed: Oct. 11, 2025. [Online]. Available: https://soracom.io/store/onyx-usb-modem-sim-connectivity/
[70] “UNI-T UT658 Digital USB Tester Voltmeter Ammeter Voltage Current Meter Capacity Tester Monitor with Backlight (Dual Interface Type A and Type C): Amazon.de: DIY & Tools.” Accessed: Oct. 03, 2025. [Online]. Available: https://www.amazon.de/Voltmeter-Amperemeter-Kapazit%C3%A4t-Hintergrundbeleuchtung-Schnittstelle/dp/B07WJV32B3
[71] Low Power Lab, “CurrentRanger.” Accessed: Oct. 02, 2025. [Online]. Available: https://lowpowerlab.com/shop/product/152
[72] “Set cpu frequency - Raspberry Pi Forums.” Accessed: Oct. 03, 2025. [Online]. Available: https://forums.raspberrypi.com/viewtopic.php?t=382765
[73] Raspberry Pi Forum, “Pico W power consumption - Raspberry Pi Forums.” Accessed: Oct. 23, 2025. [Online]. Available: https://forums.raspberrypi.com/viewtopic.php?t=337145
[74] Jeff Geerling, “Power Consumption Benchmarks | Raspberry Pi Dramble.” Accessed: Oct. 03, 2025. [Online]. Available: https://www.pidramble.com/wiki/benchmarks/power-consumption
[75] Greg, “MQTT versus HTTP for file transfer with AWS IoT Over the Air (OTA) updates,” Amazon Web Services, Inc. Accessed: Oct. 08, 2025. [Online]. Available: https://repost.aws/articles/AR2zEdivvfS1e-3mTRM3VFGg/mqtt-versus-http-for-file-transfer-with-aws-iot-over-the-air-ota-updates

[76] J. Beningo, “How to Select the Right Amount of Flash Memory,” Design News. Accessed: Oct. 08, 2025. [Online]. Available: https://www.designnews.com/embedded-systems/how-to-select-the-right-amount-of-flash-memory
[77] S. Pednekar, “Low Power Iot Market Size, Trends, Growth & Forecast,” Verified Market Research. Accessed: Oct. 08, 2025. [Online]. Available: https://www.verifiedmarketresearch.com/product/low-power-iot-market/
[78] SPOT, “SPOT.” Accessed: Oct. 08, 2025. [Online]. Available: https://ambiq.com/technology/spot/
[79] Wegmatt LLC, “dAISy FeatherWing AIS Receiver,” Wegmatt. Accessed: Oct. 11, 2025. [Online]. Available: https://shop.wegmatt.com/products/daisy-featherwing-ais-receiver
[80] Wegmatt LLC, “dAISy FeatherWing AIS Receiver Manual.” [Online]. Available: https://wegmatt.com/files/dAISy%20FeatherWing%20AIS%20Receiver%20Manual.pdf
[81] Wegmatt LLC, “dAISy Mini AIS Receiver,” Wegmatt. Accessed: Oct. 11, 2025. [Online]. Available: https://shop.wegmatt.com/products/daisy-mini-ais-receiver
[82] Wegmatt LLC, “dAISy Mini AIS Receiver Manual.” [Online]. Available: https://wegmatt.com/files/dAISy%20Mini%20AIS%20Receiver%20Manual.pdf
[83] S. V. B. GmbH, “AMEC CYPHO150S AIS Receiver only 439,95 € buy now | SVB,” SVB GmbH. Accessed: Oct. 11, 2025. [Online]. Available: https://www.svb24.com/en/amec-cypho150s-ais-receiver.html
[84] “AIS100 - AIS Receiver with USB - Digital Yacht,” Digital Yacht European. Accessed: Oct. 11, 2025. [Online]. Available: https://digitalyacht.eu.com/product/ais100-receiver-usb/
[85] “Amazon.com: RTL-SDR Blog V4 R828D RTL2832U 1PPM TCXO SMA Software Defined Radio (Dongle Only) : Electronics.” Accessed: Oct. 11, 2025. [Online]. Available: https://www.amazon.com/RTL-SDR-Blog-RTL2832U-Software-Defined/dp/B0CD745394/ref=sr_1_2?dib=eyJ2IjoiMSJ9.sfbFJPG624K7g9NEHFje-TO-6OnYQHvZUyrVys6c2Ce0wcAB_UzCQ-4ITjfB2xOjW3SUuL6R2Y7vV7jmHaakQKiBLYh2ebBmEGU59WTlG5u_3nIerI7QgukHzKASgeQabZ8l3DAm2WBD4iJg0ajDeDaJ6zQXejRG9jA-dLIyiP42CMZlkYqIscUJcIMZuNUrCB48RLuyEA2ut5kURlYtEg.tIZWkod0AKTOKsDQnzO4LAhRi84Dif2Ok4SR-VU3BaA&dib_tag=se&linkCode=sl2&linkId=5bee61ee3aaab594bf081fe6b0ed62e6&qid=1760205004&sr=8-2&srs=10230687011
[86] RTL-SDR Blog, “RTL-SDR Blog V4 Datasheet.” [Online]. Available: https://www.rtl-sdr.com/wp-content/uploads/2024/12/RTLSDR_V4_Datasheet_V_1_0.pdf
[87] STMicroelectronics, “B-L462E-CELL1 Discovery Cellular Kit Product Overview.” [Online]. Available: https://www.st.com/resource/en/product_presentation/microcontrollers_b-l462e-cell1_discovery_cellular_kit_product_overview.pdf
[88] STMicroelectronics, “B-L462E-CELL1 | Product - STMicroelectronics.” Accessed: Oct. 11, 2025. [Online]. Available: https://www.st.com/en/evaluation-tools/b-l462e-cell1.html
[89] “B-L475E-IOT01A1 STMicroelectronics | Mouser,” Mouser Electronics. Accessed: Oct. 11, 2025. [Online]. Available:

https://www.mouser.de/ProductDetail/STMicroelectronics/B-L475E-IOT01A1?qs=2m8Gdae5Lr3rq3rcxS2xEg%3D%3D
[90] "NUCLEO-L476RG STMicroelectronics | Mouser," Mouser Electronics. Accessed: Oct. 11, 2025. [Online]. Available: https://www.mouser.de/ProductDetail/STMicroelectronics/NUCLEO-L476RG?qs=PRtH0mD6DWbM6mRV5DKjBQ%3D%3D
[91] STMicroelectronics, "NUCLEO-xxxxCx Nucleo-64 Boards Data Brief (NUCLEO-C031C6)," 2025. [Online]. Available: https://www.st.com/resource/en/data_brief/nucleo-c031c6.pdf
[92] Espressif Systems, "ESP32 Series Datasheet," Version 4.7. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
[93] Renesas, "RTK7PKS5D9S00001BU - PK-S5D9 Promotion Kit | Renesas." Accessed: Oct. 11, 2025. [Online]. Available: https://www.renesas.com/en/design-resources/boards-kits/rtk7pks5d9s00001bu
[94] Texas Instruments, "MSP-EXP430FR2355 Development kit | TI.com." Accessed: Oct. 11, 2025. [Online]. Available: https://www.ti.com/tool/MSP-EXP430FR2355#tech-docs
[95] Texas Instruments, "LP-MSPM0L1306 Evaluation board | TI.com." Accessed: Oct. 11, 2025. [Online]. Available: https://www.ti.com/tool/LP-MSPM0L1306#order-start-development
[96] Renesas Electronics Corporation, "S5D9 Microcontroller Group Datasheet," 2023. [Online]. Available: https://www.renesas.com/en/document/dst/s5d9-microcontroller-group-datasheet?srsltid=
[97] "3 dB VHF Stainless Steel Antenna." Accessed: Oct. 20, 2025. [Online]. Available: https://www.scan-antenna.com/products/vhf23
[98] "tinySA ultra+ Spektrumanalysator." Accessed: Oct. 20, 2025. [Online]. Available: https://eleshop.de/tinysa-ultra-spektrumanalysator.html?gad_source=1&gad_campaignid=21117123111&gbraid=0AAAAACLWodugTBi4jaDjeKqExjQqjgnxc&gclid=EAIaIQobChMIyojcmsOykAMVWRiiAx1eHy7PEAQYASABEgILoPD_BwE
[99] "WY 140-2N WY 155-2N." Accessed: Oct. 20, 2025. [Online]. Available: https://www.sirioantenne.it/en/products/vhf/wy-140-2n-wy-155-2n
[100] "GP 3-E." Accessed: Oct. 20, 2025. [Online]. Available: https://www.sirioantenne.it/en/products/vhf/gp-3-e
[101] "VHF-74 - 4 dB Marine VHF Glass Fibre Antenna." Accessed: Oct. 20, 2025. [Online]. Available: https://www.scan-antenna.com/products/vhf74
[102] Renesas, "Synergy Software Package (SSP) Components | Renesas." Accessed: Oct. 11, 2025. [Online]. Available: https://www.renesas.com/en/products/microcontrollers-microprocessors/renesas-synergy-platform-mcus/ssp-components
[103] Renesas Electronics Corporation, "Synergy MQTT/TLS AWS Cloud Connectivity Solution – Application Project," R11AN0336EU0105, Jan. 2021. [Online]. Available: https://www.renesas.com/en/document/apn/synergy-mqtttls-aws-cloud-connectivity-solution-application-project?r=1261701
[104] Renesas, "Knowledge Base | Renesas Customer Hub." Accessed: Oct. 20, 2025. [Online]. Available: https://en-support.renesas.com/knowledgeBase

[105] Renesas Electronics Corporation, “Communications Framework on NX Module Guide,” R11AN0248EU0100, Aug. 2018. [Online]. Available: https://www.renesas.com/en/document/apn/communications-framework-nx-module-guide-application-project?srsltid=AfmBOoqnQJ8fvb-t_021Nu6CeXOHv2xkQG2SiWg-r0aY9ZZ2zoMH0i-W
[106] Renesas Electronics Corporation, “Cellular Framework Module Guide,” Application Note R11AN0253EU0100, Nov. 2018. [Online]. Available: https://www.renesas.com/en/document/apn/cellular-framework-module-guide-application-project?srsltid=AfmBOooPSqluiuBXvjVvtd5YH4AB4kROOSng9wUROF2ZC8Vcc7gHhMdy
[107] Renesas Electronics Corporation, “Promotion Kit S5D9 (PK-S5D9) User’s Manual,” R12UM0009EU0101, Apr. 2017. [Online]. Available: https://www.mouser.com/datasheet/2/698/r12um0009eu0101-synergy-pk-s5d9-1280945.pdf
[108] Renesas Electronics Corporation, “Renesas Synergy™ Promotion Kit PK-S5D9 v1.0 Design Data – MCU Main Signals (Sheet 2 of 16),” Nov. 2016. [Online]. Available: https://www.renesas.com/en/design-resources/boards-kits/rtk7pks5d9s00001bu?srsltid=AfmBOorVhwZhuBtU_HI7UBPG8M8PlWQrynk8KQ62fyrkuNk7dd9JcK_e#design_development
[109] STMicroelectronics, “STM32 microcontroller GPIO hardware settings and low-power consumption,” AN4899, Mar. 2022. [Online]. Available: https://www.st.com/resource/en/application_note/an4899-stm32-microcontroller-gpio-hardware-settings-and-lowpower-consumption-stmicroelectronics.pdf
[110] Renesas Electronics Corporation, “S5D9 Microcontroller Group Datasheet,” Datasheet R01DS0303EU0140, Oct. 2023. [Online]. Available: https://www.renesas.com/en/document/dst/s5d9-microcontroller-group-datasheet
[111] Renesas Electronics Corporation, “Renesas Synergy™ Promotion Kit PK-S5D9 v1.0 Design Data – Power Supplies (Sheet 16 of 16),” Nov. 2016. [Online]. Available: https://www.renesas.com/en/design-resources/boards-kits/rtk7pks5d9s00001bu?srsltid=AfmBOorVhwZhuBtU_HI7UBPG8M8PlWQrynk8KQ62fyrkuNk7dd9JcK_e#design_development
[112] Renesas Electronics Corporation, “Renesas Synergy™ Promotion Kit PK-S5D9 v1.0 Design Data – Ethernet Interface (Sheet 8 of 16),” Nov. 2016. [Online]. Available: https://www.renesas.com/en/design-resources/boards-kits/rtk7pks5d9s00001bu?srsltid=AfmBOorVhwZhuBtU_HI7UBPG8M8PlWQrynk8KQ62fyrkuNk7dd9JcK_e#design_development
[113] Microchip Technology Inc. (Micrel), “KSZ8081MNX/RNB Data Sheet,” Aug. 2015. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/00002202C.pdf
[114] Renesas Electronics Corporation, “S5D9 Microcontroller Group User’s Manual: Renesas Synergy™ Platform, Synergy Microcontrollers S5 Series | Chapter 11 Low Power Modes,” Oct. 2023. [Online]. Available: https://www.renesas.com/ja/document/mah/s5d9-microcontroller-group-users-manual?srsltid=AfmBOorZ7rabb2jhcfcBEDVLN0L1LY--fSzqKkWc8N82Yjki9rjf0I6F

[115] Renesas Electronics Corporation, “Renesas Synergy™ Platform Power Profiles v2 (PPv2) Framework for PK-S5D9,” R20AN0474EU0103, Mar. 2018. [Online]. Available: https://www.renesas.com/en/document/apn/power-profiles-v2-ppv2-framework-pk-s5d9-application-project?r=1261701
[116] Workshop Studio, “AWS IoT Device Client Workshop (IoT quickstart).” Accessed: Oct. 19, 2025. [Online]. Available: https://catalog.us-east-1.prod.workshops.aws/workshops/6d30487a-48e1-4631-b6bc-5602582800b5/en-US/chapter1-introduction
[117] AWS, “AWS IoT Core: Developer Guide | Connect devices to AWS IoT - AWS IoT Core,” 2025. Accessed: Oct. 19, 2025. [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/iot-connect-devices.html
[118] “TLS overhead - netsekure rng.” Accessed: Oct. 19, 2025. [Online]. Available: http://netsekure.org/2010/03/tls-overhead/
[119] Meilhaus Electronic GmbH, “PicoScope 2000 Serie USB PC-Mixed-Signal-Oszilloskop, bis 100MHz.” Accessed: Oct. 24, 2025. [Online]. Available: https://www.meilhaus.de/picoscope-2000mso.htm
[120] “Sunrise and sunset times in Hamburg, January 2025.” Accessed: Nov. 01, 2025. [Online]. Available: https://www.timeanddate.com/sun/germany/hamburg?month=1&year=2025
[121] “NASA POWER Homepage,” Prediction Of Worldwide Energy Resources (POWER). Accessed: Nov. 01, 2025. [Online]. Available: https://power.larc.nasa.gov/
[122] Felix, “Current Ranger | LowPowerLab.” Accessed: Oct. 26, 2025. [Online]. Available: https://lowpowerlab.com/guide/currentranger/